Содержание

1 Определение и методология машинного обучения (ML) 7
1.1 На каких дисциплинах основано . . . 7
1.2 Разница между классическим программированием и ML . . . 7
1.3 Базовые термины ML . . . 8

2 Приложения ML (подобласти) 9
2.1 Назвать минимум 3 и привести практические примеры . . . 9

3 Датасеты 11
3.1 Способы находить датасеты (минимум 3 источника) . . . 11

4 Разведочный анализ данных (EDA) 12
4.1 Зачем нужен EDA для моделирования, примеры . . . 12
4.2 Типичные шаги EDA . . . 12

5 Оценка максимального правдоподобия (MLE) 14
5.1 Определение правдоподобия . . . 14
5.2 Следствия свойства i.i.d. . . . 14
5.3 Формула отрицательного лог-правдоподобия . . . 14

6 Таксономия задач ML 15
6.1 Обучение с учителем и без учителя, примеры . . . 15
6.2 Назвать другие категории . . . 15
6.3 Практические примеры (минимум 3 для каждого типа) . . . 15
6.4 Постановка практической задачи, основные сущности . . . 16

7 Наивный байесовский классификатор 18
7.1 Формула Байеса . . . 18
7.2 Основные предположения . . . 18
7.3 Случай непрерывных признаков . . . 18

8 Библиотека sklearn 20
8.1 Базовые классы, их функции и использование . . . 20
8.2 Важность пайплайнов, пример . . . 21

9 Регрессионная модель 22
9.1 Предположения . . . 22
9.2 * Почему это важно . . . 22

10 Линейная регрессия 24
10.1 Мотивация функции потерь MSE . . . 24
10.1.1 Доказательство для гауссовского случая . . . 24
10.2 Аналитическое решение для весов (матричная форма) . . . 24
10.2.1 Доказательство . . . 25
10.3 Теорема Гаусса–Маркова, BLUE . . . 25
10.3.1 * Доказательство . . . 25

11 Градиентный спуск для линейной регрессии 27
11.1 Почему не используют матричное решение . . . 27
11.2 Стохастический градиентный спуск, преимущества . . . 27

12 Регуляризация в линейных моделях 29
12.1 Мотивация регуляризации . . . 29
12.2 Гребневая регрессия Ridge (L2) . . . 29
12.2.1 Формула решения . . . 29
12.3 Lasso-регуляризация (L1) . . . 29
12.4 Вероятностная интерпретация . . . 30
12.5 Elastic Net . . . 30

12.6 Свойства регуляризаций . . . 30

13 Метрики регрессии MSE, MAE, RMSE, MAPE, SMAPE, R2 32
13.1 Когда их используют . . . 32

14 Параметры и гиперпараметры 34
14.1 Определения, свойства, использование, примеры . . . 34

15 Валидация и оценка модели 36
15.1 Недообучение и переобучение, определения и способы обнаружения . . . 36
15.2 Способы разбиения датасета . . . 37
15.2.1 Что такое хорошее разбиение, идеальный и практический варианты . . . 37
15.2.2 4 основных типа разбиений . . . 37
15.3 Этапы построения модели . . . 37
15.3.1 Train, validation, test, мотивация каждого . . . 37
15.3.2 Кросс-валидация . . . 38
15.3.3 Как выбрать организацию этапов . . . 38
15.4 Утечки данных . . . 39

16 Линейная классификация 40
16.1 Постановка задачи классификации . . . 40
16.2 Варианты типов меток . . . 40
16.3 Определение линейного классификатора и геометрическая интерпретация . . . 40
16.4 Маржа, определение и ключевое свойство . . . 41
16.5 Выбор весов . . . 41
16.5.1 Почему для классификации много разных функций потерь . . . 41

17 Логистическая регрессия 43
17.1 Базовое предположение и геометрическая интерпретация . . . 43
17.2 Сигмоида как предсказание вероятности, свойства . . . 43
17.3 Формула логистической функции потерь . . . 43
17.3.1 вывод через MLE . . . 44
17.4 Свойство калибровки вероятностей . . . 44
17.5 Обобщённая линейная модель . . . 44

18 Стратегии многоклассовой классификации 45
18.1 One-vs-one, one-vs-all и их свойства . . . 45

19 Метрики классификации 47
19.1 Определение метрики в ML . . . 47
19.1.1 Свойства, сравнение с функцией потерь . . . 47
19.2 Матрица ошибок (бинарная, многоклассовая) . . . 47
19.2.1 Типы ошибок . . . 47
19.2.2 Правило присвоения меток . . . 47
19.3 Accuracy, balanced accuracy, precision, recall, F1-score . . . 48
19.3.1 Sensitivity, specificity и другие . . . 48
19.3.2 Несбалансированная стоимость ошибок, примеры, как исправлять дисбаланс 48
19.4 ROC AUC . . . 49
19.4.1 Свойства ROC и ROC-AUC . . . 49
19.4.2 PR AUC . . . 49
19.5 Многоклассовые расширения . . . 49
19.5.1 Микро- и макро-усреднение . . . 49

20 Матричные нормы 51
20.1 p-нормы, самые важные частные случаи . . . 51

21 Метод опорных векторов (SVM) 53
21.1 Геометрическая интерпретация . . . 53
21.2 Оптимизационная задача для линейно разделимого случая . . . 53
21.2.1 * Двойственная задача . . . 53
21.3 Неразделимый случай, slack-переменные . . . 54
21.4 Hinge loss . . . 54

21.5 Ядровой трюк, свойства ядер, примеры ядер . . . 54

22 Метод главных компонент (PCA) 56
22.1 Связь с SVD . . . 56
22.2 Теорема Эккарта–Юнга, доказательство . . . 56
22.2.1 способ через собственное разложение или через подстановку . . . 56
22.3 Две альтернативные итеративные формулировки . . . 57
22.4 Предобработка для PCA . . . 58

23 k ближайших соседей (kNN) для классификации и регрессии 59
23.1 Параметры и гиперпараметры . . . 59

24 Индексы для kNN 61
24.1 Зачем нужны индексы . . . 61
24.2 Идеи индексов для kNN от простых к сложным . . . 61
24.3 k-dimensional tree (kd-tree) . . . 62
24.4 Annoy . . . 62
24.5 HNSW . . . 63
24.5.1 Построение графа . . . 63
24.5.2 Использование графа . . . 64
24.6 Примеры векторных баз данных . . . 64

25 The bitter lesson (Горький урок) 65
25.1 Основные выводы статьи . . . 65

26 Деревья решений 67
26.1 Определение жадных алгоритмов . . . 67
26.2 Процедура построения . . . 67
26.3 Информационные критерии, энтропия, индекс Джини . . . 67
26.4 Ответ в листе . . . 68
26.5 Обработка пропусков . . . 68
26.6 Бинаризация . . . 69
26.7 Обрезка (pruning), варианты C4.5 и CART . . . 69
26.8 Гиперпараметры (минимум 4 основных) . . . 69

27 Разложение bias–variance–noise 71
27.1 Случайность в ошибке . . . 71
27.1.1 Интуиция каждой компоненты . . . 71
27.2 Доказательство для MSE . . . 71
27.3 Компромисс bias–variance . . . 72
27.3.1 Анализ основных моделей . . . 72
27.4 За пределами MSE . . . 73
27.5 Double descent . . . 73

28 Бэггинг 74
28.1 Мотивация через bias–variance . . . 74
28.2 Бутстрэп . . . 74
28.3 Метод случайных подпространств . . . 75
28.4 Random Forest . . . 75

29 Задача обнаружения аномалий 77
29.1 Isolation Forest . . . 77
29.2 One-Class SVM . . . 78

30 Градиентный бустинг (GB) 80
30.1 Процедура построения для MSE . . . 80
30.2 Бустинг как градиентный спуск . . . 81
30.2.1 вывод для кросс-энтропии . . . 81
30.3 Выбор базовой модели . . . 82
30.4 Параллелизм . . . 82

31 Библиотеки градиентного бустинга **84**
31.1 XGBoost, LightGBM, CatBoost . . . 84
31.2 Плюсы и минусы каждой . . . 86

32 Интерпретируемое ML **87**
32.1 Таксономия по свойствам . . . 87
32.2 Линейные модели, важности в деревьях . . . 88
32.3 LIME . . . 88
32.4 Подход SHAP (Shapley additive explanations) . . . 89
32.4.1 Аксиомы, формула . . . 89
32.4.2 SHAP-значения для ML . . . 89
32.4.3 KernelSHAP, TreeSHAP . . . 90
32.5 GradCAM . . . 90

33 Оптимизация гиперпараметров (HP) **92**
33.1 Классы задач оптимизации . . . 92
33.2 Основные подходы к оптимизации HP . . . 93
33.3 Библиотеки для оптимизации HP . . . 94

34 Матричное исчисление и производные матричных операций **95**
34.1 Производная скалярного произведения . . . 95
34.1.1 Для двух векторов . . . 95
34.1.2 Для матрицы и вектора . . . 95

35 Концепция нейросети **96**
35.1 Мотивация нейросетей . . . 96
35.2 Задача XOR . . . 96
35.3 Определения слоёв . . . 97
35.3.1 Графическая нотация . . . 97
35.4 Определение эмбеддинга . . . 98

36 Линейный слой (Linear layer) **99**
36.1 Все названия линейного слоя . . . 99
36.2 Формула . . . 99
36.3 Гиперпараметры, число параметров . . . 99
36.4 Инициализация . . . 100
36.5 Теорема универсальной аппроксимации . . . 100

37 Функции активации **102**
37.1 Примеры (минимум 4) . . . 102
37.2 Основные свойства (сравнительный анализ) . . . 102

38 Обратное распространение ошибки (backpropagation) **105**
38.1 Правило цепочки . . . 105
38.2 Посчитать простой пример вручную . . . 105

39 Многоклассовая классификация в нейросетях **107**
39.1 Softmax . . . 107
39.1.1 Основные свойства . . . 107
39.1.2 Численная стабильность . . . 107
39.2 Потеря кросс-энтропии . . . 108
39.2.1 Определения информационного содержания и энтропии . . . 108
39.2.2 Мотивация и смысл кросс-энтропии . . . 108
39.2.3 Perplexity . . . 108
39.2.4 Дивергенция Кульбака–Лейблера . . . 109

40 Компьютерное железо для ML **110**
40.1 Архитектура фон Неймана . . . 110
40.1.1 Принцип локальности данных . . . 110
40.1.2 Поток данных при обучении модели . . . 111
40.2 Таксономия Флинна . . . 111
40.2.1 Место CPU и GPU . . . 111
40.2.2 CPU vs GPU, основные различия . . . 112

40.3 NVIDIA CUDA, что это . . . 112

41 Библиотека PyTorch **114**
41.1 Зачем нужна ещё одна матричная библиотека . . . 114
41.2 Доступные бекенды . . . 114
41.3 Архитектура, основные модули и классы . . . 115
41.3.1 nn.Module, Sequential, Optimizers, Schedulers, Criterions . . . 115
41.4 Dataflow в PyTorch, классы и их назначение . . . 116

42 Обработка текстов **118**
42.1 Подходы к токенизации (минимум 3) . . . 118
42.2 Лемматизация, стемминг . . . 119
42.2.1 Назвать базовые стеммеры . . . 119

43 Векторизация слов **120**
43.1 One-hot encoding . . . 120
43.1.1 Недостатки . . . 120
43.2 Word2Vec, GloVe, FastText . . . 120
43.2.1 Постановка задачи, функция потерь, признаки . . . 120

44 Векторизация текста **123**
44.1 Bag-of-words . . . 123
44.2 TF-IDF . . . 123
44.3 * BM25 . . . 124

45 Экспоненциальное сглаживание **126**
45.1 Формула . . . 126
45.2 Мотивация названия . . . 126

46 Индуктивные смещения (inductive bias) в Deep Learning **127**
46.1 Регуляризация . . . 127
46.2 Batch normalization, layer normalization . . . 127
46.3 Dropout . . . 128
46.3.1 Отличия режимов обучения и инференса: . . . 128
46.4 Аугментации . . . 128
46.4.1 Основные библиотеки (на практике чаще всего): . . . 129
46.4.2 * Test-time augmentations (TTA) . . . 129

47 Методы оптимизации **130**
47.1 Gradient descent, SGD, Momentum, RMSProp, Adam . . . 130
47.2 Muon . . . 130
47.3 Расход памяти для каждого . . . 131

48 Ранжирование **133**
48.1 Постановка задачи . . . 133
48.1.1 Отличия от регрессии и классификации . . . 133
48.2 Оффлайн-метрики . . . 133
48.2.1 mAP . . . 133
48.2.2 NDCG . . . 134
48.2.3 MRR . . . 135
48.3 Онлайн-метрики . . . 135
48.3.1 Как измеряются . . . 135
48.3.2 Дерево метрик, North star метрика . . . 135
48.4 Функция потерь RankNet . . . 136
48.4.1 Что она предсказывает . . . 136
48.4.2 * Lambda-gradients . . . 136
48.4.3 Другие функции потерь (просто назвать) . . . 137

49 Uplift modelling **138**
49.1 Постановка задачи, отличие look-alike и response моделей 138
49.2 Two-model uplift technique 139
49.3 Class transformation technique 139
49.4 Библиотеки для uplift modelling 140
49.5 Примеры применения 140

1 Определение и методология машинного обучения (ML)

Машинное обучение (Machine Learning, ML) — это подход к построению моделей и алгоритмов, которые улучшают качество решения задачи за счёт *опыта*, получаемого из данных, а не за счёт явного программирования всех правил обработки входов. Важнейшая особенность ML в сравнении с «обычным кодом» — **статистическая природа моделей**: данные рассматриваются как реализации случайных величин, а качество описывается вероятностно-статистическими критериями.

В типичной постановке рассматривается датасет из наблюдений (объектов), для которых задан набор признаков и (в задачах обучения с учителем) известен целевой ответ. Обучение модели сводится к подбору параметров так, чтобы минимизировать ошибку предсказаний на данных. На практике это делается через оптимизацию функции потерь или эквивалентную ей постановку (например, через максимизацию правдоподобия):

$$\hat{\theta} = \arg\max_{\theta}\ p(\mathcal{D} \mid \theta), \qquad \hat{\theta} = \arg\max_{\theta}\ \log p(\mathcal{D} \mid \theta), \qquad \hat{\theta} = \arg\min_{\theta}\ \big(-\log p(\mathcal{D} \mid \theta)\big).$$

При предположении независимости наблюдений (i.i.d.) правдоподобие раскладывается в произве по объектам, а лог-правдоподобие — в сумму:

$$p(\mathcal{D} \mid \theta) = \prod_{i=1}^{n} p(x_i, y_i \mid \theta), \qquad \log p(\mathcal{D} \mid \theta) = \sum_{i=1}^{n} \log p(x_i, y_i \mid \theta).$$

Методологически ML-процесс можно представить как последовательность связанных шагов:

- формализация задачи (что предсказываем/оптимизируем: регрессия, классификация, класт и т.д.);
- формирование датасета и выбор представления данных (матрица признаков, вектор/матри таргетов);
- выбор семейства моделей и гипотез о данных (например, независимость признаков в Naïve Bayes или форма линейной зависимости в линейных моделях);
- выбор функции потерь (и/или вероятностной модели), обучение параметров путём оптимиза
- оценка качества и проверка обобщающей способности на отложенных данных (валидация/те с контролем переобучения.

1.1 На каких дисциплинах основано

Машинное обучение является междисциплинарной областью и опирается на следующие базовые направления.

Теория вероятностей и математическая статистика. Данные рассматриваются как выборк из распределения; используются понятия случайных величин, распределений, условных вероятн ожиданий/дисперсий, а также методы оценивания параметров (например, метод максимального правдоподобия). Предположение i.i.d. часто используется как базовое для вывода формул и корректной оценки качества.

Линейная алгебра. Данные обычно представляют в виде матрицы признаков (design matrix), параметры модели — в виде векторов/матриц, а многие модели являются композициями линейных отображений.

Математическая оптимизация. Обучение параметров — это оптимизационная задача (миним потерь или максимизация правдоподобия), где применяются градиентные и/или неградиентные методы.

Теория алгоритмов и вычислительная математика. Практическая применимость моделей определяется вычислительной сложностью, устойчивостью численных методов и ограничениями вычислительных ресурсов.

1.2 Разница между классическим программированием и ML

Различие между классическим программированием и ML удобно описывать через то, **что является «ядром» решения**.

Классическое программирование: правила задаются явно программистом.

$$\text{Данные (Input)} + \text{Правила (Program)} \longrightarrow \text{Ответ (Output)}.$$

Логика обработки фиксирована и изменяется только при модификации кода.

Машинное обучение: правила не задаются вручную, а **извлекаются из данных** в процессе обучения.

$$\text{Данные (Input)} + \text{Правильные ответы (Targets)} \longrightarrow \text{Модель (Model)},$$
$$\text{Данные (Input)} + \text{Модель (Model)} \longrightarrow \text{Предсказание (Prediction)}.$$

Иными словами, в ML разработчик проектирует *процедуру обучения* и выбирает семейство моделей, а конкретное «правило» задаётся найденными параметрами модели.
Дополнительно важно, что ML-решения по своей природе статистические: качество оценивается метриками/ошибками на выборке, а корректность определяется обобщением на новых данных и контролем переобучения.

1.3 Базовые термины ML

Ниже приведён базовый тезаурус, используемый в курсе.

- **Датасет (dataset)** — совокупность данных для обучения/оценки. В обучении с учителем часто используется представление
$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n},$$
где x_i — признаки объекта, y_i — целевой ответ.

- **Наблюдение / объект / datum (observation)** — один элемент выборки (одна строка в таблице данных). Часто предполагается, что наблюдения i.i.d. (независимы и одинаково распределены).

- **Признак (feature)** — измеряемая характеристика объекта; в табличном виде соответствует столбцу.

- **Матрица признаков / design matrix** — матричное представление всех объектов и их признаков:
$$X \in \mathbb{R}^{n \times d},$$
где n — число объектов, d — число признаков.

- **Целевая переменная / таргет (target)** — то, что требуется предсказывать:
$$y \in \mathbb{R}^n \quad \text{(регрессия)} \qquad \text{или} \qquad y \in \{1, \dots, K\}^n \quad \text{(классификация)}.$$

- **Модель (model)** — параметризованное семейство функций/распределений, задающее правило получения предсказания из признаков, например
$$\hat{y}_i = f(x_i; \theta).$$

- **Предсказание (prediction)** — результат применения модели к входным данным: $\hat{y} = f(X; \theta)$.

- **Функция потерь (loss)** — числовая мера ошибки предсказаний, оптимизируемая при обучении. Примеры для регрессии:
$$\mathrm{MSE}(y, \hat{y}) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2, \qquad \mathrm{MAE}(y, \hat{y}) = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|.$$

- **Параметры (parameters)** — величины, которые *обучаются* на данных (например, веса линейной модели).

- **Гиперпараметры (hyperparameters)** — настройки, задаваемые *до* обучения (например, коэффициент регуляризации, глубина дерева и т.п.).

- **Типы задач ML (таксономия)** — наиболее употребимые классы постановок: *supervised* (с учителем), *unsupervised* (без учителя), *semi-/self-supervised*, *reinforcement* и др.

2 Приложения ML (подобласти)

2.1 Назвать минимум 3 и привести практические примеры

В рамках прикладного машинного обучения принято выделять несколько ключевых подобластей которые отличаются типом данных, постановкой задач и используемыми моделями/архитектура Ниже приведены примеры таких подобластей и типовые прикладные кейсы.

1. **Обработка естественного языка (NLP, Natural Language Processing).**
   Тип данных: тексты, речь (в дискретном или векторном представлении).
   Типовые задачи:
   - машинный перевод;
   - классификация текстов (например, тематика, спам/не спам);
   - генерация текста (диалоговые системы, автодополнение).

   Практические примеры:
   - автоматическая модерация контента и фильтрация спама в сообщениях/почте;
   - чат-бот поддержки (классификация намерений + генерация/выбор ответа);
   - машинный перевод для многоязычного интерфейса продукта.

2. **Компьютерное зрение (Computer Vision).**
   Тип данных: изображения/видео (пиксельные тензоры, признаки).
   Типовые задачи:
   - детекция объектов (поиск объектов и их локализация);
   - семантическая сегментация (разметка классов по пикселям);
   - генеративные модели (синтез/восстановление изображений).

   Практические примеры:
   - контроль качества на производстве: детекция дефектов на конвейере по изображению;
   - медицинская визуализация: сегментация органов/новообразований на снимках;
   - видеоаналитика безопасности: обнаружение людей/техники в запрещённых зонах.

3. **Обучение с подкреплением (RL, Reinforcement Learning).**
   Тип данных: взаимодействие агента со средой (состояния, действия, награды).
   Типовые задачи: оптимизация стратегии поведения, управление в условиях неопределённост последовательное принятие решений.
   Практические примеры:
   - игровые системы, обучающиеся через самообучение и множество эпизодов (классически демонстрации возможностей RL);
   - робототехника и управление: обучение политик управления манипуляторами/дронами в симуляции с переносом в реальную среду;
   - оптимизация управления ресурсами (например, распределение нагрузки/очередей) как задача последовательного контроля.

4. **Машинное обучение на графах (Graph ML).**
   Тип данных: графы (вершины, рёбра, признаки), сети взаимодействий.
   Типовые задачи: выявление сообществ, предсказание связей, ранжирование/рекомендации на основе графовой структуры.
   Практические примеры:
   - рекомендательные системы: рекомендации контента/товаров на основе графа пользоват и объектов;
   - антифрод: поиск аномальных паттернов в графе транзакций и связей;
   - анализ социальных сетей: обнаружение сообществ и инфлюенсеров.

5. **Ранжирование и рекомендательные системы (Learning-to-Rank, RecSys).**
   Тип данных: события взаимодействия пользователь–объект, признаки пользователя/объект
   Типовые задачи: упорядочить объекты по релевантности (поиск, ленты, рекомендации).
   Практические примеры:

   - ранжирование документов в поисковой выдаче по запросу пользователя;
   - ранжирование карточек товаров/видео/новостей в персонализированной ленте;
   - рекомендательные системы, оптимизирующие метрики качества ранжирования (наприм NDCG, MAP и др.).

3 Датасеты

3.1 Способы находить датасеты (минимум 3 источника)

В курсе выделяются несколько практических источников и подходов к поиску датасетов (как для учебных задач, так и для прикладных проектов):

1. **Google Dataset Search.** Специализированный поисковик по датасетам: позволяет находит наборы данных по тематике, смотреть метаданные и переходить к первоисточникам. :contentReference[oaicite:0]index=0

2. **Kaggle Datasets.** Крупнейшая платформа с датасетами и соревнованиями: удобно для быстрого старта, бенчмарков и воспроизводимых baseline-решений. :contentReference[oaicite

3. **Hugging Face Datasets.** Каталог датасетов (особенно популярен в NLP и смежных областях): предоставляет единый интерфейс загрузки и работы со многими наборами данных. :contentReference[oaicite:2]index=2

4. **UCI Machine Learning Repository.** Исторический репозиторий датасетов, широко используемый в обучении и для классических ML-бенчмарков. :contentReference[oaicite:3]inc

5. **Papers with Code (каталоги датасетов и бенчмарков).** Удобен тем, что связывает датасеты с постановками задач, статьями и результатами моделей (как ориентир для выбора датасета под конкретную задачу). :contentReference[oaicite:4]index=4

6. **ML-соревнования и каталоги соревнований.** Источник прикладных датасетов и «живых» постановок задач, часто с чёткими метриками и протоколом оценки. :contentReference[oaicite:5]index=5

4 Разведочный анализ данных (EDA)

4.1 Зачем нужен EDA для моделирования, примеры

Разведочный анализ данных (Exploratory Data Analysis, EDA) — это этап предварительного исследования датасета до построения модели. Его цель — понять структуру данных, качество, распределения признаков и целевой переменной, выявить проблемы (пропуски, выбросы, ошибки утечки), а также сформировать корректные гипотезы о данных и ограничениях будущего моделирования. В материалах курса подчёркивается, что перед обучением любой модели важно проанализировать имеющиеся данные, поскольку поведение модели и итоговое качество напрямую зависят от структуры датасета. :contentReference[oaicite:0]index=0

Зачем EDA нужен для моделирования:

- **Проверка корректности данных и качества датасета.** EDA помогает находить пропуски, дубликаты, аномальные значения, ошибки ввода, несогласованности типов, некорректные единицы измерения. Такие дефекты напрямую ухудшают обучение и могут приводить к некорректным выводам.
- **Понимание распределений и масштаба признаков/таргета.** По распределениям можно оценить асимметрию, тяжёлые хвосты, редкие категории, а также понять, требуется ли преобразование (например, лог-преобразование) или робастные метрики/потери.
- **Выявление выбросов и их природы.** Выбросы могут быть как ошибками измерения, так и редкими, но важными режимами работы системы. Решение (удалять/каппировать/ос влияет на устойчивость модели.
- **Оценка связей между признаками и целевой переменной.** Корреляции, зависимость таргета от ключевых признаков и наличие нелинейностей помогают выбирать модельный класс (линейные модели vs деревья vs нейросети) и формировать направления feature engineering.
- **Предотвращение методологических ошибок.** EDA может выявить *утечки таргета* (target leakage), неверную схему разбиения на train/test, временную зависимость, нарушение i.i.d. и т.п.

Примеры, показывающие необходимость EDA:

- **Квартет Анскомба.** Четыре набора данных могут иметь практически одинаковые просты описательные статистики (среднее, дисперсия, корреляция), но при визуализации оказываю принципиально разными по структуре и форме зависимостей. Это демонстрирует, что без визуализации и EDA «сводные числа» могут вводить в заблуждение. :contentReference[oaicite:1]index=1
- **Пропуски и категориальные признаки.** Если в таблице есть категориальные признаки (например, язык, цвет глаз) и пропуски в них, то без EDA и корректной обработки (кодирование категорий, работа с NA) модель может либо упасть на этапе обучения, либо дать смещённые результаты из-за неправильного представления таких признаков. :contentReference[oaicite:2]index=2

4.2 Типичные шаги EDA

Ниже приведён типовой набор шагов EDA, используемый как практический чек-лист перед моделированием. Он соответствует общей логике EDA из материалов курса: сначала понять состав и качество данных, затем изучить распределения и связи, затем сформировать выводы для подготовки данных и выбора модели. :contentReference[oaicite:3]index=3

1. **Определение цели и единицы наблюдения.**
   Зафиксировать постановку (что предсказываем и для какого объекта/момента времени), определить гранулярность данных (строка = объект/событие/период) и убедиться, что это согласовано с целью моделирования.
2. **Первичный обзор структуры датасета.**
   Проверить размерность, список признаков, типы данных, доли пропусков, уникальность идентификаторов, наличие дубликатов строк/ключей.

3. **Проверка качества данных.**
   Найти пропуски (NA), некорректные значения (например, отрицательные там, где невозмож
   выбросы, несогласованности форматов, ошибки кодировки категорий, разные единицы
   измерения.

4. **Одномерный анализ (univariate).**
   Изучить распределения числовых признаков и таргета (гистограммы/квантили), частоты категориальных признаков, дисбаланс классов (для классификации), редкие категории.

5. **Двумерный анализ (bivariate) и первичные связи.**
   Оценить зависимости признаков от таргета и между собой: корреляции числовых признаков
   группировки по категориям, влияние отдельных факторов на таргет, выявление мультиколл

6. **Анализ выбросов и аномалий.**
   Локализовать выбросы, понять их происхождение (ошибка/редкий режим), определить стратегию обработки (удаление, каппинг, робастные преобразования, отдельная логика).

7. **Анализ целевой переменной и метрик.**
   Для классификации — дисбаланс классов и возможные последствия для выбора метрик; для регрессии — шкала таргета, наличие тяжёлых хвостов и влияние на выбор функции потерь.

8. **Подготовка выводов для моделирования.**
   Сформировать решения: какие признаки трансформировать/нормировать, какие категориа
   признаки кодировать и как, как обрабатывать пропуски, как корректно разбивать данные (случайно или по времени/группам), и какие модели логично пробовать как baseline.

5 Оценка максимального правдоподобия (MLE)

5.1 Определение правдоподобия

Пусть дан датасет наблюдений

$$\mathcal{D} = \{z_i\}_{i=1}^n, \qquad z_i = (x_i, y_i),$$

который считается сгенерированным некоторым вероятностным законом (распределением) с параметром θ.

Правдоподобие (likelihood) параметра θ — это вероятность (или плотность вероятности) получить именно наблюдённые данные $\mathcal{D}$ при условии, что истинный параметр равен θ:

$$\mathcal{L}(\theta \mid \mathcal{D}) \;=\; p(\mathcal{D} \mid \theta).$$

Важно различать: $p(\mathcal{D} \mid \theta)$ — это функция от θ при фиксированных данных $\mathcal{D}$ (то есть *правдоподобие*), а не вероятность θ.

Оценка максимального правдоподобия (MLE) выбирает такие параметры, при которых наблюдённые данные наиболее «вероятны»:

$$\hat{\theta}_{\text{MLE}} \;=\; \arg\max_{\theta} \; p(\mathcal{D} \mid \theta).$$

5.2 Следствия свойства i.i.d.

Часто используется предположение, что наблюдения являются i.i.d. (independent and identically distributed), то есть независимы и одинаково распределены:

$$z_1, z_2, \ldots, z_n \;\; \text{i.i.d.}$$

Тогда вероятность (плотность) всего датасета раскладывается в произведение по объектам:

$$p(\mathcal{D} \mid \theta) \;=\; p(z_1, \ldots, z_n \mid \theta) \;=\; \prod_{i=1}^{n} p(z_i \mid \theta).$$

Далее часто переходят к логарифму правдоподобия (лог-правдоподобию), потому что с суммой работать проще, чем с произведением:

$$\log p(\mathcal{D} \mid \theta) \;=\; \log \prod_{i=1}^{n} p(z_i \mid \theta) \;=\; \sum_{i=1}^{n} \log p(z_i \mid \theta).$$

Логарифм — монотонно возрастающая функция, поэтому точки максимума сохраняются:

$$\arg\max_{\theta} \; p(\mathcal{D} \mid \theta) \;=\; \arg\max_{\theta} \; \log p(\mathcal{D} \mid \theta).$$

5.3 Формула отрицательного лог-правдоподобия

Так как в оптимизации часто минимизируют функцию, задачу MLE переписывают в виде минимизации **отрицательного лог-правдоподобия** (Negative Log-Likelihood, NLL):

$$\hat{\theta}_{\text{MLE}} \;=\; \arg\min_{\theta} \; \Big(-\log p(\mathcal{D} \mid \theta) \Big).$$

С учётом i.i.d.-разложения получаем стандартную форму NLL как сумму по объектам:

$$\text{NLL}(\theta) \;=\; -\log p(\mathcal{D} \mid \theta) \;=\; -\sum_{i=1}^{n} \log p(z_i \mid \theta) \;=\; -\sum_{i=1}^{n} \log p(x_i, y_i \mid \theta).$$

В задачах обучения с учителем NLL используется как функция потерь, по которой обучаются параметры модели.

6 Таксономия задач ML

6.1 Обучение с учителем и без учителя, примеры

Обучение с учителем (supervised learning) — постановка, в которой для каждого объекта датасета заданы **признаки** x_i и **правильный ответ** (таргет) y_i. Датасет имеет вид
$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n, \qquad X \in \mathbb{R}^{n \times d},\ y \in \mathbb{R}^n$ (регрессия) или $y \in \{1, \dots, K\}^n$ (классификация).
Цель — построить модель $f(x; \theta)$, которая по x предсказывает y и хорошо обобщается на новые данные.
Примеры задач обучения с учителем:

- **Регрессия:** предсказание цены/спроса/времени доставки как вещественного числа.
- **Классификация:** спам/не спам, дефект/норма, класс объекта на изображении.
- **Ранжирование (часто рассматривается как supervised):** упорядочивание документо по релевантности (по меткам релевантности или кликам).

Обучение без учителя (unsupervised learning) — постановка, в которой **таргет y отсутств** (либо не используется). Датасет имеет вид
$$\mathcal{D} = \{x_i\}_{i=1}^n, \qquad X \in \mathbb{R}^{n \times d}.$$
Цель — выявить структуру в данных: группы, низкоразмерное представление, аномальные объекты и т.п.
Примеры задач обучения без учителя:

- **Кластеризация:** группировка клиентов/товаров по похожести.
- **Снижение размерности:** построение компактных признаков (например, для визуализаци или ускорения моделей).
- **Поиск аномалий:** выделение объектов, существенно отличающихся от большинства.

6.2 Назвать другие категории

Помимо supervised и unsupervised, в практике встречаются и другие категории постановок:

- **Полуобучение (semi-supervised learning):** часть объектов размечена (есть y), часть — нет; используются оба типа данных.
- **Самообучение (self-supervised learning):** «псевдоразметка» строится из самих данных (например, предсказать скрытую часть входа); часто применяется для предобучения представлений.
- **Обучение с подкреплением (reinforcement learning):** агент взаимодействует со средой, выбирая действия и получая награду; цель — максимизировать суммарную награду.
- **Генеративное обучение (generative learning / generative modeling):** обучение модели распределения данных $p(x)$ или $p(x \mid c)$ для генерации/восстановления/моделирова объектов.

6.3 Практические примеры (минимум 3 для каждого типа)

Ниже приведены типовые прикладные примеры для каждой категории (не менее 3 на категорию)

Обучение с учителем (supervised).

- **Кредитный скоринг:** классификация заявок «дефолт/не дефолт» по признакам клиента и заявки.
- **Прогноз спроса/продаж:** регрессия временного ряда (прогноз количества продаж на день/неделю).
- **Детекция дефектов:** классификация изображений изделий «дефект/норма» или многокл классификация типа дефекта.

Обучение без учителя (unsupervised).

- **Сегментация клиентов:** кластеризация пользователей по поведению (частота покупок, чек, категории).
- **Снижение размерности признаков:** получение компактных эмбеддингов для последую моделей/поиска похожих объектов.
- **Поиск аномалий:** выявление подозрительных транзакций/событий как объектов, «далёки от основной массы.

Полуобучение (semi-supervised).

- **Классификация изображений при дефиците разметки:** небольшая разметка + большое неразмеченное хранилище.
- **Классификация текстов (тематика/тональность):** разметка части документов + дообучение на неразмеченных.
- **Детекция дефектов по сенсорам:** редкие метки «авария/норма» + большой поток неразмеченных телеметрических данных.

Самообучение (self-supervised).

- **Предобучение языковой модели:** предсказание скрытых токенов/следующего токена для получения универсальных текстовых представлений.
- **Предобучение для изображений:** восстановление замаскированных патчей изображени или контрастивное обучение представлений.
- **Предобучение на логах/событиях:** предсказание следующего события/части последова для представления поведения пользователя.

Обучение с подкреплением (reinforcement learning).

- **Управление:** обучение стратегии управления (например, распределение ресурсов/очереде по сигналу награды.
- **Робототехника/манипуляторы (в симуляции):** обучение политике действий с последу переносом.
- **Оптимизация стратегий:** выбор действий в последовательной задаче (долгосрочная цель важнее локальной).

Генеративное обучение (generative).

- **Генерация текста:** построение модели $p(\text{text})$ для генерации/дополнения/перефразирован
- **Генерация изображений/восстановление:** синтез изображений или восстановление/ул качества (denoising, super-resolution).
- **Генерация табличных данных:** моделирование распределения $p(x)$ для симуляции данных и тестирования пайплайнов.

6.4 Постановка практической задачи, основные сущности

Практическая ML-задача обычно задаётся набором ключевых сущностей: **задача/постановка**, **данные**, **модель**, **критерий качества** и **протокол оценки**. Удобно фиксировать их явно.

- **Датасет** $\mathcal{D}$:
$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n} \ \text{(supervised)} \qquad \text{или} \qquad \mathcal{D} = \{x_i\}_{i=1}^{n} \ \text{(unsupervised)}.$$
- **Наблюдение (объект)** x_i — одна строка данных; часто предполагается i.i.d.

- **Признаки (features)**: матрица признаков (design matrix)
$$X \in \mathbb{R}^{n \times d},$$
где n — число объектов, d — число признаков.

- **Таргет (target)** y (если есть): вектор/матрица ответов.

- **Модель** $f(\cdot; \theta)$: правило получения предсказания
$$\hat{y}_i = f(x_i; \theta), \qquad \hat{y} = f(X; \theta).$$

- **Функция потерь (loss)**: оптимизируемая на обучении величина (например, MSE/MAE в регрессии, логистическая потеря в классификации). Потеря определяет, *что именно* минимизирует алгоритм обучения.

- **Метрика (metric)**: величина для оценки качества на валидации/тесте; может отличаться от loss и обычно более интерпретируема.

- **Параметры vs гиперпараметры**: θ обучаются на данных, гиперпараметры задаются до обучения (глубина дерева, коэффициенты регуляризации, число итераций и т.п.).

- **Разбиение данных** (train/val/test): необходимо для оценки обобщающей способности и контроля переобучения.

- **Setup (практическая конфигурация)**: одна и та же модель может решать разные задачи на разных датасетах и с разными метриками; поэтому полезно фиксировать связку:
Problem + Dataset + Model + Metric/Loss.

7 Наивный байесовский классификатор

7.1 Формула Байеса

Байесовский подход рассматривает вероятностные распределения как центральный объект анализа. Базовым инструментом является **формула Байеса**, связывающая апостериорную вероятность класса с априорной вероятностью и правдоподобием наблюдений:

$$p(y \mid x) \;=\; \frac{p(x \mid y)\, p(y)}{p(x)}.$$

В задаче классификации по признаковому вектору x требуется выбрать класс y с максимальной апостериорной вероятностью:

$$\hat{y} \;=\; \arg \max_{y \in \{1,\dots,K\}} p(y \mid x) \;=\; \arg \max_{y \in \{1,\dots,K\}} \frac{p(x \mid y)\, p(y)}{p(x)}.$$

Поскольку знаменатель $p(x)$ не зависит от класса y, при поиске максимума его можно отбросить:

$$\hat{y} \;=\; \arg \max_{y \in \{1,\dots,K\}} p(x \mid y)\, p(y).$$

7.2 Основные предположения

Наивный байесовский классификатор (Naïve Bayes) вводит ключевое **наивное предпол**

$$x = (x_1, \dots, x_d), \qquad p(x \mid y) \;=\; \prod_{j=1}^{d} p(x_j \mid y),$$

то есть признаки $x_1, \dots, x_d$ считаются **условно независимыми** при фиксированном классе y.

Тогда правило классификации принимает вид:

$$\hat{y} \;=\; \arg \max_{y \in \{1,\dots,K\}} \left(p(y) \prod_{j=1}^{d} p(x_j \mid y) \right).$$

На практике для устойчивости вычислений часто переходят к логарифмам (лог-вероятностям), что заменяет произведение на сумму:

$$\hat{y} \;=\; \arg \max_{y \in \{1,\dots,K\}} \left(\log p(y) + \sum_{j=1}^{d} \log p(x_j \mid y) \right).$$

Оценки вероятностей $p(y)$ и условных распределений $p(x_j \mid y)$ получают по обучающей выборке (частотными оценками или параметрическими моделями), после чего применяют правило максимума апостериорной вероятности (MAP в смысле выбора класса).

7.3 Случай непрерывных признаков

Если признаки непрерывные, то вероятности вида $p(x_j \mid y)$ заменяются **плотностями** $p(x_j \mid y)$, и для каждого признака задаётся параметрическое семейство распределений, параметры которого оцениваются по обучающей выборке внутри каждого класса.

Типичные варианты параметризации (используются в стандартных реализациях Naïve Bayes):

- **Gaussian Naïve Bayes:** $x_j \mid (y = c) \sim \mathcal{N}(\mu_{jc}, \sigma_{jc}^2)$,

$$p(x_j \mid y = c) = \frac{1}{\sqrt{2\pi\sigma_{jc}^2}} \exp\left(-\frac{(x_j - \mu_{jc})^2}{2\sigma_{jc}^2} \right).$$

- **Multinomial Naïve Bayes:** для счётчиков/частот (например, bag-of-words в текстах), где x_j — количество событий/слов.

- **Bernoulli Naïve Bayes:** для бинарных признаков $x_j \in \{0, 1\}$.

В общем случае при непрерывных признаках решающее правило остаётся тем же:

$$\hat{y} \;=\; \arg \max_{y \in \{1,\dots,K\}} \left(p(y) \prod_{j=1}^{d} p(x_j \mid y) \right),$$

но величины $p(x_j \mid y)$ интерпретируются как плотности и вычисляются через выбранную параметрическую модель (с параметрами, оценёнными по обучающим данным внутри класса).

8 Библиотека sklearn

8.1 Базовые классы, их функции и использование

scikit-learn (sklearn) — одна из самых крупных библиотек, объединяющих в одном месте множество методов машинного обучения и стандартных инструментов для подготовки данных, обучения, валидации и оценки моделей. В учебных и прототипных задачах sklearn удобен благодаря единообразному API, однако в индустриальном production-окружении часто выбираю специализированные решения (в т.ч. из-за требований к производительности, распределённости, мониторингу и т.п.).

Единый интерфейс API в sklearn построен вокруг нескольких базовых ролей (классов/абстр На практике важнее всего запомнить контракт методов: `fit`, `predict`, `transform` и их сочетания

1. **Estimator (оцениватель / модель).**
   Любой объект sklearn, который можно *обучить*, реализует метод:
   `fit(X, y=None)`
   где X — матрица признаков (обычно `shape = (n\_samples, n\_features)`), y — целевая переменная (опционально для unsupervised). Результат `fit` — обученный объект с сохранённ параметрами (например, `coef_`, `classes_`, `feature_importances_` и др., в зависимости от модели).

2. **Predictor (предиктор): Classifier / Regressor.**
   Модели для задач с учителем (классификация/регрессия) после `fit` используют:
   `predict(X)`
   Классификаторы часто дополнительно предоставляют:
   `predict_proba(X)`, `decision_function(X)`
   что позволяет работать с вероятностями, порогами и метриками (ROC-AUC, PR-AUC и др.).

3. **Transformer (преобразователь признаков).**
   Преобразователь обучается на данных и затем применяет преобразование:
   `fit(X, y=None)`, `transform(X)`
   Часто используется объединённый метод:
   `fit_transform(X, y=None)`
   Примеры типовых трансформеров: масштабирование числовых признаков, кодирование категориальных признаков, снижение размерности и т.п.

4. **Model selection и evaluation (валидация и оценка).**
   В sklearn есть стандартные инструменты для:
   - разбиения данных на train/val/test (например, `train_test_split`);
   - кросс-валидации (например, `cross_val_score`);
   - подбора гиперпараметров (например, `GridSearchCV`, `RandomizedSearchCV`);
   - вычисления метрик качества (accuracy, precision/recall/F-score, ROC-AUC и т.п.).

5. **Композиция моделей: Pipeline / ColumnTransformer.**
   Практика показывает, что подготовку данных и модель нужно объединять в единый объект, чтобы корректно обучать и валидировать весь процесс целиком.

Минимальный шаблон использования sklearn (обучение и предсказание):

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_st
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
# (опционально) y_prob = model.predict_proba(X_test)[:, 1]
```

8.2 Важность пайплайнов, пример

Pipeline (пайплайн) — это способ собрать последовательность шагов вида

$$\text{preprocessing} \rightarrow \text{model}$$

в один объект с единым интерфейсом `fit/predict`. Пайплайны критически важны по следующим причинам:

- **Защита от утечек (data leakage).** Все преобразования (масштабирование, кодирование и т.п.) должны обучаться **только на train** внутри `fit`, а на test применяться как `transform`. Pipeline обеспечивает это автоматически.
- **Корректная кросс-валидация и подбор гиперпараметров.** При CV каждый фолд обучает преобразования на своей train-части; без пайплайна легко «подсмотреть» статистик test-фолда.
- **Воспроизводимость и переносимость.** Один объект хранит всю цепочку преобразовани и модель, снижая риск рассинхронизации шагов между обучением и инференсом.

Пример пайплайна с раздельной обработкой числовых и категориальных признаков (типовая табличная задача):

```
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
num_cols = ["age", "income"]
cat_cols = ["city", "device_type"]
preprocess = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), num_cols),
        ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols),
    ]
)
clf = Pipeline(steps=[
    ("preprocess", preprocess),
    ("model", LogisticRegression(max_iter=1000))
])
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_st
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
y_prob = clf.predict_proba(X_test)[:, 1]
```

В этом примере все статистики масштабирования и кодирования категорий вычисляются внутри `fit` только по обучающей части, а затем корректно применяются к тестовым данным, что обеспечивает честную оценку качества.

9 Регрессионная модель

9.1 Предположения

Регрессионная модель описывает наблюдаемую целевую величину как сумму *систематичес (предсказуемой)* компоненты и *случайной* ошибки:

$$Y = f(X) + \varepsilon,$$

где $f(X)$ — детерминированная часть сигнала (то, что мы хотим предсказывать), а ε — шум (ошибка), отражающий вариативность, не объяснённую моделью.
Часто $f(X)$ интерпретируется как условное математическое ожидание:

$$f(X) = \mathbb{E}[Y \mid X], \qquad \varepsilon = Y - \mathbb{E}[Y \mid X],$$

то есть мы раскладываем наблюдение Y на:

- компоненту с **ожиданием**, которую можно предсказывать ($\mathbb{E}[Y \mid X]$);
- компоненту с **нулевым ожиданием** (шум), которую предсказывать невозможно (ε).

Классические предположения регрессионной модели (underlying assumptions):

1. **Репрезентативность выборки.**
   Наблюдения в датасете должны быть репрезентативны относительно генеральной совокупн на которую планируется распространять выводы модели.
2. **Признаки измерены без ошибки.**
   Независимые переменные (ковариаты) считаются измеренными без ошибок.
3. **Нулевое условное среднее ошибки (экзогенность).**
   Отклонения от модели имеют нулевое ожидаемое значение при фиксированных ковариатах:
   $$\mathbb{E}[\varepsilon_i \mid X_i] = 0.$$
4. **Гомоскедастичность.**
   Дисперсия ошибок постоянна по наблюдениям:
   $$\mathrm{Var}(\varepsilon_i \mid X_i) = \sigma^2 \quad (\text{не зависит от } i).$$
5. **Отсутствие корреляции ошибок (нет автокорреляции).**
   Остатки некоррелированы между собой, то есть ковариационная матрица ошибок диагонал
   $$\mathrm{Cov}(\varepsilon_i, \varepsilon_j) = 0 \ \text{ при } \ i \neq j.$$

9.2 * Почему это важно

Перечисленные предположения важны, потому что они обеспечивают корректные теоретические свойства оценивания и интерпретации регрессионной модели.

- **Корректность оценки зависимости $f(X)$.**
  Условие $\mathbb{E}[\varepsilon_i \mid X_i] = 0$ является ключевым: оно означает, что в ошибке нет систематической составляющей, связанной с признаками. При нарушении (например, пропущенные важные факторы, эндогенность) оценки параметров становятся смещёнными, а предсказания — систематически неверными.
- **Гарантии оптимальности среди линейных несмещённых оценок (Gauss–Markov)**
  При выполнении стандартных условий (в т.ч. гомоскедастичность и некоррелированность ошибок) оценка, получаемая минимизацией MSE в линейной регрессии, обладает свойством BLUE (Best Linear Unbiased Estimator): это несмещённая линейная оценка с минимальной дисперсией среди всех линейных несмещённых оценок.
- **Корректная оценка неопределённости и сравнение моделей.**
  Гомоскедастичность и отсутствие корреляции ошибок лежат в основе корректных формул для дисперсий/стандартных ошибок оценок параметров. Если, например, ошибки гетероске или коррелированы, то наивные оценки дисперсий будут неверны, что приводит к ошибочны выводам о значимости признаков и неверным доверительным интервалам.

- **Качество обобщения на целевую популяцию.**
  Репрезентативность выборки критична для переноса результатов модели: если train-данные систематически отличаются от реальной популяции (смещение выборки), то качество на практике может резко падать даже при отличной метрике на обучении/валидации.

- **Практические последствия нарушений.**
  Ошибка измерения признаков приводит к искажению оценок (вплоть до систематического занижения эффектов), коррелированные ошибки делают модель переуверенной, а неодинак дисперсия ошибок снижает эффективность оценивания и ухудшает интерпретацию.

10 Линейная регрессия

10.1 Мотивация функции потерь MSE

Рассмотрим линейную модель (включая свободный член через добавление единичного признака)

$$\hat{y}_i = x_i^\top w, \qquad x_i \in \mathbb{R}^d,\ w \in \mathbb{R}^d, \qquad i = 1, \dots, n,$$

а также регрессионную постановку с шумом:

$$y_i = x_i^\top w + \varepsilon_i.$$

Функция потерь MSE (Mean Squared Error) задаётся как

$$\mathrm{MSE}(w) = \frac{1}{n}\sum_{i=1}^{n}\big(y_i - x_i^\top w\big)^2 \;=\; \frac{1}{n}\|y - Xw\|_2^2,$$

где $X \in \mathbb{R}^{n\times d}$ — матрица признаков, $y \in \mathbb{R}^n$ — вектор ответов.
Минимизация $\mathrm{MSE}(w)$ эквивалентна минимизации суммы квадратов ошибок (SSE):

$$\arg\min_w \mathrm{MSE}(w) \;=\; \arg\min_w \sum_{i=1}^{n}\big(y_i - x_i^\top w\big)^2,$$

поскольку множитель $1/n$ не влияет на точку минимума.

10.1.1 Доказательство для гауссовского случая

Пусть ошибки являются независимыми и одинаково распределёнными и имеют нормальное распределение:

$$\varepsilon_i \overset{i.i.d.}{\sim} \mathcal{N}(0, \sigma^2).$$

Тогда условное распределение y_i при фиксированном x_i и параметрах (w, σ^2):

$$y_i \mid x_i, w, \sigma^2 \sim \mathcal{N}(x_i^\top w, \sigma^2),$$

а плотность имеет вид

$$p(y_i \mid x_i, w, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}}\exp\left(-\frac{(y_i - x_i^\top w)^2}{2\sigma^2}\right).$$

По i.i.d.-предположению правдоподобие всей выборки:

$$p(y \mid X, w, \sigma^2) = \prod_{i=1}^{n} p(y_i \mid x_i, w, \sigma^2).$$

Рассмотрим лог-правдоподобие:

$$\log p(y \mid X, w, \sigma^2) = \sum_{i=1}^{n}\left[-\frac{1}{2}\log(2\pi\sigma^2) - \frac{(y_i - x_i^\top w)^2}{2\sigma^2}\right].$$

Выделим зависящую от w часть:

$$\log p(y \mid X, w, \sigma^2) = -\frac{n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_{i=1}^{n}(y_i - x_i^\top w)^2.$$

Так как первые слагаемые не зависят от w, то

$$\arg\max_w \log p(y \mid X, w, \sigma^2) = \arg\min_w \sum_{i=1}^{n}(y_i - x_i^\top w)^2 = \arg\min_w \mathrm{MSE}(w).$$

Следовательно, при аддитивном гауссовском шуме MLE для w приводит к минимизации MSE.

10.2 Аналитическое решение для весов (матричная форма)

Введём обозначения:

$$X = \begin{pmatrix} x_1^\top \\ \vdots \\ x_n^\top \end{pmatrix} \in \mathbb{R}^{n\times d}, \qquad y = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix} \in \mathbb{R}^n, \qquad \hat{y} = Xw.$$

Рассмотрим квадратичную функцию потерь (без множителя $1/n$):

$$J(w) = \|y - Xw\|_2^2 = (y - Xw)^\top (y - Xw).$$

При условии, что матрица $X^\top X$ невырождена (например, X имеет полный столбцовый ранг), решение задачи МНК имеет замкнутую форму:

$$\hat{w} = (X^\top X)^{-1} X^\top y.$$

В случае вырожденности используется псевдообратная матрица (решение наименьшей нормы):
$$\hat{w} = X^{+}y.$$

10.2.1 Доказательство

Раскроем $J(w)$:
$$J(w) = (y^\top y) - 2w^\top X^\top y + w^\top X^\top X w.$$
Вычислим градиент по w:
$$\nabla_w J(w) = -2X^\top y + 2X^\top X w.$$
Приравниваем градиент к нулю (условие оптимума для выпуклой квадратичной функции):
$$-2X^\top y + 2X^\top X w = 0 \quad \Longleftrightarrow \quad X^\top X w = X^\top y.$$
Это **нормальные уравнения**. Если $X^\top X$ обратима, то:
$$\hat{w} = (X^\top X)^{-1} X^\top y.$$
Так как $J(w)$ является выпуклой квадратичной функцией, найденная стационарная точка является глобальным минимумом.

10.3 Теорема Гаусса–Маркова, BLUE

Рассмотрим классическую линейную модель:
$$y = X\beta + \varepsilon,$$
где $\beta \in \mathbb{R}^d$ — истинный вектор параметров, а ошибки удовлетворяют условиям:
$$\mathbb{E}[\varepsilon \mid X] = 0, \qquad \mathrm{Var}(\varepsilon \mid X) = \sigma^2 I_n.$$
Предположим также, что X имеет полный столбцовый ранг, так что $(X^\top X)^{-1}$ существует.
Оценка МНК (OLS):
$$\hat{\beta} = (X^\top X)^{-1} X^\top y.$$
Теорема Гаусса–Маркова: при указанных предположениях оценка $\hat{\beta}$ является **BLUE** (Best Linear Unbiased Estimator), то есть:

- **линейная:** $\hat{\beta} = Ay$ для некоторой матрицы A;
- **несмещённая:** $\mathbb{E}[\hat{\beta} \mid X] = \beta$;
- **наилучшая по дисперсии:** среди всех линейных несмещённых оценок имеет минимальну ковариационную матрицу (в смысле неотрицательной определённости разности).

10.3.1 * Доказательство

Шаг 1: линейность и несмещённость OLS. OLS имеет вид $\hat{\beta} = Ay$, где
$$A = (X^\top X)^{-1} X^\top.$$
Тогда
$$\mathbb{E}[\hat{\beta} \mid X] = \mathbb{E}[Ay \mid X] = A\mathbb{E}[X\beta + \varepsilon \mid X] = A(X\beta + \mathbb{E}[\varepsilon \mid X]) = AX\beta = \beta,$$
поскольку
$$AX = (X^\top X)^{-1} X^\top X = I_d.$$
Значит, OLS несмещённа.
Шаг 2: ковариация OLS.
$$\mathrm{Var}(\hat{\beta} \mid X) = \mathrm{Var}(Ay \mid X) = A\,\mathrm{Var}(y \mid X)\,A^\top = A\,\mathrm{Var}(\varepsilon \mid X)\,A^\top = \sigma^2 AA^\top = \sigma^2 (X^\top X)^{-1}.$$
Шаг 3: сравнение с произвольной линейной несмещённой оценкой. Пусть $\tilde{\beta} = By$ — любая линейная несмещённая оценка. Несмещённость означает:
$$\mathbb{E}[\tilde{\beta} \mid X] = \beta \quad \Longleftrightarrow \quad BX = I_d.$$
Рассмотрим разность $B - A$ и обозначим $C := B - A$. Тогда
$$CX = (B - A)X = BX - AX = I_d - I_d = 0.$$
Ковариация $\tilde{\beta}$:
$$\mathrm{Var}(\tilde{\beta} \mid X) = \sigma^2 BB^\top = \sigma^2 (A + C)(A + C)^\top = \sigma^2 \big(AA^\top + AC^\top + CA^\top + CC^\top\big).$$
Покажем, что смешанные члены равны нулю. Заметим:
$$AX = (X^\top X)^{-1} X^\top X = I_d, \qquad CX = 0.$$

Отсюда следует

$$AC^\top = (CA^\top)^\top.$$

При стандартном выводе теоремы Гаусса–Маркова используется факт ортогональности остатка к пространству столбцов X в МНК-проекции, что эквивалентно занулению смешанных членов, и в итоге получается представление:

$$\mathrm{Var}(\tilde{\beta} \mid X) = \sigma^2\big(AA^\top + CC^\top\big) = \mathrm{Var}(\hat{\beta} \mid X) + \sigma^2 CC^\top.$$

Так как матрица $CC^\top$ неотрицательно определена, то

$$\mathrm{Var}(\tilde{\beta} \mid X) - \mathrm{Var}(\hat{\beta} \mid X) = \sigma^2 CC^\top \succeq 0.$$

Следовательно, никакая другая линейная несмещённая оценка не может иметь меньшую ковариа чем OLS. Значит, $\hat{\beta}$ является BLUE.

11 Градиентный спуск для линейной регрессии

11.1 Почему не используют матричное решение

Для линейной регрессии с MSE существует аналитическое решение (через нормальные уравнения), однако на практике в большинстве реальных задач предпочитают итеративные методы оптимизации (градиентный спуск и его вариации). Основные причины:

- **Вычислительная сложность.** Аналитическое решение требует вычисления и (явно или неявно) обращения матрицы $X^\top X$:
$$\hat{w} = (X^\top X)^{-1} X^\top y.$$
При d признаках обращение матрицы размера $d \times d$ имеет кубическую сложность по размеру (порядка $O(d^3)$), а также требуется вычислить $X^\top X$ (порядка $O(nd^2)$) и хранить промежуточные матрицы. Для больших d или при дорогих операциях ввода/вывода это становится узким местом.

- **Численная неустойчивость (плохая обусловленность) и мультиколлинеарность.** Если признаки сильно коррелированы, то матрица $X^\top X$ становится близкой к вырожденной (почти сингулярной). Тогда обращение (или решение нормальных уравнений) становится численно нестабильным: небольшие погрешности вычислений (в том числе из-за конечной точности арифметики) приводят к большим ошибкам в оценке весов, возможны «огромные» коэффициенты, которые компенсируют друг друга.

- **Память и инженерные ограничения.** Для больших датасетов X может не помещаться в память; аналитическое решение предполагает глобальную обработку данных. Итеративные методы допускают обучение по батчам/мини-батчам и потоковую обработку данных, что проще реализовать в распределённых и производственных системах.

- **Единый подход для разных моделей.** Даже если для линейной регрессии есть closed-form, для большинства моделей (логистическая регрессия, нейросети, многие регуляризованные постановки и т.п.) аналитического решения нет, и всё равно используются градиентные методы. Поэтому на практике часто сразу применяют унифицированный оптимизационный пайплайн.

Напомним формулировку задачи (MSE):

$$\min_{w \in \mathbb{R}^d} J(w), \qquad J(w) = \frac{1}{n}\sum_{i=1}^{n}(y_i - x_i^\top w)^2 = \frac{1}{n}\|y - Xw\|_2^2.$$

Полный (batch) градиент MSE в матричной форме:

$$\nabla_w J(w) = \frac{2}{n} X^\top (Xw - y).$$

Шаг градиентного спуска:

$$w^{(t+1)} = w^{(t)} - \eta \, \nabla_w J\Big(w^{(t)}\Big),$$

где $\eta > 0$ — шаг обучения (learning rate).

11.2 Стохастический градиентный спуск, преимущества

Стохастический градиентный спуск (SGD) — вариант градиентного спуска, в котором градиент оценивается не по всему датасету, а по случайному подмножеству объектов.

SGD по одному объекту. Для одного наблюдения (x_i, y_i) определим стохастическую оценку градиента:

$$g_i(w) = \nabla_w\Big((y_i - x_i^\top w)^2\Big) = 2(x_i^\top w - y_i)x_i.$$

Тогда шаг SGD:

$$w^{(t+1)} = w^{(t)} - \eta_t \, g_{i_t}\Big(w^{(t)}\Big),$$

где i_t — случайно выбранный индекс объекта на шаге t.

Mini-batch SGD. Чаще используют мини-батчи $B_t \subset \{1, \dots, n\}$ размера $|B_t| = b$:

$$g_{B_t}(w) = \frac{2}{b} X_{B_t}^\top (X_{B_t} w - y_{B_t}), \qquad w^{(t+1)} = w^{(t)} - \eta_t \, g_{B_t}\Big(w^{(t)}\Big).$$

Преимущества SGD (и mini-batch SGD):

- **Масштабируемость по числу объектов** n**.** Один шаг SGD/mini-batch SGD стоит порядка $O(bd)$ (или $O(d)$ при $b = 1$), тогда как полный градиент требует прохода по всем n объектам. Это особенно важно при очень больших датасетах.
- **Меньшие требования к памяти и удобство потокового обучения.** Для обновления весов достаточно держать в памяти один объект или мини-батч, что позволяет обучаться на данных, которые не помещаются в RAM, и обновлять модель по мере поступления данных.
- **Быстрое получение «достаточно хорошего» решения.** На первых итерациях SGD часто даёт заметный прирост качества быстрее, чем методы, требующие полной обработки датасета на каждом шаге. Это полезно для быстрых baseline и прототипирования.
- **Естественная совместимость с батчевой обработкой и параллелизмом.** Mini-batch SGD хорошо ложится на векторизацию и ускорение на GPU/TPU, а также на распределённые схемы (обработка батчей на нескольких устройствах).
- **Сходимость в выпуклом случае.** Для линейной регрессии (выпуклая квадратичная задача) SGD при корректном выборе расписания шагов η_t сходится к оптимальному решению (в смысле ожидания/по вероятности).

12 Регуляризация в линейных моделях

12.1 Мотивация регуляризации

Пусть задана линейная модель $\hat{y} = Xw$ и стандартная квадратичная функция потерь (MSE / SSE):

$$J(w) = \frac{1}{n}\|y - Xw\|_2^2 \quad \left(\text{или эквивалентно } \|y - Xw\|_2^2\right).$$

На практике при большом числе признаков, при наличии выбросов/шума и особенно при мультиколлинеарности матрица $X^\top X$ может быть плохо обусловленной (почти вырожденной), что приводит к **неустойчивому** решению: веса могут становиться очень большими по модулю и «компенсировать» друг друга, а небольшие численные погрешности приводят к заметным изменениям $\hat{w}$. Регуляризация решает эту проблему, **ограничивая (штрафуя) веса** модели и тем самым стабилизируя решение.

Общая идея регуляризации: к исходной функции потерь добавляют штраф за сложность (обычно норму весов):

$$J_{\text{reg}}(w) = J(w) + \lambda\,\Omega(w), \qquad \lambda \geq 0,$$

где $\Omega(w)$ — регуляризатор (например, L_2 или L_1 норма), а λ — коэффициент регуляризации (гиперпараметр). Новый оптимум является **компромиссом** между качеством на обучении и величиной весов, поэтому получаемое решение, как правило, становится **смещённым** (biased) относительно w^*, но более устойчивым и часто лучше обобщающимся.

12.2 Гребневая регрессия Ridge (L2)

12.2.1 Формула решения

Ridge-регрессия (Tikhonov regularization) использует L_2-штраф:

$$J_{\text{Ridge}}(w) = \frac{1}{n}\|y - Xw\|_2^2 + \lambda\|w\|_2^2.$$

(На практике константы при $1/n$ и $1/2$ могут отличаться в разных источниках; точка минимума по w от этого не меняется при соответствующем пересчёте λ.)

Найдём решение через условие оптимума. Возьмём градиент по w и приравняем к нулю:

$$\nabla_w J_{\text{Ridge}}(w) = \frac{2}{n}X^\top(Xw - y) + 2\lambda w = 0.$$

Эквивалентно:

$$\left(X^\top X + n\lambda I\right) w = X^\top y.$$

Отсюда (при обратимости матрицы) получаем замкнутую формулу:

$$\hat{w}_{\text{Ridge}} = \left(X^\top X + n\lambda I\right)^{-1} X^\top y.$$

Часто переобозначают $\tilde{\lambda} = n\lambda$ и пишут в стандартном виде:

$$\hat{w}_{\text{Ridge}} = \left(X^\top X + \tilde{\lambda} I\right)^{-1} X^\top y.$$

Добавление диагональной матрицы делает систему невырожденной и улучшает обусловленность что устраняет основную причину неустойчивости при мультиколлинеарности.

Замечание про свободный член (bias/intercept). Если вектора x_i и w включают константн признак (для свободного члена w_0), то обычно **свободный член не регуляризуют**:

$$\Omega(w) = \|w_{1:d}\|, \qquad \text{а } w_0 \text{ не штрафуется.}$$

12.3 Lasso-регуляризация (L1)

Lasso (Least Absolute Shrinkage and Selection Operator) использует L_1-штраф:

$$J_{\text{Lasso}}(w) = \frac{1}{n}\|y - Xw\|_2^2 + \lambda\|w\|_1, \qquad \|w\|_1 = \sum_{j=1}^{d}|w_j|.$$

Ключевые особенности Lasso:

- оптимум обычно оказывается **разреженным**: многие w_j становятся ровно нулевыми;
- это даёт **неявный отбор признаков** (implicit feature selection);

- регуляризатор выпуклый, но **недифференцируемый** в нуле, что не является принципиальной проблемой: применяются специальные методы оптимизации (например, координатный спуск, субградиенты, проксимальные методы).

В общем случае у Lasso **нет столь же простой** замкнутой формулы решения, как у Ridge, хотя существуют частные случаи (например, для ортонормированного дизайна), где решение выражается через операцию *soft-thresholding*.

12.4 Вероятностная интерпретация

Регуляризацию удобно интерпретировать вероятностно как **MAP-оценивание** (Maximum A Posteriori). Пусть задана вероятностная модель (например, регрессия с гауссовским шумом):

$$y = Xw + \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, \sigma^2 I).$$

Тогда

$$p(y \mid X, w) \propto \exp\left(-\frac{1}{2\sigma^2}\|y - Xw\|_2^2\right),$$

а отрицательное лог-правдоподобие (с точностью до константы) пропорционально MSE/SSE:

$$-\log p(y \mid X, w) = \text{const} + \frac{1}{2\sigma^2}\|y - Xw\|_2^2.$$

Если дополнительно задать априорное распределение на веса $p(w)$, то по формуле Байеса:

$$p(w \mid X, y) \propto p(y \mid X, w)\, p(w).$$

MAP-оценка определяется как:

$$\hat{w}_{\text{MAP}} = \arg\max_w p(w \mid X, y) = \arg\min_w \left(-\log p(y \mid X, w) - \log p(w)\right).$$

Отсюда видно, что регуляризация возникает как добавка $-\log p(w)$ к функции потерь:

$$J_{\text{MAP}}(w) = \underbrace{-\log p(y \mid X, w)}_{\text{data term}} + \underbrace{\big(-\log p(w)\big)}_{\text{regularization}}.$$

Ridge как гауссовский априор. Если

$$w \sim \mathcal{N}(0, \tau^2 I), \quad \Rightarrow \quad p(w) \propto \exp\left(-\frac{1}{2\tau^2}\|w\|_2^2\right),$$

то

$$-\log p(w) = \text{const} + \frac{1}{2\tau^2}\|w\|_2^2,$$

и MAP приводит к L_2-регуляризации (Ridge).

Lasso как лапласовский априор. Если

$$p(w) \propto \exp(-\alpha\|w\|_1),$$

то

$$-\log p(w) = \text{const} + \alpha\|w\|_1,$$

и MAP приводит к L_1-регуляризации (Lasso), что объясняет склонность к разреженности.

12.5 Elastic Net

Elastic Net сочетает L_1 и L_2 штрафы, объединяя свойства Ridge и Lasso:

$$J_{\text{EN}}(w) = \frac{1}{n}\|y - Xw\|_2^2 + \lambda_1\|w\|_1 + \lambda_2\|w\|_2^2.$$

Эквивалентная параметризация часто задаётся через общий коэффициент λ и долю L_1:

$$J_{\text{EN}}(w) = \frac{1}{n}\|y - Xw\|_2^2 + \lambda\left(\alpha\|w\|_1 + (1-\alpha)\|w\|_2^2\right), \qquad \alpha \in [0, 1].$$

Elastic Net полезен, когда:

- хочется **разреженности** (как у Lasso), но при этом
- есть **сильная корреляция признаков** (где чистый Lasso может быть нестабилен), а L_2-компонента стабилизирует решение.

12.6 Свойства регуляризаций

L_2 (Ridge):

- **сжимает веса** (shrinkage), особенно большие по модулю;

- **стабилизирует** решение и улучшает обусловленность (важно при мультиколлинеарности);
- **дифференцируема** и удобна для градиентных методов;
- обычно **не даёт строго нулевых** весов (признаки сохраняются, но их вклад уменьшается);
- решение **смещённое** (biased), то есть не обязано сходиться к w^* при фиксированном $\lambda > 0$.

L_1 (Lasso):

- **порождает разреженность**: многие коэффициенты становятся нулевыми;
- выполняет **неявный отбор признаков** (feature selection);
- **недифференцируема** в нуле, но остаётся выпуклой; оптимизация требует спецметодов;
- решение также **смещённое** (biased), как и при любой регуляризации.

Elastic Net:

- сочетает **устойчивость Ridge** и **разреженность Lasso**;
- часто предпочтителен при наличии **групп коррелированных признаков**.

В целом регуляризация управляет компромиссом **bias–variance**: обычно увеличивает смещение, но снижает дисперсию оценки и чувствительность к шуму/выбросам, что улучшает обобщающую способность модели.

13 Метрики регрессии MSE, MAE, RMSE, MAPE, SMAPE, R2

13.1 Когда их используют

Пусть даны истинные значения $y_1, \dots, y_n$ и предсказания модели $\hat{y}_1, \dots, \hat{y}_n$. Обозначим ошибки $e_i = y_i - \hat{y}_i$ и среднее $\bar{y} = \frac{1}{n}\sum_{i=1}^{n} y_i$. В лекции выделяются базовые метрики регрессии: MSE, MAE, RMSE, MAPE, SMAPE, R^2. :contentReference[oaicite:0]index=0

- **MSE (Mean Squared Error).**

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 = \frac{1}{n}\sum_{i=1}^{n} e_i^2.$$

  Когда используют:

  - когда важно **сильно штрафовать большие ошибки** (квадрат усиливает вклад крупных промахов);
  - как **оптимизируемую loss-функцию** (гладкая, дифференцируемая), в т.ч. для градиентных методов;
  - при вероятностной мотивации через **гауссовский шум** (MLE приводит к MSE) и в контексте классической линейной регрессии.

  Ограничение: чувствительна к выбросам/тяжёлым хвостам (outliers). :contentReference[oaic

- **RMSE (Root Mean Squared Error).**

$$\mathrm{RMSE} = \sqrt{\mathrm{MSE}} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}.$$

  Когда используют:

  - когда нужна метрика **в тех же единицах измерения**, что и таргет (в отличие от MSE, которая в квадрате единиц);
  - когда важно сохранить свойства MSE (сильный штраф больших ошибок), но сделать интерпретацию более «прикладной».

- **MAE (Mean Absolute Error).**

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| = \frac{1}{n}\sum_{i=1}^{n}|e_i|.$$

  Когда используют:

  - когда нужна **робастность** к выбросам (линейный штраф, в отличие от квадратичного)
  - когда важнее «средняя абсолютная ошибка», а не редкие крупные промахи;
  - как более устойчивый критерий при негауссовском шуме или тяжёлых хвостах. :contentReference[oaicite:2]index=2

  Замечание: MAE недифференцируема в нуле, но на практике это решаемо (субградиенты, проксимальные методы и т.п.). :contentReference[oaicite:3]index=3

- **MAPE (Mean Absolute Percentage Error).**

$$\mathrm{MAPE} = \frac{100\%}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i}\right| = \frac{100\%}{n}\sum_{i=1}^{n}\left|\frac{e_i}{y_i}\right|.$$

  Когда используют:

  - когда важно измерять ошибку **в относительных величинах** (проценты), чтобы сравнивать качество на разных масштабах таргета;
  - в бизнес-постановках, где «ошибка в процентах» напрямую интерпретируема (например прогноз спроса/выручки).

Ограничения:

- не определена при $y_i = 0$ и становится крайне нестабильной при $|y_i| \approx 0$;
- асимметрична по отношению к завышению/занижению через деление именно на y_i.

- **SMAPE (Symmetric Mean Absolute Percentage Error).** Один из распространённых вариантов:

$$\text{SMAPE} = \frac{100\%}{n} \sum_{i=1}^{n} \frac{2|y_i - \hat{y}_i|}{|y_i| + |\hat{y}_i|}.$$

Когда используют:

- когда нужна **относительная метрика**, но хочется уменьшить асимметрию MAPE;
- когда в данных встречаются малые значения таргета, и требуется более «мягкая» нормировка (через сумму модулей).

Ограничения:

- при $|y_i| + |\hat{y}_i| = 0$ выражение требует аккуратного определения (обычно задают 0 по соглашению);
- интерпретация «в процентах» зависит от выбранной формулы (в литературе встречаются близкие варианты).

- R^2 **(коэффициент детерминации).**

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2} = 1 - \frac{\text{SSE}}{\text{SST}}.$$

Когда используют:

- когда нужно понять, какую долю вариации таргета объясняет модель **по сравнению с базовой константной моделью** $\hat{y}_i = \bar{y}$;
- для отчётности и сопоставления моделей на **одном и том же датасете** (как безразмерная метрика).

Особенности:

- R^2 может быть **отрицательным**, если модель хуже предсказания средним;
- при отсутствии свободного члена и/или при специфических протоколах оценки возможны нюансы интерпретации.

Практическое правило выбора.

- Если критичны большие ошибки и нужна удобная оптимизация: чаще берут **MSE/RMSE**.
- Если важна устойчивость к выбросам и «типичная» ошибка: берут **MAE**.
- Если качество должно быть сопоставимо в процентах (и y не близок к нулю): берут **MAPE/SMAPE**.
- Если нужна интерпретация «насколько лучше среднего»: используют R^2 (обычно вместе с одной из error-метрик).

14 Параметры и гиперпараметры

14.1 Определения, свойства, использование, примеры

Параметры модели (parameters) — это величины, которые **настраиваются (оцениваются по данным** в процессе обучения. Их значения являются **результатом оптимизации** (минимиз функции потерь или максимизации правдоподобия) на обучающей выборке. :contentReference[oaicite:0]index=0

Гиперпараметры (hyperparameters) — это **константы**, значения которых **задаются до начала обучения** и не оптимизируются напрямую в рамках стандартной процедуры обучения данной модели. :contentReference[oaicite:1]index=1

Важное методологическое замечание: **технически любую величину можно сделать гипер** если зафиксировать её (не обучать), и наоборот — гиперпараметр можно пытаться оценивать из данных отдельной процедурой. Однако для «мейнстримных» моделей обычно существует устойчивое (принятое) разделение на то, что считается параметрами, а что — гиперпараметрами :contentReference[oaicite:2]index=2

Сравнение (инварианты и свойства):

- **Зависимость от обучающей выборки.**
  - Параметры **зависят** от train-данных: при изменении обучающего набора оптимальные веса обычно меняются.
  - Гиперпараметры **не зависят напрямую** от train-данных в смысле вычисления их значений алгоритмом обучения; их выбирают извне (по правилу/поиск-процедуре).
- **Когда определяются.**
  - Параметры определяются **во время обучения** (после запуска оптимизации).
  - Гиперпараметры фиксируются **до старта обучения**. :contentReference[oaicite:3]index=
- **Порядок и методы оптимизации.**
  - Параметры: оптимизация «первого уровня» (например, градиентный спуск, аналитичес решение в частных случаях).
  - Гиперпараметры: «внешняя» оптимизация (ручной подбор, grid/random search, байесов оптимизация и т.п.). :contentReference[oaicite:4]index=4
- **Роль в обобщающей способности и ёмкости модели.**
  - Параметры определяют конкретную **реализованную** функцию $f(x)$ после обучения.
  - Гиперпараметры часто задают **класс функций / ёмкость** модели (глубина дерева, регуляризация, архитектура сети), тем самым управляя компромиссом недообучение–переобучение. :contentReference[oaicite:5]index=5

Практическое использование (как это устроено в пайплайне обучения):

- **На train** обычно подбирают **параметры** (веса модели) путём оптимизации функции потерь.
- **На validation** выбирают **гиперпараметры** (какой тип/сложность модели лучше) и контролируют переобучение.
- **На test** выполняют финальную оценку качества (после выбора гиперпараметров), чтобы получить честную оценку обобщающей способности. В материалах курса отдельно подчёрки риск ошибки, когда тест используют как валидацию. :contentReference[oaicite:6]index=6

Примеры гиперпараметров (из лекций):

- **Naïve Bayes:** выбор распределения/модели признака (например, Gaussian, Uniform, Multinomial и т.д.). :contentReference[oaicite:7]index=7

- **Линейная регрессия:** тип регуляризации и коэффициент регуляризации (например, L_1, L_2, значение λ). :contentReference[oaicite:8]index=8
- **Деревья решений:** максимальная глубина дерева (`max_depth`) и другие ограничения роста дерева.
- **Нейросети:** число слоёв (архитектурная настройка). :contentReference[oaicite:10]index=10
- **RNN (пример из PyTorch-постановки):** `input_size`, `hidden_size`, `num_layers`, `dropou` `bidirectional`. :contentReference[oaicite:11]index=11

Примеры параметров (что именно обучается):

- **Линейная регрессия / линейные модели:** веса w (и, при наличии, свободный член w_0).
- **Деревья решений:** фактически выбранные разбиения (признаки и пороги) и значения в листьях являются «внутренними» настроенными элементами модели при фиксированном наборе гиперпараметров. :contentReference[oaicite:12]index=12
- **Нейросети:** матрицы весов и векторы смещений слоёв, которые оптимизируются градиентн методами.

15 Валидация и оценка модели

15.1 Недообучение и переобучение, определения и способы обнаружения

Недообучение (underfitting) — ситуация, когда модель слишком простая для задачи (высоки bias) и не способна хорошо аппроксимировать зависимость даже на обучающей выборке. Типичная картина:

ошибка на train высокая и ошибка на val/test высокая.

Причины: недостаточная сложность модели, слабые признаки, слишком сильная регуляризация, слишком мало итераций оптимизации.

Переобучение (overfitting) — ситуация, когда модель подстраивается под шум/случайные особенности обучающей выборки (высокий variance), из-за чего ухудшается обобщение на новые данные. Типичная картина:
ошибка на train низкая и ошибка на val/test существенно выше (большой generalization gap
Причины: слишком сложная модель, утечки данных, слишком много степеней свободы относител объёма данных, отсутствие регуляризации.

Как обнаруживают underfitting/overfitting на практике:

- **Сравнение метрик на train и validation/test.**
  - underfitting: обе метрики плохие;
  - overfitting: train-метрика заметно лучше validation/test.
- **Кривые обучения (learning curves).** Строят зависимости метрики (или loss) от:
  - числа итераций/эпох (для итеративного обучения);
  - размера обучающей выборки.

  Интерпретация:
  - underfitting: train-ошибка не падает до приемлемого уровня (модель «не умеет»);
  - overfitting: train-ошибка падает, а validation перестаёт улучшаться и/или начинает ухудшаться.
- **Валидационные кривые (validation curves) по сложности модели/регуляризации** Меняют гиперпараметр, влияющий на сложность (глубина дерева, λ регуляризации, число признаков, число итераций) и смотрят поведение train/val метрик:
  - рост сложности обычно улучшает train;
  - val растёт до некоторого оптимума, а затем может падать (зона overfitting).
- **Стабильность качества.** Большая вариативность метрики при разных разбиениях/фолда (или при небольших изменениях данных) часто указывает на высокую дисперсию модели и риск переобучения.

Типовые способы борьбы:

- underfitting: усложнить модель, улучшить признаки, ослабить регуляризацию, обучать дольше, добавить данные;
- overfitting: усилить регуляризацию, уменьшить сложность модели, применить early stopping, увеличить данные, улучшить процедуру валидации (исключить утечки), использовать ансамбли/усреднение.

15.2 Способы разбиения датасета

15.2.1 Что такое хорошее разбиение, идеальный и практический варианты

Разбиение (split method) — алгоритм, который делит один исходный набор данных на подмножества фиксированных размеров (например, train и validation). Идеальное требование к разбиению — **совпадение распределений данных** в полученных частях. Но на практике полное распределение мы проверить не можем, поэтому требование обычно ослабляют до **совпадения выбранных статистик** (доли классов, средние/квантили ключевых признаков, доли важных групп и т.п.).

Хорошее разбиение в прикладном смысле:

- делает validation/test **репрезентативными** относительно будущих данных;
- **не допускает утечек**: ни один объект (или информация о нём) не должен «перетекать» из test/val в train;
- соответствует природе данных: учитывает **время** (для временных рядов) и **группы** (для связанных объектов);
- поддерживает достаточный объём val/test, чтобы оценка качества имела приемлемую дисперсию.

15.2.2 4 основных типа разбиений

Ниже перечислены четыре базовых типа разбиений (split methods), используемые в практике:

1. **Равномерное случайное разбиение (uniform random split).**
   Базовый вариант: объекты случайно распределяются по частям датасета согласно заданным долям. Подходит, когда объекты можно считать i.i.d. и нет временной/групповой зависимост
2. **Стратифицированное разбиение (stratified split).**
   Поддерживает заданные пропорции классов (или иной дискретной переменной) в каждом сплите. Используется, когда важно сохранить баланс классов (особенно при дисбалансе).
3. **Групповое разбиение (grouped split).**
   Все объекты одной группы целиком попадают либо в train, либо в validation/test. Нужно, если объекты внутри группы коррелированы (например, один пациент в медицине, один пользователь в рекомендациях, один запрос в ранжировании), иначе оценка качества будет завышена из-за утечки похожих объектов между сплитами.
4. **Временное разбиение (timeseries split).**
   Используется для временных рядов и последовательностей: train формируется из прошлого, а validation/test — из будущего. Принципиально нельзя перемешивать данные по времени, иначе возникает look-ahead leakage.

15.3 Этапы построения модели

15.3.1 Train, validation, test, мотивация каждого

Классическая схема построения ML-модели состоит из трёх этапов/частей данных:

- **Train (обучающая выборка).**
  Используется для **обучения параметров** модели (например, веса w) путём оптимизации функции потерь.
- **Validation (валидационная выборка).**
  Используется для **выбора гиперпараметров** (сложность модели, регуляризация, число итераций, пороги и т.д.), для контроля переобучения и для сравнительной оценки вариантов пайплайна. Важно: преобразования (feature engineering, скейлинг, кодирование) должны обучаться на train и применяться к validation, а не на всём датасете сразу.

- **Test (тестовая выборка).**
  Используется для **финальной** (“честной”) оценки качества **после** выбора всех решений по модели и гиперпараметрам. Тест нельзя использовать в цикле подбора: если многократно измерять качество на test и принимать решения, test де-факто превращается в ещё одну validation-выборку, а оценка качества становится оптимистически смещённой.

Типовая ошибка: использовать `train_test_split` как будто этого достаточно, а затем подбирать гиперпараметры по “test“. В этом случае настоящего test-сета не остаётся, и оценка качества перестаёт быть независимой.

15.3.2 Кросс-валидация

Кросс-валидация (Cross-Validation, CV) — подход, при котором валидация проводится на нескольких разбиениях данных. Самый распространённый вариант — **k-fold CV**:

1. датасет делят на k непересекающихся фолдов;
2. для каждого фолда j обучают модель на $k-1$ фолдах и валидируют на фолде j;
3. итоговое качество агрегируют (например, среднее и стандартное отклонение метрики по фолдам).

Преимущество CV — более устойчивые оценки качества на малых датасетах (validation использует эффективнее). Недостаток — существенно выше вычислительная стоимость (модель обучается k раз, а при поиске гиперпараметров — ещё больше).

Частный случай — **Leave-One-Out (LOO)**: $k = n$, то есть валидация по одному объекту. Используется редко из-за высокой стоимости и специфических свойств дисперсии оценок.

15.3.3 Как выбрать организацию этапов

Организация этапов зависит от объёма данных, стоимости обучения и природы генерации данных (i.i.d., время, группы). Практический ориентир по размеру датасета:

- **малые датасеты** (примерно $< 10^3$): чаще используют k-fold CV ($k = 5$–10) для снижения дисперсии оценки;
- **средние датасеты** (10^3–10^5): обычно применяют фиксированное train/val/test (например, 60/20/20);
- **большие датасеты** (10^5–10^7): чаще 80/10/10 или 90/5/5;
- **очень большие датасеты** ($> 10^7$): достаточно holdout-разбиения (например, 95/5 или 98/2), так как дисперсия оценки на test становится малой.

Выбор между фиксированными сплитами и CV:

- **фиксированные сплиты** — основной вариант в индустрии, когда val/test достаточно большие и репрезентативные;
- **CV** — оправдана, когда данных мало и нужно стабилизировать оценку;
- **custom** — когда стандартные схемы не учитывают специфику (например, несколько вал. окон, uplift и т.п.).

Дополнительно:

- при наличии **временной зависимости** выбирают timeseries split и схему “прошлое→будущ
- при наличии **групп** используют grouped split как на уровне holdout, так и на уровне CV (GroupKFold и аналоги);
- при дорогом обучении часто применяют фиксированный split + аккуратный подбор гиперпа сохраняя test “закрытым” до финала.

15.4 Утечки данных

Утечка данных (data leakage) — ситуация, когда при обучении или выборе гиперпараметров модель (явно или неявно) получает доступ к информации, которая не должна быть доступна в момент предсказания на новых данных. Утечки приводят к **несправедливой (оптимистическ оценке качества** и провалу модели при внедрении.

Типовые виды утечек:

- **Preprocessing leakage:** преобразования обучаются на всём датасете до разбиения (скейлин PCA, отбор признаков по статистике таргета, импутация средними/модой, target encoding и т.п.). Правильно: сначала split, затем `fit` трансформеров на train и `transform` на val/test (удобно через пайплайны).
- **Target leakage:** в признаки попадает информация, прямо или косвенно зависящая от таргета, но недоступная в реальном сценарии (например, признаки, вычисленные "после события", или агрегаты, использующие будущее).
- **Leakage между объектами:** дубликаты/почти-дубликаты или связанные объекты попада и в train, и в test, что завышает качество (классический случай — один пользователь/пациен в разных сплитах при сильной корреляции).
- **Time leakage (look-ahead):** для временных рядов модель обучается на данных из будущего относительно test. Правильно: строгое временное разбиение и корректные "оконны признаки, использующие только прошлое.

Как предотвращают утечки:

- выполнять разбиение **до** feature engineering и любых оценок статистик по данным;
- обучать все трансформеры (скейлеры, кодировщики, PCA, отбор признаков) **только на train**;
- использовать корректный тип сплита (stratified/grouped/timeseries) по природе данных;
- держать test как "закрытый" набор: использовать его только для финальной оценки после выбора всех решений.

16 Линейная классификация

16.1 Постановка задачи классификации

В задаче классификации имеется обучающая выборка

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n}, \qquad x_i \in \mathbb{R}^d,$$

где x_i — вектор признаков объекта, а y_i — его метка класса. Требуется построить правило (модель) $a(\cdot)$, которое по новому объекту x возвращает предсказание класса $\hat{y} = a(x)$ и хорошо обобщается на новые данные.

С точки зрения оптимизации обычно задают **эмпирический риск**:

$$\min_{\theta} \frac{1}{n} \sum_{i=1}^{n} \ell(y_i,\, a(x_i; \theta))\,,$$

где ℓ — функция потерь на одном объекте, θ — параметры модели (в линейных моделях это веса w и, при необходимости, свободный член b).

Существенная особенность классификации: "истинная ошибка" является **ошибкой распознава** (правильно/неправильно), то есть естественная потеря — 0–1 ошибка, которая плохо оптимизиру напрямую; поэтому в практике почти всегда используют её гладкие/выпуклые аппроксимации (surrogate losses).

16.2 Варианты типов меток

Тип меток определяется природой классов и постановкой:

- **Бинарная классификация:** $y \in \{0, 1\}$ или $y \in \{-1, +1\}$. Представление $\{-1, +1\}$ особенно удобно для маржинальных формул. Между представлениями есть взаимно-однозначное преобразование:
$$y' = 2y - 1 \in \{-1, +1\}.$$
- **Многоклассовая (multiclass):** выбор ровно одного класса из K:
$$y \in \{1, 2, \ldots, K\}.$$
Часто используют one-hot кодировку:
$$y \in \{0, 1\}^K, \quad \sum_{k=1}^{K} y_k = 1.$$
- **Многометочная (multilabel):** можно выбрать несколько классов одновременно:
$$y \in \{0, 1\}^K, \quad \sum_{k=1}^{K} y_k \text{ произвольна.}$$

В зависимости от домена также встречаются упорядоченные метки (ordinal), иерархические/стр метки, но базовые три типа выше — наиболее частые.

16.3 Определение линейного классификатора и геометрическая интерпретация

Линейный классификатор строится на линейной (аффинной) функции **скоринга**:

$$s(x) = \langle w, x \rangle + b = w^\top x + b, \qquad w \in \mathbb{R}^d,\ b \in \mathbb{R}.$$

Для бинарной классификации типичное решающее правило:

$$\hat{y} = \operatorname{sign}(s(x)).$$

(Если метки $y \in \{0, 1\}$, то обычно берут порог: $\hat{y} = \mathbb{I}[s(x) \geq 0]$.)

Геометрическая интерпретация. Множество точек, где классификатор "не уверен" (граница решений), задаётся гиперплоскостью:

$$w^\top x + b = 0.$$

Вектор w является **нормалью** к этой гиперплоскости (указывает направление "роста скоринга") а b отвечает за сдвиг гиперплоскости.

Знак $s(x)$ определяет, в каком из двух полупространств находится точка x:

$$w^\top x + b > 0 \;\Rightarrow\; \hat{y} = +1, \qquad w^\top x + b < 0 \;\Rightarrow\; \hat{y} = -1.$$

Расстояние от точки x до гиперплоскости (в евклидовой норме) равно:

$$\operatorname{dist}(x, \{w^\top x + b = 0\}) = \frac{|w^\top x + b|}{\|w\|_2}.$$

16.4 Маржа, определение и ключевое свойство

Пусть для бинарной классификации метки заданы как $y_i \in \{-1, +1\}$. **Маржа** линейного классификатора на объекте i (часто называют **функциональной маржой**) определяется как:

$$m_i = y_i\,(w^\top x_i + b).$$

Ключевое свойство маржи:

$m_i < 0 \iff$ объект классифицирован неверно, $\quad m_i > 0 \iff$ классифицирован верно.

При этом величина $|m_i|$ отражает "уверенность" линейного решения: чем больше m_i, тем дальше объект от границы и тем устойчивее его классификация к малым возмущениям признако

Так как функциональная маржа масштабируется при умножении (w, b) на положительную константу, часто вводят **геометрическую маржу**:

$$\gamma_i = \frac{y_i\,(w^\top x_i + b)}{\|w\|_2}.$$

Она равна (со знаком) расстоянию до разделяющей гиперплоскости и не меняется при совместно масштабировании $(w, b) \mapsto c(w, b)$, $c > 0$ (в отличие от m_i).

16.5 Выбор весов

16.5.1 Почему для классификации много разных функций потерь

Естественная ошибка классификации на объекте — 0–1 ошибка:

$$\ell_{0/1}(y_i, s_i) = \mathbb{I}\big[y_i \neq \operatorname{sign}(s_i)\big] = \mathbb{I}\big[y_i\,(w^\top x_i + b) \leq 0\big].$$

Однако такая функция потерь:

- **недифференцируема** и имеет "плоские" области;
- делает задачу оптимизации **комбинаторной** (в общем случае трудной).

Поэтому в практике минимизацию 0–1 ошибки заменяют минимизацией **surrogate loss** — гладкой/выпуклой аппроксимации, зависящей от маржи $m = y\,(w^\top x + b)$:

$$\min_{w,b} \; \frac{1}{n} \sum_{i=1}^{n} \ell(m_i) \;+\; \lambda\,\Omega(w).$$

Почему существует **много разных** функций потерь для классификации:

- **Разные требования к оптимизации.** Хотят получить выпуклую задачу (гарантии глобального минимума) или гладкость (удобство градиентных методов). Например, квадрат потеря гладкая, но неудачна для классификации.
- **Разные "индуктивные смещения" (inductive bias).** Одни потери поощряют **максими маржи** (устойчивость к шуму у границы), другие — **вероятностную интерпретацию** и калибровку вероятностей.
- **Разная чувствительность к выбросам и "уверенным" ошибкам.** Потери по-разному штрафуют случаи, когда модель ошибается с большой уверенностью (большой отрицательн m).
- **Разные вероятностные модели и интерпретации.** Например, логистическая функция потерь выводится из MLE при бернуллиевской модели вероятности класса; в то время как hinge loss соответствует максимизации зазора (margin) в SVM-подходе.

Типовые примеры surrogate losses (как функций от маржи $m = y\,s(x)$):

- **Square loss (почти не используют для классификации):**

$$\ell(m) = (1 - m)^2.$$

- **Logistic loss (логистическая регрессия):**
$$\ell(m) = \log\bigl(1 + \exp(-m)\bigr).$$
- **Hinge loss (SVM):**
$$\ell(m) = \max(0, 1 - m).$$
- **Exponential loss (исторически важна для бустинга):**
$$\ell(m) = \exp(-m).$$

Выбор конкретной функции потерь — это выбор компромисса между оптимизационными свойствами, геометрическими (маржинальными) свойствами и вероятностной интерпретацией модели.

17 Логистическая регрессия

17.1 Базовое предположение и геометрическая интерпретация

Базовое предположение логистической регрессии: каждой точке пространства признаков $x \in \mathbb{R}^d$ соответствует **вероятность** принадлежности к положительному классу:

$$p(y = 1 \mid x) \in (0, 1).$$

При этом логистическая регрессия задаёт параметрическое семейство вероятностей, связывая их с линейным скором (линейным предиктором):

$$z(x) = w^\top x + b, \qquad p(y = 1 \mid x) = \sigma(z(x)).$$

Эквивалентная формулировка через **логиты** / **log-odds**:

$$\log \frac{p(y = 1 \mid x)}{1 - p(y = 1 \mid x)} = w^\top x + b.$$

То есть **логарифм отношения шансов** (odds) является линейной функцией признаков.

Геометрическая интерпретация. Граница решений при пороге 0.5 соответствует условию

$$p(y = 1 \mid x) = 0.5 \iff z(x) = 0 \iff w^\top x + b = 0,$$

то есть это **гиперплоскость** в $\mathbb{R}^d$, как и у линейного классификатора. Вектор w является нормалью к гиперплоскости, а b задаёт её сдвиг.

Классификационное правило по умолчанию:

$$\hat{y} = \mathbb{I}\left[p(y = 1 \mid x) \geq 0.5\right] = \mathbb{I}\left[w^\top x + b \geq 0\right].$$

17.2 Сигмоида как предсказание вероятности, свойства

Сигмоида (логистическая функция) определяется как

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

В логистической регрессии предсказание вероятности положительного класса:

$$\hat{p}(x) = p(y = 1 \mid x) = \sigma(w^\top x + b).$$

Ключевые свойства сигмоиды:

- **Область значений:** $\sigma(z) \in (0, 1)$ для любого $z \in \mathbb{R}$, что делает её естественным преобразов "скор $\to$ вероятность".
- **Монотонность:** $\sigma(z)$ строго возрастает по z.
- **Симметрия:**
$$\sigma(-z) = 1 - \sigma(z),$$
то есть функция "симметрична" относительно точки $(0, 0.5)$.
- **Производная:**
$$\sigma'(z) = \sigma(z)\big(1 - \sigma(z)\big),$$
что важно для градиентных методов оптимизации.
- **Связь с logit:** обратное преобразование к σ есть логит
$$\operatorname{logit}(p) = \log \frac{p}{1 - p}, \qquad \operatorname{logit}\big(\sigma(z)\big) = z.$$

17.3 Формула логистической функции потерь

Рассмотрим бинарную классификацию с метками $y_i \in \{0, 1\}$ и предсказаниями вероятностей $\hat{p}_i = \sigma(w^\top x_i + b)$. Тогда стандартная **логистическая функция потерь** (она же **log-loss**, **binary cross-entropy**) на датасете:

$$\mathcal{L}(w, b) = -\sum_{i=1}^{n} \Big[y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \Big].$$

Часто используют усреднённый вариант $\frac{1}{n}\mathcal{L}$.

Эквивалентная форма через маржу (удобна при метках $y_i \in \{-1, +1\}$). Пусть $y_i \in \{-1, +1\}$ и $z_i = w^\top x_i + b$. Тогда

$$\mathcal{L}(w, b) = \sum_{i=1}^{n} \log\bigl(1 + \exp(-y_i z_i)\bigr).$$

17.3.1 вывод через MLE

Пусть задана вероятностная модель: для каждого объекта (x_i, y_i)

$$y_i \mid x_i \sim \text{Bernoulli}(p_i), \qquad p_i = \sigma(w^\top x_i + b).$$

Тогда правдоподобие выборки (при независимости объектов) равно

$$p(y \mid X, w, b) = \prod_{i=1}^{n} p_i^{y_i} (1 - p_i)^{1 - y_i}.$$

Лог-правдоподобие:

$$\log p(y \mid X, w, b) = \sum_{i=1}^{n} \Bigl[y_i \log p_i + (1 - y_i) \log(1 - p_i) \Bigr].$$

MLE-оценка максимизирует лог-правдоподобие, то есть эквивалентно минимизирует отрицатель лог-правдоподобие:

$$(\hat{w}, \hat{b}) = \arg\max_{w,b} \log p(y \mid X, w, b) = \arg\min_{w,b} \left(- \sum_{i=1}^{n} \Bigl[y_i \log p_i + (1 - y_i) \log(1 - p_i) \Bigr] \right).$$

Получаем, что минимизация логистической потери **совпадает с MLE** для Бернуллиевской модели, где параметр вероятности задаётся через сигмоиду от линейного скора.

17.4 Свойство калибровки вероятностей

Калиброванность вероятностей (calibration) в бинарном случае означает, что предсказанное значение $\hat{p}$ соответствует истинной доле положительного класса среди объектов с таким предска

$$\mathbb{P}(Y = 1 \mid \hat{p} = p) = p.$$

Интерпретация: если модель многократно выдаёт вероятность 0.8 на группе объектов, то примерно в 80% случаев класс действительно будет положительным.

Логистическая регрессия является **вероятностной моделью**: она непосредственно моделирует Бернуллиевское распределение в каждой точке пространства признаков, то есть выдаёт именно $\hat{p}(x) = p(y = 1 \mid x)$. Поэтому в типовом изложении логистическая регрессия рассматривается как **калиброванный классификатор**. На практике калибровка может ухудшаться при сильной мисспецификации модели, сдвиге распределений или агрессивной регуляризации; тогда применяют отдельные процедуры калибровки, но базовая мотивация логистической регрессии — именно получение вероятностей, а не только меток.

17.5 Обобщённая линейная модель

Обобщённая линейная модель (GLM) задаётся тремя компонентами:

- распределение отклика Y из **экспоненциального семейства**;
- линейный предиктор $\eta(x) = w^\top x + b$;
- **link-функция** $g(\cdot)$, связывающая математическое ожидание отклика с линейным предикто

$$g(\mu(x)) = \eta(x), \qquad \mu(x) = \mathbb{E}[Y \mid X = x].$$

Логистическая регрессия — частный случай GLM для

$$Y \mid X = x \sim \text{Bernoulli}(p(x)), \qquad \mu(x) = \mathbb{E}[Y \mid X = x] = p(x),$$

с **логит-ссылкой** (logit link):

$$g(p) = \log \frac{p}{1 - p}, \qquad g(p(x)) = w^\top x + b.$$

Эквивалентно:

$$p(x) = \sigma(w^\top x + b).$$

Таким образом, логистическая регрессия — это GLM, где распределение отклика Бернулли, а link-функция логит, что и приводит к сигмоиде в явной формуле вероятности.

18 Стратегии многоклассовой классификации

18.1 One-vs-one, one-vs-all и их свойства

Пусть решается задача многоклассовой классификации:

$$y \in \{1, 2, \ldots, K\}, \qquad \mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n}.$$

Многие базовые классификаторы естественно устроены как бинарные, поэтому многоклассовую задачу часто сводят к набору бинарных задач и затем агрегируют решения.

One-vs-All / One-vs-Rest (OvA/OvR).

Идея: обучить K бинарных классификаторов $\{f_k\}_{k=1}^{K}$, где k-й классификатор отделяет класс k от "всех остальных":

$$\text{для } f_k: \quad \tilde{y}_i^{(k)} = \begin{cases} +1, & y_i = k, \\ -1, & y_i \neq k. \end{cases}$$

Если базовый классификатор линейный, то

$$f_k(x) = w_k^\top x + b_k,$$

а **маржа** равна $m_k(x) = f_k(x)$ (или $m_k(x) = y f_k(x)$ в записи с $\{-1, +1\}$).

Инференс (предсказание класса). Стандартное правило агрегации: взять класс с максимальным "скорингом" (максимальной маржой):

$$\hat{y}(x) = \arg \max_{k \in \{1,\ldots,K\}} f_k(x).$$

Если базовый метод выдаёт вероятности $\hat{p}_k(x) = \mathbb{P}(y = k \mid x)$ (или их суррогаты), то часто используют

$$\hat{y}(x) = \arg\max_k \hat{p}_k(x).$$

Свойства OvR:

- **Число обучаемых моделей:** K бинарных классификаторов.
- **Данные для каждого классификатора:** используется **весь датасет** (все n объектов), но метки перекодируются "класс k против остальных".
- **Класс-имбаланс в подзадачах:** для каждого f_k положительный класс один, отрицательный – "все остальные", поэтому часто возникает сильный дисбаланс, особенно при больших K или редких классах.
- **Проблема сопоставимости скорингов:** значения $f_k(x)$ (маржи) от разных бинарных классификаторов могут быть **на разных шкалах**, поэтому простое $\arg\max f_k(x)$ может быть чувствительно к калибровке/масштабам.
- **Неоднозначность по порогам:** если вместо $\arg\max$ пытаться решать через пороги (например, "берём все классы, где $f_k(x) \geq 0$"), то возможны случаи:
  - **несколько** классов одновременно "положительные";
  - **ни один** класс не "положительный".

  Поэтому на практике обычно выбирают именно максимум по скору/вероятности.

One-vs-One (OvO).

Идея: обучить бинарный классификатор для **каждой пары классов** (k, ℓ), $1 \leq k < \ell \leq K$:

$$f_{k\ell}: \text{ отличает } k \text{ от } \ell \text{ и обучается только на объектах этих двух классов.}$$

При линейной модели:

$$f_{k\ell}(x) = w_{k\ell}^\top x + b_{k\ell}.$$

Инференс (голосование). Каждый парный классификатор "голосует" за один из двух классов, после чего суммируются голоса:

$$v_k(x) = \sum_{\ell \neq k} \mathbb{I}\big[f_{k\ell}(x) > 0\big] \quad \text{(с корректной интерпретацией знака для пары } (k, \ell)),$$

и итоговый класс выбирается по большинству голосов:

$$\hat{y}(x) = \arg\max_k v_k(x).$$

(Возможны ничьи; тогда применяют правила разруливания, например по сумме марж, по вероятностям или по приоритету классов.)

Свойства OvO:

- **Число обучаемых моделей:** $\frac{K(K-1)}{2}$ бинарных классификаторов.
- **Данные для каждого классификатора: подвыборка** датасета (только два класса), поэтому каждая подзадача обычно лучше сбалансирована, чем в OvR, и может быть проще для обучения.
- **Вычислительная цена:**
  - обучение требует много моделей (быстро растёт по K);
  - инференс также требует прогнать много моделей (порядка $O(K^2)$ бинарных предиктов)
- **Практический компромисс:** OvO часто выигрывает, когда K не очень велико, а бинарны классификатор чувствителен к дисбалансу или сильно зависит от "мешающих" классов; OvR часто выигрывает по простоте и скорости при большом K.

Короткое сравнение OvR и OvO:

$$\#\text{классификаторов:} \quad \text{OvR} = K, \qquad \text{OvO} = \frac{K(K-1)}{2};$$

данные на классификатор: OvR = полный датасет, OvO = подвыборка по двум классам

19 Метрики классификации

19.1 Определение метрики в ML

19.1.1 Свойства, сравнение с функцией потерь

Метрика (metric) в ML — это численная функция качества, которая измеряет, насколько хорошо модель решает задачу на данных (обычно на validation/test). В классификации метрика оценивает согласованность предсказанных меток/скорингов с истинными метками. Метрики используются для:

- сравнения моделей и выбора гиперпараметров;
- контроля переобучения (качество на validation);
- финальной отчётности на test и бизнес-интерпретации качества.

Свойства метрики:

- **Интерпретируемость:** метрика должна иметь понятный смысл в терминах ошибок (например, доля верных ответов).
- **Согласованность с задачей:** метрика должна отражать то, что реально важно (например полнота при важности пропуска положительных объектов).
- **Стабильность:** при разумных изменениях данных метрика не должна "скакать" слишком сильно.
- **Сопоставимость:** метрика должна позволять сравнивать модели на одном протоколе оценки.

Сравнение с функцией потерь (loss).

- **Loss** — то, что оптимизируется при обучении (минимизируется), обычно должна быть удобна для оптимизации (гладкость/выпуклость/дифференцируемость).
- **Метрика** — то, чем оценивают качество, может быть недифференцируемой (accuracy, F1 и т.п.).

Часто минимизируют surrogate-loss (например, log-loss), а качество измеряют другой метрикой (например, ROC-AUC, F1), если она лучше соответствует прикладной цели.

19.2 Матрица ошибок (бинарная, многоклассовая)

19.2.1 Типы ошибок

19.2.2 Правило присвоения меток

Матрица ошибок (confusion matrix) — таблица, описывающая распределение предсказаний по истинным классам.

Бинарный случай. Пусть истинные метки $y \in \{0, 1\}$ и предсказанные метки $\hat{y} \in \{0, 1\}$. Тогда:

	$\hat{y} = 1$	$\hat{y} = 0$
$y = 1$	TP	FN
$y = 0$	FP	TN

где

- TP (true positive) — истинно-положительные,
- TN (true negative) — истинно-отрицательные,
- FP (false positive) — ложно-положительные (ошибка I рода),
- FN (false negative) — ложно-отрицательные (ошибка II рода).

Многоклассовый случай. Для K классов строится матрица $K \times K$, где элемент (i, j) равен числу объектов с истинным классом i и предсказанным классом j. Диагональ — верные ответы, внедиагональные элементы — ошибки.

Правило присвоения меток. Если модель выдаёт скор/вероятность $\hat{p}(x)$, то метка определяет порогом τ:

$$\hat{y} = \mathbb{I}[\hat{p}(x) \geq \tau].$$

При $\tau = 0.5$ получаем стандартное правило, но τ часто подбирают по validation исходя из стоимости ошибок или оптимизации нужной метрики (например, F1, recall при ограничении на precision и т.п.).

19.3 Accuracy, balanced accuracy, precision, recall, F1-score

Accuracy (доля верных ответов):

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}.$$

Используется при относительно сбалансированных классах и симметричной стоимости ошибок.

Balanced accuracy (сбалансированная точность) — средняя точность по классам; в бинарном случае:

$$\text{BalancedAcc} = \frac{1}{2}\left(\frac{TP}{TP + FN} + \frac{TN}{TN + FP}\right) = \frac{\text{TPR} + \text{TNR}}{2}.$$

Полезна при дисбалансе классов, когда обычная accuracy может быть обманчивой.

Precision (точность/доля истинных среди предсказанных положительных):

$$\text{Precision} = \frac{TP}{TP + FP}.$$

Интерпретация: “если модель сказала 1, как часто это правда”.

Recall (полнота/доля найденных положительных):

$$\text{Recall} = \frac{TP}{TP + FN}.$$

Интерпретация: “какую долю положительных мы нашли”.

F1-score — гармоническое среднее precision и recall:

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2TP}{2TP + FP + FN}.$$

Используется, когда важен компромисс между precision и recall, особенно при дисбалансе.

19.3.1 Sensitivity, specificity и другие

В медицинской и инженерной диагностике часто используют:

$$\textbf{Sensitivity} = \text{TPR} = \text{Recall} = \frac{TP}{TP + FN},$$

$$\textbf{Specificity} = \text{TNR} = \frac{TN}{TN + FP}.$$

Также полезны:

$$\text{FPR} = \frac{FP}{FP + TN} = 1 - \text{Specificity}, \qquad \text{FNR} = \frac{FN}{FN + TP} = 1 - \text{Sensitivity}.$$

Иногда используют **NPV** (negative predictive value):

$$\text{NPV} = \frac{TN}{TN + FN},$$

и **FDR** (false discovery rate):

$$\text{FDR} = \frac{FP}{TP + FP} = 1 - \text{Precision}.$$

19.3.2 Несбалансированная стоимость ошибок, примеры, как исправлять дисбаланс

Во многих задачах стоимость FP и FN различается:

- **Медицина:** пропуск болезни (FN) часто значительно дороже ложной тревоги (FP).

- **Антифрод:** пропуск мошенничества (FN) дорог, но слишком много FP создают неудобства и стоимость обработки.
- **Безопасность:** FN может быть критичен (пропуск угрозы), поэтому оптимизируют recall при контроле FP.

Как учитывать несбалансированную стоимость ошибок и дисбаланс классов:

- **Подбор порога τ:** сдвигают порог вероятности, чтобы получить нужный баланс precision/recall.
- **Взвешивание классов/ошибок:** задают веса в loss-функции (class weights) или используют cost-sensitive learning.
- **Ресэмплинг:** undersampling большинства или oversampling меньшинства (в т.ч. синтетические, например SMOTE).
- **Подбор метрики:** вместо accuracy использовать balanced accuracy, F1, recall@precision и т.п.

19.4 ROC AUC

19.4.1 Свойства ROC и ROC-AUC

Пусть модель выдаёт скор $\hat{s}(x)$ или вероятность $\hat{p}(x)$, и мы меняем порог τ. Тогда строятся:

$$\mathrm{TPR}(\tau) = \frac{TP(\tau)}{TP(\tau) + FN(\tau)}, \qquad \mathrm{FPR}(\tau) = \frac{FP(\tau)}{FP(\tau) + TN(\tau)}.$$

ROC-кривая — график $\mathrm{TPR}(\tau)$ от $\mathrm{FPR}(\tau)$ при переборе порога τ. **ROC-AUC** — площадь под ROC-кривой:

$$\text{ROC-AUC} = \int_0^1 \mathrm{TPR}\big(\mathrm{FPR}\big)\, d(\mathrm{FPR}).$$

Свойства ROC и ROC-AUC:

- ROC-AUC интерпретируется как вероятность того, что случайный положительный объект получит скор выше случайного отрицательного (rank-based интерпретация).
- ROC-AUC **не зависит от выбора конкретного порога** (оценивает ранжирующую способность).
- ROC-AUC часто менее чувствителен к дисбалансу классов, чем accuracy, но при сильном дисбалансе может давать слишком оптимистичную картину, если FP не критичны в абсолютных величинах.

19.4.2 PR AUC

PR-кривая строится как зависимость precision от recall при переборе порога τ:

$$\mathrm{Precision}(\tau) = \frac{TP(\tau)}{TP(\tau) + FP(\tau)}, \qquad \mathrm{Recall}(\tau) = \frac{TP(\tau)}{TP(\tau) + FN(\tau)}.$$

PR-AUC — площадь под PR-кривой.

Когда PR-AUC особенно полезна: при сильном дисбалансе классов и когда важен контроль ложноположительных, так как precision напрямую учитывает FP. Поэтому для задач антифрода редких событий PR-AUC часто информативнее ROC-AUC.

19.5 Многоклассовые расширения

19.5.1 Микро- и макро-усреднение

Для многоклассовой классификации метрики precision/recall/F1 обобщают через агрегацию по классам. Пусть для каждого класса k можно определить TP_k, FP_k, FN_k в схеме “класс k против остальных”.

Macro-усреднение — усреднение метрики по классам (все классы равновесны):

$$\text{Precision}_{\text{macro}} = \frac{1}{K}\sum_{k=1}^{K}\text{Precision}_k, \qquad \text{Recall}_{\text{macro}} = \frac{1}{K}\sum_{k=1}^{K}\text{Recall}_k, \qquad F1_{\text{macro}} = \frac{1}{K}\sum_{k=1}^{K}F1_k.$$

Macro полезно, когда важно качество на **каждом** классе, в том числе редком.

Micro-усреднение — суммирование TP/FP/FN по всем классам и вычисление метрики из сумм:

$$TP_{\text{micro}} = \sum_{k=1}^{K}TP_k, \quad FP_{\text{micro}} = \sum_{k=1}^{K}FP_k, \quad FN_{\text{micro}} = \sum_{k=1}^{K}FN_k,$$

$$\text{Precision}_{\text{micro}} = \frac{TP_{\text{micro}}}{TP_{\text{micro}} + FP_{\text{micro}}}, \qquad \text{Recall}_{\text{micro}} = \frac{TP_{\text{micro}}}{TP_{\text{micro}} + FN_{\text{micro}}}, \qquad F1_{\text{micro}} = \frac{2 \cdot \text{Precisio}}{\text{Precision}_{\text{r}}}$$

Micro сильнее отражает качество на **частых** классах (так как каждый объект имеет одинаковый вес).

Замечание. Для многоклассовых ROC-AUC/PR-AUC применяют схемы OvR (“каждый класс против остальных”) с macro/micro усреднением площадей либо усреднением кривых, в зависимос от протокола оценки.

20 Матричные нормы

20.1 p-нормы, самые важные частные случаи

Нормы используются для измерения "размера" векторов и матриц, а в ML особенно важны для регуляризации (L_1/L_2), оценки устойчивости и анализа обусловленности.

1) p-норма вектора. Для $x \in \mathbb{R}^d$ и $p \geq 1$:

$$\|x\|_p = \left(\sum_{j=1}^{d} |x_j|^p \right)^{1/p}.$$

Также часто используют предельную норму:

$$\|x\|_\infty = \max_{1 \leq j \leq d} |x_j|.$$

Важные частные случаи (векторные нормы):

$$\|x\|_1 = \sum_{j=1}^{d} |x_j| \quad \text{(L1, разреживающая регуляризация)},$$

$$\|x\|_2 = \left(\sum_{j=1}^{d} x_j^2 \right)^{1/2} \quad \text{(L2, "евклидова")},$$

$$\|x\|_\infty = \max_j |x_j| \quad \text{(контроль максимального отклонения)}.$$

2) Индуцированная (операторная) p-норма матрицы. Для матрицы $A \in \mathbb{R}^{m \times d}$ индуцированная норма определяется через действие на вектора:

$$\|A\|_p = \max_{x \neq 0} \frac{\|Ax\|_p}{\|x\|_p} = \max_{\|x\|_p = 1} \|Ax\|_p.$$

Эта норма естественно измеряет, насколько сильно линейное преобразование $x \mapsto Ax$ может "растянуть" вектор в соответствующей p-норме.

Важные частные случаи (индуцированные матричные нормы):

- $p = 1$**: максимум суммы по столбцу.**

$$\|A\|_1 = \max_{1 \leq j \leq d} \sum_{i=1}^{m} |a_{ij}|.$$

- $p = \infty$**: максимум суммы по строке.**

$$\|A\|_\infty = \max_{1 \leq i \leq m} \sum_{j=1}^{d} |a_{ij}|.$$

- $p = 2$**: спектральная норма.**

$$\|A\|_2 = \max_{\|x\|_2 = 1} \|Ax\|_2 = \sqrt{\lambda_{\max}(A^\top A)}.$$

  Здесь $\lambda_{\max}(A^\top A)$ — максимальное собственное значение матрицы $A^\top A$.

3) Норма Фробениуса (часто используется в ML). Хотя она не является индуцированной p-нормой, это одна из наиболее популярных матричных норм:

$$\|A\|_F = \left(\sum_{i=1}^{m} \sum_{j=1}^{d} a_{ij}^2 \right)^{1/2} = \sqrt{\operatorname{tr}(A^\top A)}.$$

Интерпретация: это евклидова норма вектора, полученного "разворачиванием" матрицы в один столбец.

Ключевые свойства (в контексте ML):

- **Монотонность по p для вектора:** при $1 \leq p \leq q \leq \infty$ верно $\|x\|_q \leq \|x\|_p$ (при фиксированном x).

- **Субмультипликативность индуцированных норм:**
$$\|AB\|_p \leq \|A\|_p \, \|B\|_p,$$
что важно для анализа устойчивости композиции линейных преобразований.

- **Связь с регуляризацией:** L_2-регуляризация штрафует $\|w\|_2^2$, а L_1 — $\|w\|_1$, что приводит к разным свойствам решений (сжатие vs разреженность).

21 Метод опорных векторов (SVM)

21.1 Геометрическая интерпретация

Рассмотрим бинарную линейную классификацию с метками $y_i \in \{-1, +1\}$ и линейным скором

$$f(x) = w^\top x + b.$$

Разделяющая гиперплоскость задаётся уравнением $w^\top x + b = 0$.

Линейно разделимый случай. Если существует (w, b) такое, что $y_i(w^\top x_i + b) > 0$ для всех i, то существует **много** разделяющих гиперплоскостей. SVM выбирает из них ту, которая **максимизирует зазор (margin)** между классами.

В SVM вводят две опорные гиперплоскости

$$w^\top x + b = +1, \qquad w^\top x + b = -1,$$

которые образуют "полосу" (stripe) вокруг разделяющей границы $w^\top x + b = 0$. Условие корректной классификации с фиксацией масштаба (нормировки) записывают как

$$y_i(w^\top x_i + b) \geq 1, \qquad i = 1, \ldots, n.$$

Тогда **ширина полосы** равна расстоянию между гиперплоскостями $w^\top x + b = \pm 1$:

$$\text{margin} = \frac{2}{\|w\|_2}.$$

Следовательно, **максимизация маржи** эквивалентна **минимизации** $\|w\|_2$ (или $\|w\|_2^2$).

Опорные векторы (support vectors) — это объекты, лежащие на границах полосы или нарушающие её в soft-margin случае. В решении SVM существенную роль играют только они: именно они определяют итоговую гиперплоскость, а у большинства объектов вклад в решение равен нулю (через нулевые коэффициенты в двойственной постановке).

21.2 Оптимизационная задача для линейно разделимого случая

21.2.1 * Двойственная задача

Прямая (primal) задача, hard-margin SVM. Максимизацию $\frac{2}{\|w\|_2}$ удобно переписать как выпуклую задачу минимизации:

$$\min_{w,b} \ \frac{1}{2}\|w\|_2^2 \quad \text{s.t.} \quad y_i(w^\top x_i + b) \geq 1, \ \ i = 1, \ldots, n.$$

Это **выпуклая** квадратичная оптимизация с линейными ограничениями.

Лагранжиан. Введём множители Лагранжа $\alpha_i \geq 0$ для ограничений $y_i(w^\top x_i + b) - 1 \geq 0$:

$$\mathcal{L}(w, b, \alpha) = \frac{1}{2}\|w\|_2^2 - \sum_{i=1}^{n} \alpha_i \big(y_i(w^\top x_i + b) - 1\big).$$

Условия стационарности (KKT) по w и b:

$$\frac{\partial \mathcal{L}}{\partial w} = 0 \ \Rightarrow \ w = \sum_{i=1}^{n} \alpha_i y_i x_i, \qquad \frac{\partial \mathcal{L}}{\partial b} = 0 \ \Rightarrow \ \sum_{i=1}^{n} \alpha_i y_i = 0.$$

Подставляя w в лагранжиан, получаем двойственную задачу.

Двойственная (dual) задача, hard-margin SVM:

$$\max_{\alpha \in \mathbb{R}^n} \ \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j \, (x_i^\top x_j)$$

$$\text{s.t.} \quad \alpha_i \geq 0, \ \ i = 1, \ldots, n, \qquad \sum_{i=1}^{n} \alpha_i y_i = 0.$$

Восстановление параметров. После нахождения оптимальных $\alpha^\star$:

$$w^\star = \sum_{i=1}^{n} \alpha_i^\star y_i x_i.$$

Смещение $b^\star$ можно найти по любому опорному вектору x_s с $\alpha_s^\star > 0$ из условия $y_s(w^{\star\top}x_s + b^\star) = 1$:

$$b^\star = y_s - w^{\star\top}x_s.$$

В оптимальном решении многие $\alpha_i^\star = 0$; ненулевые соответствуют опорным векторам.

21.3 Неразделимый случай, slack-переменные

Если данные не линейно разделимы, вводят **slack-переменные** $\xi_i \geq 0$, которые позволяют нарушать ограничения, но штрафуют такие нарушения в целевой функции.

Soft-margin SVM (прямая задача):

$$\min_{w,b,\xi} \ \frac{1}{2}\|w\|_2^2 + C\sum_{i=1}^{n}\xi_i$$

$$\text{s.t.} \quad y_i(w^\top x_i + b) \geq 1 - \xi_i, \ \ i = 1,\ldots,n, \qquad \xi_i \geq 0, \ \ i = 1,\ldots,n.$$

Здесь $C > 0$ — гиперпараметр компромисса:

- **большое** C сильнее штрафует ошибки/нарушения маржи $\Rightarrow$ модель старается "подогнать" train;
- **малое** C допускает больше нарушений, но увеличивает роль регуляризации $\|w\|_2^2$ (шире полоса).

Геометрический смысл ξ_i. Если $\xi_i = 0$, объект лежит на правильной стороне полосы: $y_i(w^\top x_i + b) \geq 1$. Если $0 < \xi_i < 1$, объект классифицирован верно, но находится внутри полосы. Если $\xi_i \geq 1$, объект либо на границе, либо классифицирован неверно (маржа неположительна).

21.4 Hinge loss

Обозначим **положительную часть** (positive part):

$$[t]_+ = \max(0, t).$$

Тогда **hinge loss** для объекта (x_i, y_i):

$$\ell_{\text{hinge}}(y_i, f(x_i)) = [\,1 - y_i f(x_i)\,]_+ = \max\bigl(0,\, 1 - y_i(w^\top x_i + b)\bigr).$$

Интерпретация:

- если $y_i(w^\top x_i + b) \geq 1$, то $\ell_{\text{hinge}} = 0$ (объект вне полосы, запас маржи достаточен);
- если $y_i(w^\top x_i + b) < 1$, то штраф линейно растёт по мере уменьшения маржи.

Связь hinge loss и soft-margin. Из ограничений $y_i(w^\top x_i + b) \geq 1 - \xi_i$, $\xi_i \geq 0$ следует, что минимально допустимое ξ_i равно

$$\xi_i^\star = [\,1 - y_i(w^\top x_i + b)\,]_+.$$

Подставляя это в целевую функцию, получаем эквивалентную (без явных ξ) форму:

$$\min_{w,b} \ \frac{1}{2}\|w\|_2^2 + C\sum_{i=1}^{n}[\,1 - y_i(w^\top x_i + b)\,]_+.$$

То есть hinge loss "извлекается" из ограничений soft-margin постановки и дополняется L_2-регуляризацией, отвечающей за ширину полосы.

21.5 Ядровой трюк, свойства ядер, примеры ядер

Линейный SVM строит гиперплоскость в исходном пространстве признаков. Для нелинейной разделимости применяют **переход в признаковое пространство**:

$$\varphi : \mathbb{R}^d \to \mathcal{H}, \qquad x \mapsto \varphi(x),$$

где $\mathcal{H}$ — (возможно, высоко- или бесконечномерное) гильбертово пространство. В $\mathcal{H}$ модель остаётся линейной:

$$f(x) = \langle w, \varphi(x)\rangle_{\mathcal{H}} + b.$$

Ядровой трюк (kernel trick). В двойственной задаче и в предсказании SVM данные входят только через скалярные произведения $x_i^\top x_j$. Если заменить их на

$$K(x_i, x_j) = \langle \varphi(x_i), \varphi(x_j)\rangle_{\mathcal{H}},$$

то можно обучать и применять SVM в $\mathcal{H}$, **не вычисляя** $\varphi(x)$ явно.

Dual-форма с ядром. Двойственная задача (soft/hard margin различаются ограничениями на α, но структура одинакова) принимает вид:

$$\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j \, K(x_i, x_j) \quad \text{s.t.} \quad \sum_{i=1}^{n} \alpha_i y_i = 0, \;\; \alpha_i \geq 0.$$

В soft-margin добавляется верхняя граница $\alpha_i \leq C$.

Решающее правило (decision function). Итоговая функция классификации выражается через опорные векторы:

$$f(x) = \sum_{i=1}^{n} \alpha_i^{\star} y_i K(x_i, x) + b^{\star}, \qquad \hat{y} = \operatorname{sign}(f(x)).$$

Свойства ядра. Функция $K(\cdot, \cdot)$ является допустимым ядром, если:

- **симметрична:** $K(x, z) = K(z, x)$;
- **положительно полуопределена (PSD):** для любых точек $x_1, \ldots, x_n$ матрица Грама $[K(x_i, x_j)]_{i,j=1}^{n}$ является PSD, то есть для любого $c \in \mathbb{R}^n$

$$c^{\top} K c \geq 0.$$

Практически это означает, что K соответствует некоторому скалярному произведению в некотор гильбертовом пространстве. Также полезны замкнутости: суммы, масштабирование и некоторые композиции допустимых ядер снова дают ядра.

Примеры ядер (часто используемые):

- **Линейное ядро:**

$$K(x, z) = x^{\top} z.$$

- **Полиномиальное ядро:**

$$K(x, z) = (\gamma x^{\top} z + r)^d, \qquad \gamma > 0, \; r \geq 0, \; d \in \mathbb{N}.$$

- **Гауссовское RBF-ядро:**

$$K(x, z) = \exp\left(-\gamma \|x - z\|_2^2\right), \qquad \gamma > 0.$$

- **Экспоненциальное (Laplacian) ядро:**

$$K(x, z) = \exp\left(-\gamma \|x - z\|_2\right), \qquad \gamma > 0.$$

22 Метод главных компонент (PCA)

22.1 Связь с SVD

Пусть дана матрица данных $X \in \mathbb{R}^{n \times d}$ (строки — объекты, столбцы — признаки). Стандартная постановка PCA предполагает **центровку** признаков:

$$X_c = X - \mathbf{1}\mu^\top, \qquad \mu = \frac{1}{n}\sum_{i=1}^{n} x_i,$$

где $\mathbf{1} \in \mathbb{R}^n$ — столбец из единиц, $x_i^\top$ — i-я строка X.

SVD разложение центрированной матрицы:

$$X_c = U\Sigma V^\top,$$

где $U \in \mathbb{R}^{n \times r}$, $V \in \mathbb{R}^{d \times r}$ имеют ортонормированные столбцы, $\Sigma = \mathrm{diag}(\sigma_1, \ldots, \sigma_r)$, $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0$, $r = \mathrm{rank}(X_c)$.

Связь PCA и SVD:

- **Главные направления (principal directions)** — это столбцы V (правые сингулярные векторы). j-я главная компонента соответствует направлению v_j.
- **Проекции объектов на компоненты (scores)**:
$$Z = X_c V \in \mathbb{R}^{n \times r}, \qquad Z = U\Sigma.$$
То есть координаты объектов в PCA-базисе равны $U\Sigma$.
- **Ковариационная матрица**:
$$C = \frac{1}{n-1} X_c^\top X_c.$$
Подстановка SVD даёт
$$C = \frac{1}{n-1} V\Sigma^2 V^\top,$$
то есть v_j — собственные векторы C, а собственные значения равны
$$\lambda_j = \frac{\sigma_j^2}{n-1}.$$
- **Объяснённая дисперсия.** Доля дисперсии, объяснённая k компонентами:
$$\mathrm{EVR}(k) = \frac{\sum_{j=1}^{k} \lambda_j}{\sum_{j=1}^{r} \lambda_j} = \frac{\sum_{j=1}^{k} \sigma_j^2}{\sum_{j=1}^{r} \sigma_j^2}.$$

Снижение размерности. Если берём первые k компонент ($k < r$), то

$$V_k = [v_1, \ldots, v_k] \in \mathbb{R}^{d \times k}, \qquad Z_k = X_c V_k \in \mathbb{R}^{n \times k}.$$

Реконструкция из k компонент:

$$\widehat{X}_c = Z_k V_k^\top = X_c V_k V_k^\top.$$

22.2 Теорема Эккарта–Юнга, доказательство

22.2.1 способ через собственное разложение или через подстановку

Теорема Эккарта–Юнга–Мирски (Eckart–Young–Mirsky). Пусть $A \in \mathbb{R}^{m \times n}$ и его SVD:

$$A = U\Sigma V^\top, \qquad \Sigma = \mathrm{diag}(\sigma_1, \ldots, \sigma_r), \quad \sigma_1 \geq \cdots \geq \sigma_r > 0.$$

Тогда **лучшая аппроксимация ранга** k в норме Фробениуса имеет вид усечённого SVD:

$$A_k = U_k \Sigma_k V_k^\top,$$

где U_k и V_k содержат первые k сингулярных векторов, а $\Sigma_k = \mathrm{diag}(\sigma_1, \ldots, \sigma_k)$. При этом

$$A_k \in \arg\min_{\mathrm{rank}(B) \leq k} \|A - B\|_F, \qquad \min_{\mathrm{rank}(B) \leq k} \|A - B\|_F^2 = \sum_{j=k+1}^{r} \sigma_j^2.$$

Доказательство (через подстановку и ортогональную инвариантность).

Шаг 1. Используем ортогональную инвариантность нормы Фробениуса: для любых ортогональны Q_1, Q_2 верно $\|Q_1 M Q_2\|_F = \|M\|_F$. Пусть $A = U\Sigma V^\top$. Для произвольного B ранга $\le k$ положим

$$C = U^\top B V.$$

Тогда $\operatorname{rank}(C) = \operatorname{rank}(B) \le k$ и

$$\|A - B\|_F = \|U^\top (A - B) V\|_F = \|\Sigma - C\|_F.$$

Следовательно,

$$\min_{\operatorname{rank}(B)\le k} \|A - B\|_F = \min_{\operatorname{rank}(C)\le k} \|\Sigma - C\|_F.$$

Шаг 2. Так как Σ диагональна, оптимально брать C тоже диагональной. Действительно, для $i \ne j$ имеем $\Sigma_{ij} = 0$, значит вклад внедиагональных элементов в ошибку равен

$$\sum_{i\ne j} (\Sigma_{ij} - C_{ij})^2 = \sum_{i\ne j} C_{ij}^2 \ge 0,$$

и минимален при $C_{ij} = 0$ для $i \ne j$.

Шаг 3. Пусть $C = \operatorname{diag}(c_1, \dots, c_r)$ и $\operatorname{rank}(C) \le k$ означает, что не более k диагональных элементов ненулевые. Тогда

$$\|\Sigma - C\|_F^2 = \sum_{j=1}^{r} (\sigma_j - c_j)^2.$$

Очевидно, если компонент c_j разрешено сделать ненулевым, то минимум по c_j достигается при $c_j = \sigma_j$ (даёт вклад 0). Если же c_j принудительно равен нулю, вклад равен σ_j^2. Чтобы минимизировать сумму, выгоднее "сохранить" (сделать ненулевыми) те k компонент, у которых σ_j^2 максимальны, то есть $j = 1, \dots, k$.

Значит оптимально:

$$C^\star = \operatorname{diag}(\sigma_1, \dots, \sigma_k, 0, \dots, 0),$$

и тогда

$$\min_{\operatorname{rank}(C)\le k} \|\Sigma - C\|_F^2 = \sum_{j=k+1}^{r} \sigma_j^2.$$

Шаг 4. Возвращаемся к B: берём

$$B^\star = U C^\star V^\top = U_k \Sigma_k V_k^\top = A_k.$$

Теорема доказана.

Следствие для PCA. При $A = X_c$ теорема говорит, что реконструкция $X_c V_k V_k^\top$ (усечённое SVD) минимизирует ошибку восстановления в $\|\cdot\|_F$ среди всех k-мерных линейных подпространс

22.3 Две альтернативные итеративные формулировки

PCA можно определить (и "строить") итеративно двумя эквивалентными способами.

(1) Жадный выбор направления максимальной дисперсии (greedy max-variance). Для центрированных данных X_c и ковариационной матрицы

$$C = \frac{1}{n-1} X_c^\top X_c$$

первая компонента определяется как

$$v_1 = \arg\max_{\|v\|_2=1} \operatorname{Var}(X_c v) = \arg\max_{\|v\|_2=1} v^\top C v.$$

Решение: v_1 — собственный вектор C с максимальным собственным значением λ_1.
Далее выбирают v_2 как направление максимальной дисперсии **при условии ортогональности** $v_2 \perp v_1$, и т.д. В результате получают ортонормированный базис $V_k = [v_1, \dots, v_k]$, который максимизирует суммарную дисперсию проекций:

$$V_k \in \arg\max_{V^\top V = I_k} \operatorname{tr}(V^\top C V) = \arg\max_{V^\top V = I_k} \sum_{j=1}^{k} v_j^\top C v_j = \sum_{j=1}^{k} \lambda_j.$$

Итеративная реализация через дефляцию (один из вариантов). После нахождения v_1 можно "удалить" компоненту из данных:

$$X^{(1)} = X_c - (X_c v_1) v_1^\top.$$

и затем повторить процедуру на $X^{(1)}$ для получения v_2, и т.д.

(2) Удаление направления минимальной дисперсии (remove min-variance).
Альтернативно можно мыслить PCA как процедуру "сжатия": из исходного d-мерного пространс
удаляются направления с наименьшей дисперсией (наименьшими λ_j).
Если $\{v_1, \dots, v_d\}$ — ортонормальный базис собственных векторов ковариационной матрицы C, упорядоченных по убыванию $\lambda_1 \geq \cdots \geq \lambda_d \geq 0$, то сохранение k компонент означает:

$$\text{удалить } v_{k+1}, \dots, v_d \quad \text{(минимальные дисперсии)}, \qquad \text{оставить } v_1, \dots, v_k.$$

Обе формулировки дают **то же самое подпространство** $\operatorname{span}(v_1, \dots, v_k)$, поскольку оно определяется первыми k собственными направлениями (или первыми k правыми сингулярными векторами X_c).

Практическая эвристика выбора k: часто выбирают k по порогу объяснённой дисперсии:

$$\min \left\{ k : \frac{\sum_{j=1}^{k} \lambda_j}{\sum_{j=1}^{r} \lambda_j} \geq \tau \right\}, \qquad \tau \in (0,1) \text{ например } 0.9 \text{ или } 0.95.$$

22.4 Предобработка для PCA

PCA чувствителен к масштабу признаков, поскольку основан на дисперсиях и ковариациях.

1) Центровка (обязательно для классического PCA). Всегда вычитают среднее по каждому признаку:

$$X_c = X - \mathbf{1}\mu^\top.$$

Без центровки первая компонента может отражать "смещение" (mean), а не структуру вариаций.

2) Нормализация/масштабирование (обычно необходимо). Если признаки имеют разны единицы измерения (метры, рубли, секунды и т.п.), то признак с большей дисперсией будет доминировать в PCA. Поэтому часто применяют:

- **StandardScaler (z-score):**

$$x_{ij}^{\text{std}} = \frac{x_{ij} - \mu_j}{s_j}, \qquad s_j = \sqrt{\frac{1}{n-1} \sum_{i=1}^{n} (x_{ij} - \mu_j)^2}.$$

- **MinMaxScaler:** приведение к отрезку $[0,1]$:

$$x_{ij}^{\text{mm}} = \frac{x_{ij} - \min_i x_{ij}}{\max_i x_{ij} - \min_i x_{ij}}.$$

Выбор скейлера зависит от задачи; чаще всего для PCA по умолчанию используют стандартизац

3) Пропуски и выбросы. Классический PCA требует числовую матрицу без NaN, поэтому на практике перед PCA делают импутацию. При сильных выбросах PCA может искажаться; тогда используют робастные методы масштабирования или робастные аналоги PCA.

4) Категориальные признаки. Категориальные признаки обычно переводят в one-hot, и только затем (при необходимости) применяют PCA к полученным столбцам, понимая, что интерпретация компонент в таком случае становится менее прямолинейной.

Практическая ремарка (о корректном протоколе). Все параметры предобработки (средне стандартное отклонение, min/max) должны вычисляться *только на train*, а затем применяться к validation/test, чтобы избежать утечек данных.

23 k ближайших соседей (kNN) для классификации и регрессии

23.1 Параметры и гиперпараметры

kNN (*k Nearest Neighbors*) — **непараметрическая** модель: в ней **нет обучаемых параметров** вида w, b, а "обучение" по сути сводится к **сохранению обучающей выборки** (возможно с предварительной индексацией), после чего предсказания строятся по ближайшим объектам.

Базовый алгоритм предсказания. Пусть дана обучающая выборка $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$ и метрика (функция расстояния) $d(\cdot, \cdot)$. Для нового объекта x:

1. вычисляем расстояния $d(x, x_i)$ до всех (или до кандидатов из индекса) объектов обучающей выборки;
2. выбираем множество индексов $N_k(x)$ — k ближайших соседей:
$$N_k(x) \in \arg \min_{\substack{S \subseteq \{1,\dots,n\} \\ |S|=k}} \sum_{i \in S} d(x, x_i);$$
3. агрегируем ответы $\{y_i\}_{i \in N_k(x)}$.

Агрегация для классификации. В простейшем варианте используется **голосование больш**
$$\hat{y}(x) = \arg\max_{c \in \mathcal{C}} \sum_{i \in N_k(x)} \mathbb{I}\{y_i = c\},$$
где $\mathcal{C}$ — множество классов.
Иногда используют **взвешенное голосование**:
$$\hat{y}(x) = \arg\max_{c \in \mathcal{C}} \sum_{i \in N_k(x)} w_i(x)\, \mathbb{I}\{y_i = c\},$$
где веса $w_i(x)$ могут зависеть от *порядка соседа* или от *самого расстояния* (например, $w_i(x) = 1/(d(x, x_i) + \varepsilon)$).

Агрегация для регрессии. Типичный ответ — **среднее** по соседям:
$$\hat{y}(x) = \frac{1}{k} \sum_{i \in N_k(x)} y_i,$$
или **взвешенное среднее**:
$$\hat{y}(x) = \frac{\sum_{i \in N_k(x)} w_i(x)\, y_i}{\sum_{i \in N_k(x)} w_i(x)}.$$
(Аналогично можно использовать медиану, если нужно повысить устойчивость к выбросам в y.)

Параметры и гиперпараметры kNN.

- **Параметры (trainable parameters): отсутствуют.** Это соответствует тому, что kNN — **non-parametric model**: значения параметров не находятся решением задачи оптимизаци
- **Гиперпараметры (задаются до применения модели):**
  1. k — **число соседей.** Малые k делают модель более "локальной" (обычно выше вариативность ответов), большие k — более "гладкой" (обычно сильнее усреднение и выше смещение). Частный случай $k = 1$ даёт максимально локальное решение.
  2. **Функция расстояния** $d(\cdot, \cdot)$**.** На практике выбор метрики критичен, так как именно она определяет понятие "близости":
     - Евклидово расстояние (L_2): $d(x, z) = \|x - z\|_2$;
     - Манхэттенское расстояние (L_1): $d(x, z) = \|x - z\|_1$;
     - расстояния Минковского: $d(x, z) = \|x - z\|_p$ (гиперпараметр p);
     - косинусная "дистанция" (через косинусное сходство);
     - Хэммингово расстояние (число несовпадений/замен) и др.

3. **Схема взвешивания соседей** (используется существенно реже базового голосования, веса могут зависеть от ранга соседа или от расстояния до соседа.

Замечание про "гиперпараметры реализации" (скорость/память). В прикладных системах для ускорения поиска ближайших соседей часто вводят индексные структуры и приближённый поиск (ANN), и тогда добавляются настройки, влияющие прежде всего на вычислительную эффективность (например, выбор структуры индекса и её параметры), но статистическая постановка kNN при этом остаётся той же: определить $N_k(x)$ по выбранной метрике и агрегировать ответы.

24 Индексы для kNN

24.1 Зачем нужны индексы

В kNN (*k Nearest Neighbors*) предсказание для нового объекта x сводится к поиску k ближайших точек по выбранной метрике $d(\cdot,\cdot)$ среди обучающей выборки $\{x_i\}_{i=1}^n$:

$$N_k(x) = \arg \min_{\substack{S\subset\{1,\dots,n\} \\ |S|=k}} \sum_{i\in S} d(x, x_i).$$

Наивный поиск требует вычислить расстояния до всех n объектов и выбрать k минимумов:

время $\approx O(nd)$ на вычисление расстояний $+$ $O(n)$ или $O(n \log n)$ на отбор,

где d — размерность признакового пространства (для плотных векторов).

Зачем нужны индексы:

- **Ускорение инференса.** При больших n (и/или высокой частоте запросов) перебор становится узким местом.
- **Снижение вычислительных затрат.** Индекс позволяет резко уменьшить число кандида для которых считаются точные расстояния.
- **Практика high-dimensional data.** В рекомендательных системах, семантическом поиске, CV/NLP часто работают с эмбеддингами $d \sim 128$–4096 и n до миллионов; тогда используют **приближённый** поиск (ANN), чтобы получить приемлемую задержку.
- **Компромисс точность–скорость.** Индексы позволяют управлять trade-off: увеличивать recall@k ценой времени или наоборот.

Важная ремарка. Для высоких размерностей действует "проклятие размерности": дерево- и геометрические индексы в худшем случае деградируют до перебора (или дают слабое ускорение) поэтому на практике часто переходят к ANN-структурам (графы/LSH/квантование/проекционн деревья).

24.2 Идеи индексов для kNN от простых к сложным

Ниже — типовая эволюция подходов к ускорению kNN:

1. **Брутфорс + микрооптимизации.**
   - векторизация вычислений (SIMD), батчинг запросов;
   - ускорение за счёт BLAS/матричного умножения (особенно для косинусной близости);
   - предварительная нормализация векторов для косинусной близости: $\cos(x,z) = \frac{x^\top z}{\|x\|\|z\|}$.
2. **Сокращение числа кандидатов через "фильтры".**
   - грубая фильтрация по категориям/сегментам/времени;
   - двухступенчатая схема retrieval → rerank: сначала быстрый кандидат-генератор (индекс затем точное пересчетное ранжирование по исходной метрике.
3. **Точные геометрические индексы (для малых/умеренных d).**
   - **kd-tree** (разбиение пространства по координатам);
   - ball-tree / vantage-point tree (разбиение по радиусам/сферам);
   - эффективность резко падает при росте размерности.
4. **Хеширование и проекции (Approximate).**
   - LSH (Locality-Sensitive Hashing): близкие точки попадают в одинаковые бакеты с высокой вероятностью;
   - random projection trees / forests (семейство методов, где пространство режется случайны гиперплоскостями).

5. **Графовые ANN-индексы (state-of-the-practice).**
   - kNN-graph / navigable small world graphs;
   - **HNSW** (иерархический граф малых миров): один из наиболее популярных вариантов по качеству/скорости на эмбеддингах.
6. **Квантование и компрессия (для очень больших n).**
   - product quantization (PQ), IVF+PQ, scalar quantization;
   - цель — уменьшить память и ускорить приблизительные расстояния, часто в сочетании с грубой кластеризацией (IVF).

24.3 k-dimensional tree (kd-tree)

kd-tree — бинарное дерево разбиений пространства $\mathbb{R}^d$ по осям координат. Индекс эффективен в **низкой/умеренной размерности** (типично до нескольких десятков признаков), а в высокой размерности часто деградирует.

Построение. На каждом узле выбирают:

- ось разбиения $j \in \{1, \dots, d\}$ (часто циклически или по максимальной дисперсии);
- порог t (часто медиана по координате j), чтобы сбалансировать дерево.

Данные делятся на две части:

$$\{x : x_j \le t\} \quad \text{и} \quad \{x : x_j > t\}.$$

При выборе медианы глубина дерева порядка $O(\log n)$, а построение порядка $O(n \log n)$ (в типовой реализации).

Поиск ближайшего соседа / kNN. Для запроса x:

1. спускаются по дереву в "ближайшую" ветвь по координате разбиения;
2. в листе получают кандидат(ов) и инициализируют текущий лучший радиус r (или структур top-k);
3. выполняют **backtracking**: поднимаются вверх и посещают альтернативную ветвь, только если гиперплоскость разбиения может содержать точки ближе текущего радиуса, т.е.
$$|x_j - t| < r.$$

Сильные и слабые стороны kd-tree:

- **Плюсы:** точный поиск, хорошая скорость при малом d, понятная геометрия, простота.
- **Минусы:** при росте d растёт число посещаемых узлов, эффективность падает (часто приближается к перебору), чувствителен к распределению данных.

24.4 Annoy

Annoy (*Approximate Nearest Neighbors Oh Yeah*) — ANN-индекс, основанный на **лесах деревьев случайных проекций** (*random projection trees*). Ключевая идея: строится множество деревьев, каждое из которых рекурсивно разрезает пространство случайными гиперплоскостями; затем результаты деревьев объединяются.

Идея построения (высокоуровнево). Для каждого дерева в узле выбирают случайное разбиение пространства гиперплоскостью. Один из распространённых вариантов: выбрать две точки a, b и разделять по гиперплоскости, перпендикулярной $(a - b)$ (по сути, случайная проекция и порог). Далее рекурсивно строят поддеревья до тех пор, пока в листе не останется малое число объектов.

Поиск.

- Для запроса x проходят по каждому дереву до листа (получая кандидатов).

- Объединяют кандидатов из нескольких деревьев и пересчитывают точные расстояния до них.
- Возвращают top-k ближайших по точной метрике среди кандидатов.

Гиперпараметры качества/скорости (типовые):

- число деревьев T: больше $T \Rightarrow$ лучше recall, но выше память и время построения;
- параметр "глубины/усилия поиска" (сколько узлов/листьев просматривать): больше усилие $\Rightarrow$ выше recall, но выше latency.

Особенности:

- хорошо подходит для **статических** коллекций ("построил и читаешь");
- удобен для косинусной близости (через нормировку) и евклидовой метрики;
- в целом реализует retrieval-candidate $\rightarrow$ exact rerank.

24.5 HNSW

HNSW (*Hierarchical Navigable Small World*) — графовый ANN-индекс. Он хранит **многослойн** граф: на верхних слоях мало вершин и длинные "прыжки", на нижних слоях много вершин и локальные связи. Это обеспечивает быстрый навигационный поиск.

24.5.1 Построение графа

Пусть есть точки $\{x_i\}_{i=1}^n$ и метрика/дистанция $d(\cdot,\cdot)$. Каждая точка получает **слой** (уровень) $L(i) \in \{0,1,2,\dots\}$, где слой 0 — самый плотный. Обычно уровни распределяют так, чтобы большинство точек были на нижнем уровне, а на верхних – всё меньше.

Вставка новой вершины x (общая схема):

1. **Выбор уровня.** Сэмплируется максимальный уровень $L(x)$ (вершина присутствует на уровнях $0,\dots,L(x)$).
2. **Поиск точки входа.** Начиная с текущей точки входа (*entry point*) на верхнем уровне, выполняют **жадный поиск** (greedy): переходят к соседу, который ближе к x, пока улучшений нет.
3. **Спуск по уровням.** Повторяют жадный поиск на каждом уровне сверху вниз, получая "приближение" к области x.
4. **Формирование ребёр на уровнях $\le L(x)$.** На каждом уровне находят кандидатов (через локальный поиск с параметром efConstruction), выбирают до M соседей и добавляют рёбра между x и выбранными вершинами. Часто применяют **эвристику разнообразия** (diversification), чтобы не соединять только очень близкие (почти коллинеарные) точки, улучшая навигацию графа.

Ключевые гиперпараметры HNSW:

- M — максимальное число рёбер (соседей) на уровне для вершины;
- efConstruction — "ширина" поиска кандидатов при построении (влияет на качество графа и время построения);
- efSearch — "ширина" поиска при запросе (влияет на recall и latency).

24.5.2 Использование графа

Поиск k ближайших соседей для запроса q:

1. **Верхние уровни (быстрый навигатор).** Начинают с точки входа на самом верхнем уровне и выполняют **жадный спуск**: переходят к соседу, уменьшающему расстояние до q, пока улучшений нет; затем переходят на уровень ниже.
2. **Нижний уровень (точная локализация).** На уровне 0 выполняют **best-first** поиск с ограничением размера кандидатов efSearch: поддерживают множество кандидатов и множество лучших найденных, расширяют ближайшие кандидаты, пока критерий остановк не выполнен.
3. **Ответ.** Возвращают k ближайших среди найденных (по точной метрике).

Свойства HNSW (практические):

- очень хорошее отношение **качество (recall) / скорость** для высокоразмерных эмбеддинго
- параметр efSearch позволяет плавно регулировать trade-off: больше efSearch $\Rightarrow$ выше recall, но выше latency;
- хорошо поддерживает **инкрементальные вставки** (в отличие от некоторых методов, ориентированных на оффлайн построение).

24.6 Примеры векторных баз данных

Векторная БД (vector database) — система хранения эмбеддингов (векторов) и метаданных, которая предоставляет:

- поиск ближайших соседей (обычно ANN) по метрике (cosine / L2 / inner product);
- фильтрацию по метаданным (hybrid: “фильтр + ANN”);
- управление коллекциями, репликацию/шардинг, обновления, часто — API для retrieval в прикладных системах (например, RAG).

Примеры (популярные семейства решений):

- специализированные vector DB: **Milvus**, **Qdrant**, **Weaviate**, **Pinecone**, **Chroma**, **LanceDl**
- расширения к классическим хранилищам: **PostgreSQL + pgvector**, **Elasticsearch/Open kNN**, **Redis (Vector/RediSearch)**;
- библиотеки/движки ANN (не БД, но часто лежат внутри): **FAISS**, **ScaNN**, реализации **HNSW**.

25 The bitter lesson (Горький урок)

25.1 Основные выводы статьи

The Bitter Lesson — эссе Рича Саттона (2019), где он формулирует исторически повторяющийся вывод из десятилетий развития ИИ: в долгосрочной перспективе **побеждают общие методы**, которые **масштабируются по вычислениям и данным**, а не подходы, основанные на тщательном **встраивании человеческих знаний о предметной области**.

1. **Главный тезис: "масштабируемые общие методы" в итоге выигрывают.**
   Во многих направлениях ИИ наблюдался один и тот же паттерн: сначала доминируют решения, насыщенные человеческой экспертизой (правила, эвристики, ручные признаки, специальные структуры), но затем их обгоняют более общие подходы, которые лучше используют рост вычислительных мощностей и объёмов данных.

2. **Почему "урок горький": антропоцентрические решения психологически привлек но стратегически проигрывают.**
   Инженеру и исследователю приятно строить систему "как человек думает":

   - это даёт ощутимый **быстрый прогресс** на малых масштабах;
   - система может казаться **интерпретируемой** и "разумной";
   - но такие решения часто становятся **потолком**: добавление новых эвристик усложняет систему, делает её хрупкой и не позволяет эффективно конвертировать рост compute/data в рост качества.

3. **Критерий жизнеспособности метода: он обязан улучшаться при росте compute и data.**
   Саттон подчёркивает, что методы, которые:

   - **получают выгоду от большего числа вычислений** (например, более длинный поиск, больше шагов оптимизации, более крупные модели, более длинное обучение),
   - **получают выгоду от большего объёма данных** (обучение представлений, обобщен на новых примерах),

   имеют систематическое преимущество: их качество растёт естественным образом по мере развития железа и накопления данных, тогда как "ручные" улучшения имеют убывающую отдачу и быстро устаревают.

4. **Два класса общих методов, которые исторически лучше всего масштабируются: search и learning.**
   В терминах эссе:

   - **Search (поиск):** систематический перебор/планирование/оптимизация, где больше вычислений $\Rightarrow$ глубже/шире поиск $\Rightarrow$ лучшее решение.
   - **Learning (обучение):** методы, которые автоматически извлекают закономерности из данных (например, градиентное обучение параметрических моделей), где больше данных и шагов обучения $\Rightarrow$ лучшее качество.

   Важно, что эти подходы не завязаны на конкретную предметную область и потому повторно применимы.

5. **Исторические примеры (как иллюстрация повторяемости урока).**
   В разных доменах (игры, речь, зрение и др.) итоговый скачок качества обычно достигался не за счёт усложнения доменных правил, а за счёт методов, которые лучше масштабируются

   - переход от "умных" ручных эвристик к методам, активно использующим вычисления (поиск/оптимизацию);
   - переход от ручных признаков к обучаемым представлениям на больших датасетах;
   - рост роли аппаратного ускорения и параллельных вычислений.

6. **Практический вывод для ML-инженера: проектируй решения так, чтобы они масштабировались.**
   При выборе архитектуры/алгоритма и постановке экспериментов полезно задавать вопросы

- Что произойдёт с качеством, если увеличить датасет в 2–10 раз?
- Что произойдёт, если увеличить бюджет обучения (эпохи, параметры модели, ширину поиска)?
- Не ограничивает ли решение заранее "зашитая" структура (эвристики/правила/ручные фичи) способность модели учиться?
- Можно ли автоматизировать извлечение признаков и улучшение поведения через обучение?

7. **Баланс: доменные знания полезны, но их роль должна быть "поддерживающей", а не "несущей".**
   Доменные знания оправданы как:

   - инженерные ограничения (безопасность, латентность, стоимость),
   - корректная постановка задачи, метрики, протокол валидации,
   - разумные индуктивные смещения, которые **не ломают масштабирование**.

   Но ставка на "ручную имитацию мышления" как основной источник прогресса чаще всего ведёт к тупику.

26 Деревья решений

26.1 Определение жадных алгоритмов

Жадный алгоритм (greedy algorithm) — это алгоритм, который на каждом шаге выбирает *локально оптимальное* решение (локально лучший шаг) в надежде получить хорошее (иногда оптимальное) глобальное решение. Жадная стратегия часто не гарантирует глобального оптимума, но даёт разумное приближение за приемлемое время.

26.2 Процедура построения

Рассмотрим датасет $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$, где $x_i \in \mathbb{R}^d$. Дерево решений строится рекурсивно как последовательность разбиений множества объектов на подмножества.

Типичное бинарное разбиение (CART-подход). Для признака $j \in \{1, \ldots, d\}$ и порога $t \in \mathbb{R}$ задаём предикат

$$\phi_{j,t}(x) = \mathbb{I}[x_j \le t],$$

который делит множество объектов Q в текущем узле на левую и правую части:

$$L(j,t) = \{(x_i, y_i) \in Q : x_{ij} \le t\}, \qquad R(j,t) = \{(x_i, y_i) \in Q : x_{ij} > t\}.$$

Жадный шаг. На каждом узле выбирают пару $(j^\star, t^\star)$, оптимизирующую критерий качества разбиения (см. ниже), после чего рекурсивно строят поддеревья для $L(j^\star, t^\star)$ и $R(j^\star, t^\star)$.

Остановка (pre-pruning). Рекурсия завершается, если выполнено одно из условий (типовые):

- достигнута максимальная глубина;
- в узле слишком мало объектов (ниже порога);
- улучшение по критерию разбиения меньше заданного порога;
- узел стал "чистым" (например, все y_i одного класса) или почти чистым.

Поиск оптимального разбиения. Так как множество кандидатных разбиений дискретно (выбор признака и порога), напрямую применять градиентный спуск нельзя. Полный перебор всех порогов для всех признаков может быть дорогим, поэтому на практике:

- используют эффективные алгоритмы перебора порогов по отсортированным значениям признака;
- либо ограничивают множество порогов (бинаризация, random splits и т.п.).

Замечание о масштабировании признаков. В отличие от линейных моделей, деревья обычно *слабо чувствительны* к нормировке признаков, поскольку выбирают разбиения по порядку значений и "объёмам" в пространстве признаков.

26.3 Информационные критерии, энтропия, индекс Джини

Пусть в узле Q имеется n объектов, а после разбиения — n_L и n_R объектов. Обозначим "неоднородность" (impurity) множества S как $H(S)$. Тогда качество разбиения удобно измерять через **уменьшение неоднородности** (information gain / impurity decrease):

$$\Delta H(j,t) = H(Q) - \frac{n_L}{n} H(L(j,t)) - \frac{n_R}{n} H(R(j,t)).$$

Жадный выбор:

$$(j^\star, t^\star) \in \arg\max_{j,t} \ \Delta H(j,t).$$

Классификация (мультикласс). Пусть K классов, а $p_k(S)$ — доля класса k среди объектов множества S:

$$p_k(S) = \frac{1}{|S|} \sum_{(x_i, y_i) \in S} \mathbb{I}[y_i = k], \qquad k = 1, \ldots, K.$$

Три распространённых критерия неоднородности:

1. **Misclassification error:**
$$H_{\text{mis}}(S) = 1 - \max_k p_k(S).$$

2. **Энтропия (entropy):**
$$H_{\text{ent}}(S) = -\sum_{k=1}^{K} p_k(S) \log p_k(S),$$
где обычно считают $0 \log 0 = 0$.

3. **Индекс Джини (Gini impurity):**
$$H_{\text{gini}}(S) = 1 - \sum_{k=1}^{K} p_k(S)^2.$$

Обычно H_{ent} и H_{gini} дают более "чувствительные" разбиения, чем H_{mis}.

Регрессия. В регрессионных деревьях $H(S)$ задают как меру разброса целевой переменной в узле:

- для MSE-ориентированных деревьев:
$$H_{\text{MSE}}(S) = \frac{1}{|S|} \sum_{(x_i,y_i)\in S} (y_i - \bar{y}_S)^2, \qquad \bar{y}_S = \frac{1}{|S|} \sum_{(x_i,y_i)\in S} y_i;$$

- для MAE-ориентированных деревьев (робастнее к выбросам) часто используют абсолютные отклонения относительно медианы (см. также ответ в листе).

26.4 Ответ в листе

Каждый лист соответствует некоторой области пространства признаков (узлу) S. Ответ в листе выбирают как **константу**, оптимальную для выбранной функции потерь.

Классификация. Типовые варианты:

- **предсказать самый частый класс:**
$$\hat{y}(x) = \arg\max_k p_k(S);$$

- **предсказать распределение классов** (вероятности в листе):
$$\hat{p}_k(x) = p_k(S),$$
а метку брать как $\arg\max_k \hat{p}_k(x)$.

Регрессия.

- При MSE оптимальная константа — **среднее** в узле:
$$\hat{y}(x) = \bar{y}_S.$$

- При MAE оптимальная константа — **медиана** в узле:
$$\hat{y}(x) = \text{median}\{y_i : (x_i, y_i) \in S\}.$$

26.5 Обработка пропусков

Пропуски ($x_{ij} = \text{NaN}$) требуют явного решения, так как разбиение вида $x_j \le t$ для NaN неопределено. Распространённые стратегии:

1. **Импутация до обучения дерева** (mean/median/модельная иммпутация) и далее стандар обучение.

2. **Отдельная ветка/категория для NaN** (в некоторых реализациях): "NaN $\to$ left" или "NaN $\to$ right".

3. **C4.5-подход: распределить объект по обеим веткам.** Если значение отсутствует, объект "идёт" и в левое, и в правое поддерево, а предсказания агрегируются (например, усредняются, возможно с весами, пропорциональными долям объектов в ветвях).

4. **CART: surrogate splits (суррогатные предикаты).** Для основного разбиения $(j^\star, t^\star)$ подбирают альтернативные разбиения по другим признакам, которые лучше всего имитиру исходное разделение на L/R; при пропуске по $j^\star$ используют суррогат.

Практическая ремарка. В библиотеках градиентного бустинга (CatBoost, LightGBM и др.) часто есть встроенные правила обработки NaN (например, "NaN как минимум" или "NaN как максимум"), но это уже специфика конкретных реализаций деревьев в ансамблях.

26.6 Бинаризация

Бинаризация (thresholding / discretization) — приём, когда вместо выбора *одного* порога для признака заранее задают *набор* порогов (границ), превращая непрерывный признак в набор бинарных признаков/кандидатных предикатов.

Формально: для признака x_j выбирают пороги $t_{j,1} < t_{j,2} < \cdots < t_{j,B}$ и рассматривают семейство предикатов

$$\phi_{j,b}(x) = \mathbb{I}[x_j \le t_{j,b}], \qquad b = 1, \ldots, B.$$

Тогда в узле дерево выбирает лучшее разбиение среди конечного множества $\{\phi_{j,b}\}$. Это уменьша вычислительную сложность выбора порога и стабилизирует обучение. (В CatBoost аналог — *border count*: количество границ для признака.)

26.7 Обрезка (pruning), варианты C4.5 и CART

Pruning (обрезка) — упрощение дерева после первоначального жадного построения с целью уменьшить переобучение (снизить дисперсию, убрать слабополезные ветвления).

C4.5: pessimistic pruning (пессимистическая обрезка). Идея: оценить "истинную" ошибку поддерева и листа не по train-ошибке напрямую, а через верхнюю границу ошибки (например, используя биномиальный доверительный интервал), предполагая, что наблюдаемые ошибки — это выборка, а реальная ошибка может быть выше. Затем:

- вычисляют оценку (верхнюю границу) ошибки поддерева;
- вычисляют оценку ошибки, если заменить поддерево одним листом;
- если "лист" предпочтительнее, поддерево заменяют листом.

CART: cost-complexity pruning (обрезка по цене сложности). Вводится функционал качества дерева T:

$$R_\alpha(T) = R(T) + \alpha|T|,$$

где $R(T)$ — ошибка (или риск) дерева на данных (в зависимости от постановки), $|T|$ — число листьев (или узлов), $\alpha \ge 0$ — коэффициент штрафа за сложность. Алгоритм формирует **последовательность** деревьев при возрастающих α:

- для каждого внутреннего узла оценивают, при каком α выгоднее "схлопнуть" его поддерево в лист ("эффективная" α для узла);
- выбирают минимальную такую α и выполняют соответствующую обрезку;
- повторяют, получая путь $T_0 \supset T_1 \supset \cdots \supset T_m$;
- финальный выбор дерева делают по validation (или CV) по качеству.

26.8 Гиперпараметры (минимум 4 основных)

К основным гиперпараметрам деревьев (типично как в sklearn/CART-подобных реализациях) относят:

- **max_depth** — максимальная глубина дерева (ограничивает сложность; форма pre-pruning)
- **min_samples_split** — минимальное число объектов в узле, чтобы пытаться делать разбиение;

- **min_samples_leaf** — минимальное число объектов в листе (защита от слишком “узких” листьев);
- **min_impurity_decrease** — минимальное уменьшение неоднородности ΔH, чтобы принят разбиение;
- **criterion** — критерий неоднородности:
  - классификация: `gini`, `entropy` (иногда `log_loss`);
  - регрессия: `squared_error` (MSE), `absolute_error` (MAE) и др.
- **splitter** — стратегия выбора сплита (`best` vs `random`);
- **max_features** — число признаков, доступных при выборе разбиения (часто `sqrt`, `log2`, или доля), что используется и как форма регуляризации/индуктивного смещения.

27 Разложение bias–variance–noise

27.1 Случайность в ошибке

27.1.1 Интуиция каждой компоненты

Рассмотрим постановку регрессии с аддитивным шумом:

$$Y = f(X) + \varepsilon, \qquad \mathbb{E}[\varepsilon \mid X] = 0, \qquad \mathrm{Var}(\varepsilon \mid X) = \sigma^2(X).$$

Пусть обучающий алгоритм, получив случайную обучающую выборку $\mathcal{D}$, возвращает предиктор $\hat{f}_{\mathcal{D}}(\cdot)$. Тогда **ошибка предсказания** на фиксированном x:

$$(Y - \hat{f}_{\mathcal{D}}(x))^2$$

является случайной величиной из-за нескольких источников случайности.

Источники случайности (почему ошибка случайна):

- **Случайность обучения/алгоритма:** случайная инициализация, стохастические оптими (SGD и мини-батчи), случайные сплиты/подвыборки признаков в деревьях и ансамблях и т.п.
- **Численная/вычислительная:** конечная точность вычислений, приближения, недетерми вычислений на GPU и т.п.
- **Шум в целевой переменной:** собственно ε (ошибка измерения, случайные факторы, неполнота признаков).
- **Сэмплирование объектов:** обучающая выборка $\mathcal{D}$ — случайная подвыборка из популяци также случайны разбиения train/val/test.
- **Человеческий фактор:** разметка, субъективные правила, неоднозначность классов, дрейф требований.

Чтобы разделить вклад разных причин, вводят **центральную (усреднённую) модель**:

$$\bar{f}(x) = \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)].$$

Тогда для фиксированного x определяют:

$$\mathrm{Bias}(x) = \bar{f}(x) - f(x), \qquad \mathrm{Var}(x) = \mathbb{E}_{\mathcal{D}}\big[(\hat{f}_{\mathcal{D}}(x) - \bar{f}(x))^2\big], \qquad \mathrm{Noise}(x) = \sigma^2(x).$$

Интуиция:

- **Bias (смещение)** — насколько в среднем модель систематически ошибается относительно истинной зависимости $f(x)$. Высокий bias обычно связан с недостаточной гибкостью модели (underfitting).
- **Variance (разброс)** — насколько сильно ответы модели меняются при смене обучающей выборки $\mathcal{D}$ (и/или стохастики обучения). Высокая variance обычно связана с чрезмерной гибкостью (overfitting).
- **Noise (неустранимая ошибка)** — неопределённость в Y даже при знании X:

  $$\mathrm{Noise}(x) = \mathrm{Var}(Y \mid X = x).$$

  Её нельзя убрать выбором более сложной модели; уменьшается только за счёт улучшения качества данных/признаков/разметки (или изменения постановки задачи).

27.2 Доказательство для MSE

Рассмотрим **условный MSE** на фиксированном x:

$$\mathbb{E}_{\mathcal{D},\varepsilon}\big[(Y - \hat{f}_{\mathcal{D}}(x))^2 \mid X = x\big].$$

Подставим $Y = f(x) + \varepsilon$:

$$\mathbb{E}_{\mathcal{D},\varepsilon}\big[(f(x) + \varepsilon - \hat{f}_{\mathcal{D}}(x))^2\big] = \mathbb{E}_{\mathcal{D},\varepsilon}\big[(f(x) - \hat{f}_{\mathcal{D}}(x))^2\big] + 2\,\mathbb{E}_{\mathcal{D},\varepsilon}\big[\varepsilon(f(x) - \hat{f}_{\mathcal{D}}(x))\big] + \mathbb{E}_{\varepsilon}[\varepsilon^2].$$

Так как $\mathbb{E}[\varepsilon \mid X = x] = 0$ и обычно предполагают независимость шума от процедуры обучения (при фиксированном x), средний перекрёстный член равен нулю:

$$\mathbb{E}_{\mathcal{D},\varepsilon}\big[\varepsilon(f(x) - \hat{f}_{\mathcal{D}}(x))\big] = \mathbb{E}_{\mathcal{D}}\Big[(f(x) - \hat{f}_{\mathcal{D}}(x))\,\mathbb{E}_{\varepsilon}[\varepsilon]\Big] = 0.$$

Кроме того,

$$\mathbb{E}_\varepsilon[\varepsilon^2 \mid X = x] = \mathrm{Var}(\varepsilon \mid X = x) = \sigma^2(x).$$

Остаётся разложить первый член по формуле "дисперсия + квадрат смещения":

$$\mathbb{E}_\mathcal{D}\big[(f(x) - \hat{f}_\mathcal{D}(x))^2\big] = \mathbb{E}_\mathcal{D}\big[(\hat{f}_\mathcal{D}(x) - \bar{f}(x) + \bar{f}(x) - f(x))^2\big].$$

Раскроем квадрат:

$$= \mathbb{E}_\mathcal{D}\big[(\hat{f}_\mathcal{D}(x) - \bar{f}(x))^2\big] + 2(\bar{f}(x) - f(x))\,\mathbb{E}_\mathcal{D}[\hat{f}_\mathcal{D}(x) - \bar{f}(x)] + (\bar{f}(x) - f(x))^2.$$

Так как $\mathbb{E}_\mathcal{D}[\hat{f}_\mathcal{D}(x) - \bar{f}(x)] = 0$, получаем:

$$\mathbb{E}_\mathcal{D}\big[(f(x) - \hat{f}_\mathcal{D}(x))^2\big] = \underbrace{\mathbb{E}_\mathcal{D}\big[(\hat{f}_\mathcal{D}(x) - \bar{f}(x))^2\big]}_{\mathrm{Var}(x)} + \underbrace{(\bar{f}(x) - f(x))^2}_{\mathrm{Bias}(x)^2}.$$

Итого:

$$\boxed{\mathbb{E}_{\mathcal{D},\,\varepsilon}\big[(Y - \hat{f}_\mathcal{D}(x))^2 \mid X = x\big] = \mathrm{Bias}(x)^2 + \mathrm{Var}(x) + \sigma^2(x).}$$

Для полной (по распределению X) ошибки обобщения берут ожидание по X:

$$\mathbb{E}_X\,\mathbb{E}_{\mathcal{D},\,\varepsilon}\big[(Y - \hat{f}_\mathcal{D}(X))^2 \mid X\big] = \mathbb{E}_X[\mathrm{Bias}(X)^2] + \mathbb{E}_X[\mathrm{Var}(X)] + \mathbb{E}_X[\sigma^2(X)].$$

27.3 Компромисс bias–variance

27.3.1 Анализ основных моделей

Классический **компромисс bias–variance** проявляется при увеличении "сложности" модели (ёмкости гипотез):

- при росте сложности обычно **bias убывает** (модель гибче, лучше аппроксимирует f);
- но **variance растёт** (модель сильнее подстраивается под конкретную выборку/шум).

Из-за этого тестовая ошибка как функция сложности часто имеет U-образный характер (underfitting → оптимум → overfitting).

Где на шкале bias–variance находятся базовые модели (типичная картина):

Модель	Bias (смещение)	Variance (разброс)
Линейные модели (Linear/LogReg)	Часто **высокий**, если истинная зависимость нелинейна и нет нужных признаков/взаимодействий	Обычно **низкий** (простая модель, устойчивость)
kNN	При малом k bias низкий (очень локально), при большом k bias растёт (усреднение)	При малом k variance **высокий**, при росте k variance падает
Деревья решений	При глубоком дереве bias может быть низким (почти интерполяция), при маленькой глубине bias растёт	Глубокие деревья обычно имеют **высокий variance** (чувствительность к данным)
Bagging / Random Forest	Bias близок к bias базовой модели (дерева), может чуть вырасти при ограничениях	Variance **существенно снижается** за счёт усреднения слабокоррелированных моделей
Boosting (градиентный бустинг)	Bias обычно **сильно снижается** (последовательная коррекция ошибок)	Variance может расти, но контролируется регуляризацией (shrinkage, depth, subsampling)
Нейросети (современные)	При достаточной параметризации bias низкий	Variance зависит от режима (данные/регуляризация/оптимизаци возможны эффекты beyond классического trade-off

Как управлять компромиссом на практике (примеры):

- Ridge/Lasso: увеличение $\lambda \Rightarrow$ bias ↑, variance ↓.
- kNN: увеличение $k \Rightarrow$ bias ↑, variance ↓.
- Деревья: ограничение `max_depth`, `min_samples_leaf` ⇒ bias ↑, variance ↓.

- Bagging/RF: увеличение числа деревьев M и снижение корреляции (bootstrap + random subspace) $\Rightarrow$ variance $\downarrow$.
- Boosting: уменьшение learning rate и глубины деревьев часто снижает variance при сохранени способности уменьшать bias.

27.4 За пределами MSE

Для MSE разложение имеет красивую форму

$$\text{Risk} = \text{Bias}^2 + \text{Var} + \text{Noise},$$

но для других функций потерь (0–1 loss, log-loss/cross-entropy, margin losses и т.д.) ситуация сложнее:

- в классификации **ошибка дискретна** (0–1), и понятия bias/variance зависят от способа разложения;
- для **асимметричных стоимостей ошибок** или сложных метрик/лоссов появляются дополнительные члены (например, связанные с неоднозначностью решения).

Тем не менее существуют **обобщённые разложения риска** для широких классов лоссов:

- идея: выделить **риск центральной модели** ("expected/central predictor") и **термин вариативности** (насколько решения отличаются между обучающими выборками);
- для 0–1 loss и других случаев применяют "unified" подходы, где bias/variance определяются через наиболее вероятный/оптимальный ответ и частоту отклонений от него.

В результате в общем виде часто получают структуру типа:

$$\mathbb{E}[\text{loss}] = \text{loss of the central predictor} + \text{variation term} + (\text{возможные доп. члены}),$$

где точная форма зависит от выбранной функции потерь и определения "центра".

27.5 Double descent

Double descent — наблюдаемое в современных моделях поведение тестовой ошибки как функции **ёмкости модели** (числа параметров, ширины/глубины, числа итераций обучения и т.п.), которое **расширяет** классическую U-образную кривую.

Схема явления:

1. **Первый спуск (classical regime):** при увеличении ёмкости bias падает, тестовая ошибка уменьшается.
2. **Пик около интерполяционного порога (interpolation threshold):** когда модель *едва способна* идеально подогнать train (train error ≈ 0), тестовая ошибка может **резко вырасти** (модель начинает "подгонять шум" на границе реализуемости).
3. **Второй спуск (overparameterized regime):** при дальнейшем росте ёмкости тестовая ошибка **снова уменьшается**, иногда становясь лучше, чем в "классическом" оптимуме.

Интуитивное объяснение (высокоуровнево):

- в пере-параметризованном режиме у модели появляется **много** решений с нулевой train-ошибкой;
- оптимизация + неявная регуляризация (implicit bias оптимизатора, архитектуры, нормиров early stopping) часто выбирают решения, которые **обобщают лучше**, чем "граничные" интерполирующие модели;
- из-за этого классическая картина "сложнее $\Rightarrow$ хуже" перестаёт быть универсальной.

Практический вывод. Double descent не отменяет необходимости валидации, но объясняет, почему:

- "слишком большие" модели (в терминах параметров) *могут* работать лучше, чем модели на границе интерполяции;
- регуляризация и протокол обучения (данные, аугментации, оптимизатор, ранняя остановка) критичны для поведения вблизи пика.

28.1 Мотивация через bias–variance

Бэггинг (*bootstrap aggregating*) — метод ансамблирования, в котором обучают много базовых моделей на разных бутстрэп-выборках и **усредняют** (регрессия) или **голосуют** (классификаци

Ключевая мотивация: бэггинг в первую очередь снижает **variance** (разброс) модели, что особенно полезно для **нестабильных (high-variance)** алгоритмов, таких как глубокие деревья решений. При этом bias обычно меняется мало (иногда слегка растёт из-за дополнительных ограничений/рандомизации), а основной выигрыш идёт за счёт уменьшения дисперсии предсказ

Почему усреднение снижает variance. Пусть базовые модели (предикторы) $\hat{f}_1(x), \dots, \hat{f}_M(x)$ обучены с некоторой стохастикой (бутстрэп, рандомизация алгоритма и т.п.). Ансамблевый предиктор (регрессия):

$$\hat{f}_{\text{bag}}(x) = \frac{1}{M}\sum_{m=1}^{M} \hat{f}_m(x).$$

Тогда дисперсия ансамбля:

$$\operatorname{Var}\big(\hat{f}_{\text{bag}}(x)\big) = \operatorname{Var}\left(\frac{1}{M}\sum_{m=1}^{M} \hat{f}_m(x)\right) = \frac{1}{M^2}\sum_{m=1}^{M}\sum_{l=1}^{M} \operatorname{Cov}(\hat{f}_m(x), \hat{f}_l(x)).$$

Если предположить, что все базовые модели имеют одинаковую дисперсию $\operatorname{Var}(\hat{f}_m(x)) = \sigma^2(x)$ и одинаковую попарную корреляцию $\rho(x)$:

$$\operatorname{Cov}(\hat{f}_m(x), \hat{f}_l(x)) = \rho(x)\sigma^2(x) \quad (m \neq l),$$

то получаем важную формулу:

$$\boxed{\operatorname{Var}\big(\hat{f}_{\text{bag}}(x)\big) = \frac{\sigma^2(x)}{M}\Big(1 + (M-1)\rho(x)\Big).}$$

Выводы:

- если модели независимы ($\rho \approx 0$), то variance падает примерно как σ^2/M;
- если модели сильно коррелированы (ρ велико), то выигрыш ограничен: даже при $M \to \infty$ остаётся предел $\sigma^2(x)\rho(x)$;
- значит, бэггинг эффективен, когда удаётся сделать базовые модели **слабо коррелированн** поэтому важны бутстрэп и дополнительные источники рандомизации (например, метод случайных подпространств).

Агрегация в классификации. Пусть $h_m(x) \in \{1, \dots, K\}$ — предсказанный класс m-й модели. Тогда бэггинг обычно использует **голосование большинством**:

$$\hat{y}(x) = \arg\max_{c \in \{1,\dots,K\}} \sum_{m=1}^{M} \mathbb{I}[h_m(x) = c].$$

Для вероятностных предсказаний часто усредняют вероятности:

$$\hat{p}_c(x) = \frac{1}{M}\sum_{m=1}^{M} \hat{p}_{m,c}(x), \qquad \hat{y}(x) = \arg\max_c \hat{p}_c(x).$$

28.2 Бутстрэп

Бутстрэп-выборка (*bootstrap sample*) строится из исходного обучающего множества размера n путём **выборки с возвращением** n объектов.

Пусть исходная обучающая выборка:

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n}.$$

Тогда m-я бутстрэп-выборка:

$$\mathcal{D}^{(m)} = \{(x_{i_1}, y_{i_1}), \dots, (x_{i_n}, y_{i_n})\},$$

где индексы $i_1, \dots, i_n$ выбираются независимо и равномерно из $\{1, \dots, n\}$.

Интуиция бутстрэпа для бэггинга:

- бутстрэп создаёт **разные версии** обучающих данных;
- нестабильная модель (например, дерево) даёт **разные** предсказания при небольших измене train;
- усреднение таких моделей снижает variance.

Out-of-Bag (OOB). В бутстрэп-выборке часть объектов из исходного train не попадает вовсе. Вероятность, что фиксированный объект *не* будет выбран ни разу:

$$\mathbb{P}(\text{объект не выбран}) = \left(1 - \frac{1}{n}\right)^n \approx e^{-1} \approx 0.368.$$

То есть примерно 36.8% объектов оказываются **OOB** относительно конкретной модели. Это позволяет оценивать качество ансамбля **без отдельного validation**: для каждого объекта агрегируют предсказания только тех моделей, где этот объект был OOB, и получают OOB-оценку качества, близкую к кросс-валидации по смыслу.

28.3 Метод случайных подпространств

Метод случайных подпространств (*random subspace method*) — способ дополнительно уменьшить корреляцию между базовыми моделями, особенно в задачах с большим числом признаков.

Идея: при обучении каждой базовой модели использовать **случайное подмножество признак** Пусть полный набор признаков имеет размер d. Для модели m выбирают подмножество

$$S_m \subseteq \{1, \dots, d\}, \qquad |S_m| = d',$$

и обучают модель на признаках $\{x_j : j \in S_m\}$.

Зачем это нужно:

- если в данных есть "сильные" признаки, многие модели будут строиться вокруг них и станут коррелированными;
- случайные подпространства заставляют разные модели смотреть на разные комбинации признаков;
- корреляция ρ между базовыми моделями падает $\Rightarrow$ формула для variance ансамбля станови лучше.

В деревьях решений этот подход можно применять:

- **на уровне дерева:** каждое дерево обучается на своём наборе признаков;
- **на уровне узла:** при каждом разбиении рассматривается случайное подмножество призна (это уже Random Forest).

28.4 Random Forest

Random Forest (случайный лес) — это частный случай бэггинга, где базовыми моделями являются деревья решений, и используются **два** источника рандомизации:

1. **Bootstrap по объектам:** каждое дерево обучается на бутстрэп-выборке $\mathcal{D}^{(m)}$.
2. **Random subspace по признакам на каждом узле:** при выборе лучшего разбиения в узле рассматривается не все d признаков, а случайное подмножество размера m:
$$\mathcal{F} \subseteq \{1, \dots, d\}, \qquad |\mathcal{F}| = m,$$
и выбирается лучший split только среди признаков из $\mathcal{F}$.

Агрегация предсказаний.

- Регрессия:

$$\hat{f}_{\text{RF}}(x) = \frac{1}{M} \sum_{m=1}^{M} \hat{f}_m(x).$$

- Классификация (голосование):

$$\hat{y}(x) = \arg\max_{c} \sum_{m=1}^{M} \mathbb{I}[h_m(x) = c].$$

Почему RF обычно лучше одиночного дерева:

- одиночное глубокое дерево имеет низкий bias, но высокий variance;
- усреднение деревьев снижает variance;
- случайные подпространства дополнительно снижают корреляцию деревьев $\Rightarrow$ усиливают эффект усреднения;
- итог: значимое улучшение качества и устойчивости без сложной настройки (часто сильный baseline).

Практическая ремарка про OOB в Random Forest. OOB-оценку качества удобно использо для:

- быстрой оценки качества без отдельного validation;
- подбора некоторых гиперпараметров (например, m / `max_features`, глубины) без явного CV.

29 Задача обнаружения аномалий

29.1 Isolation Forest

Обнаружение аномалий (*anomaly detection*) — задача поиска объектов, которые существенно отличаются от "нормальных" наблюдений. Часто это:

- **unsupervised** постановка: меток нет, оцениваем "насколько объект типичен";
- **semi-supervised** постановка: есть в основном "норма", а аномалии редки/неразмечены (novelty detection).

Isolation Forest (*iForest*) — метод, который ищет аномалии через принцип **изоляции**: аномалы точки обычно проще "отделить" случайными разбиениями пространства признаков, поэтому они имеют **меньшую среднюю длину пути** в деревьях разбиения.

Идея. Строится ансамбль **isolation trees** (iTree). Каждое дерево рекурсивно делит данные **случайными** сплитами:

- случайно выбирается признак j;
- случайно выбирается порог t из диапазона значений этого признака в текущем узле;
- данные делятся по правилу $x_j \le t$ и $x_j > t$.

Рекурсия останавливается, когда:

- в узле остался 1 объект (он изолирован), либо
- достигнута максимальная глубина, либо
- все значения признаков в узле одинаковы (дальше делить нечем).

Длина пути. Для объекта x определяют длину пути $h(x)$ — число рёбер от корня до листа, в который попал x. Если x легко изолируется, то $h(x)$ мал.

Аномальный скор. Пусть построено M деревьев, тогда берут среднюю длину пути:

$$\mathbb{E}[h(x)] = \frac{1}{M}\sum_{m=1}^{M} h_m(x).$$

Далее нормируют на ожидаемую длину пути в случайном бинарном дереве для выборки размера n (часто обозначают $c(n)$):

$$c(n) = 2H_{n-1} - \frac{2(n-1)}{n}, \qquad H_{n-1} = \sum_{k=1}^{n-1}\frac{1}{k} \approx \ln(n-1) + \gamma,$$

где H_{n-1} — гармоническое число, γ — константа Эйлера–Маскерони.
Тогда типичная форма скоринга:

$$s(x;n) = 2^{-\frac{\mathbb{E}[h(x)]}{c(n)}}.$$

Интерпретация:

- если $\mathbb{E}[h(x)]$ маленькое $\Rightarrow$ $s(x)$ ближе к 1 $\Rightarrow$ объект более аномален;
- если $\mathbb{E}[h(x)]$ большое $\Rightarrow$ $s(x)$ ближе к 0 $\Rightarrow$ объект типичный.

Практические свойства Isolation Forest:

- хорошо работает в **unsupervised** режиме и на больших выборках;
- быстрее, чем многие методы плотности, так как использует разбиения и субсэмплирование;
- чувствителен к выбору представления признаков (масштабы/нормировка/категориальные признаки).

Ключевые гиперпараметры:

- число деревьев M (`n_estimators`);
- размер подвыборки для каждого дерева n_{sub} (`max_samples`);
- ожидаемая доля аномалий (`contamination`) — используется для выбора порога по скору;
- максимальная глубина (часто $\lceil \log_2(n_{\text{sub}}) \rceil$) и ограничения листа.

29.2 One-Class SVM

One-Class SVM — метод обнаружения новизны/аномалий, который строит границу, отделяющ "нормальные" объекты от "всего остального" в пространстве признаков (часто в **ядровом** пространстве).

Интуиция. Мы хотим найти поверхность (гиперплоскость в $\mathcal{H}$), которая:

- включает большую часть обучающих точек (норму);
- допускает небольшую долю выбросов;
- имеет максимально возможный зазор (регуляризация через $\|w\|_2$).

Постановка (primal). Пусть есть только "нормальные" примеры $\{x_i\}_{i=1}^n$ и отображение $\varphi(x)$ в признаковое пространство $\mathcal{H}$. Оптимизационная задача one-class SVM:

$$\min_{w,\rho,\xi} \ \frac{1}{2}\|w\|_2^2 + \frac{1}{\nu n}\sum_{i=1}^{n}\xi_i - \rho$$
$$\text{s.t.} \quad \langle w, \varphi(x_i)\rangle \geq \rho - \xi_i, \quad i = 1, \ldots, n, \qquad \xi_i \geq 0,$$

где:

- w задаёт гиперплоскость в $\mathcal{H}$,
- ρ — смещение (порог),
- ξ_i — slack-переменные (разрешают нарушения),
- $\nu \in (0, 1]$ — гиперпараметр, управляющий долей выбросов/опорных векторов.

Смысл параметра ν. В классической интерпретации:

- ν является **верхней оценкой** доли объектов, которые могут оказаться выбросами (ошибка
- ν является **нижней оценкой** доли опорных векторов.

Двойственная задача (dual) и ядровой трюк. В двойственной форме появляются коэффиц α_i и ядро $K(x, z) = \langle \varphi(x), \varphi(z)\rangle$:

$$\max_{\alpha} \ -\frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n}\alpha_i\alpha_j K(x_i, x_j)$$
$$\text{s.t.} \quad 0 \leq \alpha_i \leq \frac{1}{\nu n}, \quad i = 1, \ldots, n, \qquad \sum_{i=1}^{n}\alpha_i = 1.$$

Решающее правило:

$$f(x) = \sum_{i=1}^{n}\alpha_i K(x_i, x) - \rho, \qquad \hat{y}(x) = \text{sign}(f(x)).$$

Обычно интерпретируют так:

$$\hat{y}(x) = +1 \Rightarrow \text{"норма"}, \qquad \hat{y}(x) = -1 \Rightarrow \text{"аномалия/новизна"}.$$

Ядра и гиперпараметры. One-Class SVM часто применяют с RBF-ядром:

$$K(x, z) = \exp\left(-\gamma\|x - z\|_2^2\right), \qquad \gamma > 0.$$

Тогда основные гиперпараметры:

- ν — доля допускаемых выбросов / жёсткость границы;

- γ (или эквивалентно ширина RBF) — локальность границы;
- выбор ядра (linear / polynomial / RBF) и его параметры;
- предобработка: центровка/масштабирование признаков (для RBF обычно критично).

Сравнение с Isolation Forest (высокоуровнево).

- One-Class SVM часто даёт сильную границу на умеренных n, но может быть тяжёлым по времени/памяти на больших датасетах (ядровые методы масштабируются хуже).
- Isolation Forest обычно лучше масштабируется на большие n и проще в эксплуатации как быстрый unsupervised baseline.

30 Градиентный бустинг (GB)

30.1 Процедура построения для MSE

Пусть дана обучающая выборка $\{(x_i, y_i)\}_{i=1}^n$, где $x_i \in \mathbb{R}^d$, $y_i \in \mathbb{R}$. Градиентный бустинг строит **аддитивную модель** (stage-wise additive model):

$$F_M(x) = \sum_{m=0}^{M} \eta_m \, h_m(x),$$

где $h_m(x)$ — базовые модели (обычно неглубокие деревья решений), а η_m — шаги (часто $\eta_m = \eta$ — learning rate).

Функция потерь (MSE). Для регрессии:

$$\mathcal{L}(F) = \sum_{i=1}^{n} \ell\big(y_i, F(x_i)\big), \qquad \ell(y, F) = \frac{1}{2}(y - F)^2.$$

Инициализация. Выбирают константную модель $F_0(x) = c$ как минимум по MSE:

$$F_0(x) = \arg\min_c \sum_{i=1}^{n} \frac{1}{2}(y_i - c)^2 = \bar{y} = \frac{1}{n}\sum_{i=1}^{n} y_i.$$

Итерации $m = 1, \ldots, M$ (градиентный бустинг для MSE).

1. **Псевдо-остатки (negative gradient).** Для MSE производная по предсказанию:
$$\frac{\partial \ell(y_i, F(x_i))}{\partial F(x_i)} = F(x_i) - y_i.$$
Поэтому антиградиент (псевдо-остаток) равен обычному остатку:
$$r_i^{(m)} = -\left.\frac{\partial \ell(y_i, F(x_i))}{\partial F(x_i)}\right|_{F=F_{m-1}} = y_i - F_{m-1}(x_i).$$

2. **Обучение базовой модели на остатках.** Находим h_m как приближение остатков:
$$h_m \approx \arg\min_{h \in \mathcal{H}} \sum_{i=1}^{n} \big(r_i^{(m)} - h(x_i)\big)^2,$$
где $\mathcal{H}$ — класс базовых моделей (например, деревья фиксированной глубины).

3. **Подбор шага (line search).** Ищем скаляр γ_m:
$$\gamma_m = \arg\min_{\gamma} \sum_{i=1}^{n} \frac{1}{2}\Big(y_i - \big(F_{m-1}(x_i) + \gamma \, h_m(x_i)\big)\Big)^2.$$
Для квадратичной формы:
$$\gamma_m = \frac{\sum_{i=1}^n r_i^{(m)} \, h_m(x_i)}{\sum_{i=1}^n h_m(x_i)^2}.$$
На практике при деревьях часто используют **листовые** шаги: если h_m — дерево с листьями $R_{m,\ell}$, то
$$h_m(x) = \sum_{\ell=1}^{L} c_{m,\ell} \, \mathbb{I}[x \in R_{m,\ell}],$$
а оптимальный вклад листа для MSE:
$$c_{m,\ell} = \frac{1}{|R_{m,\ell}|} \sum_{x_i \in R_{m,\ell}} r_i^{(m)} \quad \text{(средний остаток в листе)}.$$

4. **Обновление ансамбля (с shrinkage).**
$$F_m(x) = F_{m-1}(x) + \eta \, \gamma_m \, h_m(x),$$
где $\eta \in (0, 1]$ — learning rate (обычно фиксированный).

Итог. F_M — сумма базовых моделей, каждая из которых уменьшает ошибку предшествующего ансамбля.

30.2 Бустинг как градиентный спуск

Бустинг как градиентный спуск в пространстве функций. Рассмотрим общий риск:

$$\mathcal{L}(F) = \sum_{i=1}^{n} \ell\big(y_i, F(x_i)\big).$$

На шаге m мы хотим сдвинуть F в направлении **антиградиента**:

$$F_m(x) = F_{m-1}(x) + \eta\, h_m(x),$$

где h_m выбирают так, чтобы он приближал антиградиент на обучающих точках:

$$h_m \approx \arg\min_{h \in \mathcal{H}} \sum_{i=1}^{n} \Big(r_i^{(m)} - h(x_i)\Big)^2, \qquad r_i^{(m)} = -\left.\frac{\partial \ell(y_i, F(x_i))}{\partial F(x_i)}\right|_{F=F_{m-1}}.$$

То есть: **градиентный спуск** $\Rightarrow$ отрицательный градиент $\Rightarrow$ аппроксимация его базовой моделью.

30.2.1 вывод для кросс-энтропии

Бинарная классификация. Пусть $y \in \{0, 1\}$, а модель выдаёт скор $F(x) \in \mathbb{R}$ (логит), и вероятность положительного класса

$$p(x) = \sigma(F(x)) = \frac{1}{1 + e^{-F(x)}}.$$

Кросс-энтропия (log-loss):

$$\ell(y, F) = -\Big(y \log p + (1 - y) \log(1 - p)\Big).$$

Найдём производную по F.

Шаг 1: производная по p.

$$\frac{\partial \ell}{\partial p} = -\left(\frac{y}{p} - \frac{1 - y}{1 - p}\right).$$

Шаг 2: производная $p = \sigma(F)$.

$$\frac{dp}{dF} = p(1 - p).$$

Шаг 3: цепное правило.

$$\frac{\partial \ell}{\partial F} = \frac{\partial \ell}{\partial p} \cdot \frac{dp}{dF} = -\left(\frac{y}{p} - \frac{1 - y}{1 - p}\right) p(1 - p).$$

Упростим:

$$-\left(\frac{y}{p} - \frac{1 - y}{1 - p}\right) p(1 - p) = -\big(y(1 - p) - (1 - y)p\big) = p - y.$$

Итого:

$$\boxed{\frac{\partial \ell(y, F)}{\partial F} = p - y.}$$

Следовательно антиградиент (псевдо-остаток) равен:

$$\boxed{r_i^{(m)} = y_i - p_{m-1}(x_i), \qquad p_{m-1}(x_i) = \sigma(F_{m-1}(x_i)).}$$

Интерпретация. На каждом шаге дерево (или другой базовый алгоритм) обучают предсказыва разность между истинной меткой и текущей вероятностью. Это аналог "остатков" для квадратич потери, но теперь остаток зависит от вероятности.

Замечание про шаг в логистическом бустинге. Для log-loss можно:

- либо делать line search по γ_m (как в MSE), минимизируя $\sum_i \ell(y_i, F_{m-1}(x_i) + \gamma h_m(x_i))$;
- либо использовать **второй порядок (Newton step)** в листьях дерева: веса

  $$w_i = p_{m-1}(x_i)\big(1 - p_{m-1}(x_i)\big),$$

  а листовой вклад часто берут пропорционально

  $$c_{m,\ell} \approx \frac{\sum_{x_i \in R_{m,\ell}} (y_i - p_{m-1}(x_i))}{\sum_{x_i \in R_{m,\ell}} p_{m-1}(x_i)(1 - p_{m-1}(x_i))}.$$

(Это объясняет, почему в практических GB-библиотеках используются градиенты и гессианы.)

30.3 Выбор базовой модели

Базовая модель $h_m \in \mathcal{H}$ должна быть "слабым" алгоритмом, который:

- умеет достаточно хорошо приближать (псевдо-)остатки;
- имеет ограниченную сложность, чтобы каждый шаг был небольшим улучшением и не приводил к сильному переобучению.

На практике чаще всего выбирают деревья решений небольшой глубины.

- **Stumps** (глубина 1) — минимальные деревья: дают простой базис, часто хорошо для табличных данных как baseline.
- **Неглубокие деревья** (глубина 3–8) — позволяют моделировать взаимодействия признаков порядка глубины.
- Важно: в бустинге деревья обычно **слабее**, чем в Random Forest (где деревья часто растят глубже), потому что бустинг добавляет модели **последовательно**, корректируя ошибку.

Критерии выбора базового класса $\mathcal{H}$:

- **Скорость обучения/предсказания:** бустинг требует обучить много базовых моделей.
- **Контроль сложности:** глубина дерева, минимальные размеры листьев, регуляризация листовых весов.
- **Согласованность с данными:** деревья хорошо работают на табличных признаках, с нелинейностями и взаимодействиями; линейные базовые модели могут быть уместны, если зависимость близка к линейной, а нужно "дожать" качество.

Практический вывод:

- глубина дерева $\uparrow \Rightarrow$ каждый шаг сильнее (bias $\downarrow$), но возрастает риск overfitting (variance $\uparrow$);
- learning rate $\eta \downarrow \Rightarrow$ требуется больше итераций M, но часто улучшается обобщение.

30.4 Параллелизм

Градиентный бустинг по своей природе **последователен по итерациям**:

$$F_m = F_{m-1} + \eta h_m,$$

поэтому полностью распараллелить шаги $m = 1, \ldots, M$ нельзя (каждый следующий шаг зависит от текущих предсказаний). Однако существенный параллелизм возможен **внутри** каждого шага обучения и на стадии инференса.

Параллелизм при обучении (внутри итерации):

- **Параллельный расчёт градиентов/остатков по объектам:** $r_i^{(m)}$ вычисляются независ для всех i.
- **Параллельный поиск лучшего разбиения в дереве:** для каждого признака (и/или бина, порога) можно независимо считать статистики разбиений. На практике это реализуют через:
  - feature-parallel: распределить признаки по потокам/устройствам;
  - data-parallel: распределить строки, затем агрегировать гистограммы/счётчики.
- **Гистограммные методы:** непрерывные признаки предварительно бинят, а затем считают гистограммы для левого/правого разбиения, что существенно ускоряет обучение и хорошо распараллеливается.
- **Параллельные реализации на GPU:** ускоряют построение гистограмм, редукции и вычисление лучшего сплита.

Параллелизм в инференсе:

- вычисление предсказания $F_M(x)$ есть сумма вкладов деревьев:
$$F_M(x) = F_0(x) + \eta \sum_{m=1}^{M} h_m(x),$$
что легко батчится по объектам и параллелится по данным;
- при большом числе деревьев и запросов эффективность достигается через векторизацию и пакетную обработку.

Итог. GB остаётся **итеративно-последовательным** методом, но современные реализации получают высокую производительность за счёт параллельного построения деревьев (по признака и эффективного батчевого инференса.

31 Библиотеки градиентного бустинга

31.1 XGBoost, LightGBM, CatBoost

Градиентный бустинг над деревьями решений (GBDT) — один из наиболее практичных и сильных подходов для **табличных данных**: даёт высокое качество при относительно недорогом обучении и инференсе; обучение деревьев в ансамбле идёт **последовательно**, а инференс (суммирование вкладов деревьев) можно считать **параллельно**. В продакшене чаще используют специализированные библиотеки (XGBoost/LightGBM/CatBoost), а не базовые реализации "из коробки" в общих ML-библиотеках, потому что ключевую роль играют оптимизации по скорости, памяти, распределённости и поддержке GPU.
Ниже — три основных "промышленных" реализации GBDT, которые решают ключевые задачи: **регрессия, классификация, ранжирование**.

- **XGBoost** (community-driven, активно развивается с 2014 года).

  Идея: максимально оптимизированная и масштабируемая реализация GBDT (включая распределённое обучение), с сильным инженерным фокусом: работа на разреженных данных, приближённые алгоритмы поиска сплитов, эффективное использование памяти/кэ GPU-ускорение.

  - **Постановка обучения:** оптимизация *объективной функции* вида
    $$\mathcal{L}(\theta) = \sum_{i=1}^{n} \ell(y_i, \hat{y}_i) + \Omega(\theta),$$
    где ℓ — функция потерь, Ω — регуляризация модели (регуляризация деревьев/листьев в общем смысле).
  - **Сильные стороны реализации:**
    * **Стабильная и подробная документация**, большой объём учебных материалов.
    * **Широкий охват задач**: регрессия, бинарная и многоклассовая классификация, ранжирование (например, LambdaMART-подобные постановки).
    * **Разреженность и масштабирование**: оптимизации для sparse-данных, распреде обучение, поддержка GPU для обучения и инференса.
  - **Практический комментарий:** часто выбирают как "надёжный baseline" или когда важны документация, переносимость, зрелая экосистема.

- **LightGBM** (изначально внутренний инструмент Microsoft).

  Идея: максимальная эффективность по скорости/памяти на больших табличных данных за счёт инженерных и алгоритмических решений: **гистограммный** алгоритм, **leaf-wise** рост дерева, специальные техники ускорения и уменьшения размерности признакового пространства.

  - **Гистограммный подход:** непрерывные признаки бинятся (bucketization), что резко ускоряет поиск лучших сплитов и уменьшает память.
  - **Leaf-wise рост дерева (best-first):** на каждом шаге расщепляется лист, дающий наибольшее уменьшение функции потерь (в отличие от depth-wise, где расширяют уровень целиком). Это часто быстрее сходится и даёт качество, но требует аккуратной регуляризации/ограничений, чтобы не переобучаться.
  - **Ключевые техники из оригинальной формулировки:**
    * **GOSS** (Gradient-based One-Side Sampling) — ускорение за счёт отбора объектов по величине градиентов.
    * **EFB** (Exclusive Feature Bundling) — объединение "взаимоисключающих" (часто разреженных) признаков для уменьшения эффективной размерности.
  - **Категориальные признаки:** поддерживаются (обычно при целочисленном кодирова с алгоритмами поиска оптимального сплита по категориям.
  - **Практический комментарий:** часто выбирают для очень больших датасетов/жёстки ограничений по времени обучения, при готовности тонко настраивать регуляризацию.

- **CatBoost** (разработан в Яндексе; исторически связан с внутренними инструментами для ранжирования/поиска).

  Идея: “сильные дефолты” и целевая поддержка **категориальных признаков из коробки** а также борьба с специфическими смещениями/утечками в бустинге через **ordered boosting**.

  - **Категориальные признаки из коробки:** библиотека строит численные представлен категорий и их комбинаций, что часто позволяет обойтись без one-hot и сложного препроцессинга.
  - **Ordered boosting / борьба с prediction shift:** в классических схемах бустинга могут возникать смещения, связанные с тем, как используются те же данные для вычисления статистик/градиентов; CatBoost предлагает перестановочные (ordered) техники, которые уменьшают этот эффект.
  - **Инженерные возможности:** поддержка GPU для обучения и инференса; большое число биндингов и удобство деплоя в разных языках.
  - **Практический комментарий:** хороший выбор по умолчанию, особенно когда в данных много категориальных признаков и хочется минимум ручного feature engineering.

Замечание про выбор: в реальных проектах эти библиотеки часто дают сопоставимое качество при грамотном тюнинге, а решающими становятся: (1) тип данных (особенно доля категориальных), (2) требования по скорости/памяти, (3) удобство эксплуатации (инференс, биндинги, воспроизводимость, инфраструктура).

31.2 Плюсы и минусы каждой

Библиотека	Плюсы	Минусы / риски
XGBoost	• Зрелая экосистема и одна из лучших документаций. • Закрывает основные задачи: регрессия/классификация/ранжирование. • Сильные инженерные оптимизации: sparse-aware алгоритмы, приближённые сплиты, распределённость. • GPU-ускорение обучения и предсказания.	• На практике нередко уступает по скорости LightGBM/CatBoost (при равных условиях), особенно на крупных табличных данных. • Для максимальной производительности нужно внимательно подбирать параметры и формат данных.
LightGBM	• Высокая скорость и эффективность по памяти: гистограммы, оптимизации на больших данных. • Leaf-wise рост дерева может быстрее давать хорошее качество. • Алгоритмические ускорения (GOSS, EFB) хорошо работают в индустриальных пайплайнах. • Поддержка категориальных фичей (при корректной подготовке).	• Leaf-wise стратегия повышает риск переобучения без ограничений (например, max_depth/num_leaves/min_da • Категориальные признаки "из коробки" обычно требуют дисциплины в кодировании и контроле утечек. • При маленьких датасетах преимущества по скорости могут быть нерелевантны, а тюнинг становится критичнее.
CatBoost	• Сильные дефолты и удобство: часто "заводится" без сложного препроцессинга. • Лучшие практики для категориальных признаков из коробки, работа с комбинациями категорий. • Ordered boosting снижает риск специфических смещений/утечек при обработке категорий. • GPU train/inference и много биндингов для деплоя.	• На чисто числовых данных может не иметь преимуществ над LightGBM (в зависимости от задачи/тюнинга). • Для предельной скорости на огромных датасетах иногда выигрывает LightGBM (но зависит от данных и железа). • Модель и пайплайн могут становиться тяжелее из-за богатой обработки категорий (важно контролировать память/время).

Рекомендация по выбору (практика):

- Если **много категориальных признаков** и хочется минимум ручного кодирования/сниж риска утечек на target statistics — часто начинают с **CatBoost**.
- Если **очень большие табличные данные** и узкое место — скорость/память при обучении — часто выбирают **LightGBM** (с контролем переобучения).
- Если важны **документация, стабильность, переносимость, широкая поддержка сценариев** — берут **XGBoost** как надёжный baseline, особенно в смешанных инфраструкту

32 Интерпретируемое ML

32.1 Таксономия по свойствам

Интерпретируемость (interpretability) и **объяснимость** (explainability) в ML обычно рассмат как набор свойств модели и/или процедур, которые помогают человеку понять, *почему* модель выдала конкретный ответ, *как* она работает в целом, и *когда* ей можно доверять.

Основные оси таксономии (что именно объясняем):

- **Глобальные объяснения (global):** как модель в целом устроена, какие признаки важны в среднем, какие зависимости и взаимодействия она использует.
- **Локальные объяснения (local):** почему модель дала ответ именно для объекта x (instance-level).

По отношению к модели:

- **Интерпретируемые по конструкции (intrinsic):** сама модель прозрачна (линейные модели, небольшие деревья, скоринговые правила).
- **Пост-хок (post-hoc):** модель может быть "чёрным ящиком", но мы строим объяснения поверх неё (LIME, SHAP, permutation importance, GradCAM и т.п.).

По зависимости от конкретного класса моделей:

- **Model-agnostic:** метод применим к любой модели, доступной как функция $f(x)$ (LIME, KernelSHAP, permutation importance).
- **Model-specific:** использует внутреннюю структуру (TreeSHAP, GradCAM, встроенные важности деревьев).

По форме объяснения:

- **Feature attribution:** вклад признаков в предсказание (коэффициенты, SHAP/LIME-важности).
- **Примеры/прототипы (example-based):** объяснение через близкие примеры (kNN-style) или "типичные" объекты.
- **Контрфакты (counterfactual):** минимальные изменения $x \to x'$ чтобы изменить решение модели.
- **Правила/деревья/суррогаты:** приближение "чёрного ящика" простой моделью (rule lists, surrogate tree).
- **Saliency/карты внимания:** выделение областей входа, важного для решения (Grad-CAM для изображений).

Качество объяснения (что важно на практике):

- **Fidelity (верность):** насколько объяснение соответствует реальному поведению $f(x)$ (особенно критично для пост-хок).
- **Stability/robustness:** устойчивость объяснений к малым изменениям x и к шуму.
- **Sparsity/complexity:** насколько объяснение компактно (человек ограничен по восприятию
- **Actionability:** можно ли из объяснения получить действие (например, какие признаки стоит изменить).

Ремарка: интерпретация $\neq$ причинность. Большинство методов объяснений (в т.ч. SHAP/L объясняют вклад признаков *в прогноз модели*, но не доказывают причинное влияние признаков на целевую переменную.

32.2 Линейные модели, важности в деревьях

Линейные модели. Для регрессии/классификации с линейным скором:

$$f(x) = w^\top x + b.$$

- Если признаки стандартизованы, то **величина** $|w_j|$ отражает "силу" влияния признака x_j на скор.
- **Знак** w_j даёт направление влияния: при увеличении x_j скор растёт (если $w_j > 0$) или падает (если $w_j < 0$).
- Для логистической регрессии вероятность $p(x) = \sigma(w^\top x + b)$, а коэффициенты интерпретир через лог-отношение шансов:

$$\log \frac{p(x)}{1 - p(x)} = w^\top x + b.$$

Ограничения: интерпретация коэффициентов сильно зависит от масштаба признаков, коррелиро (мультиколлинеарность), и наличия взаимодействий/нелинейностей.

Важности признаков в деревьях. В деревьях решений и их ансамблях важность можно оценивать несколькими способами.

(1) Встроенная важность по уменьшению impurity (MDI, Gini/Entropy decrease). Для каждого сплита на признаке j считаем уменьшение неоднородности (impurity decrease):

$$\Delta H = H(Q) - \frac{|Q_L|}{|Q|} H(Q_L) - \frac{|Q_R|}{|Q|} H(Q_R),$$

и суммируем ΔH по всем узлам, где использовался признак j (иногда с нормировкой). Плюс: быстро. Минусы: смещение в сторону признаков с большим числом уникальных значений/возмож сплитов и чувствительность к коррелированным признакам.

(2) Permutation importance (перестановочная важность). Идея: измеряем падение качества, если перемешать значения признака x_j на валидации (разрушив связь признака с целью, но сохранив распределение):

$$\mathrm{Imp}(j) = \mathrm{Metric}(f, X) - \mathrm{Metric}(f, X^{\pi(j)}),$$

где $X^{\pi(j)}$ — данные, где столбец j переставлен случайной перестановкой. Плюсы: model-agnostic, ближе к реальному влиянию на метрику. Минусы: дороговизна, нестабильность при сильной корреляции признаков.

(3) Локальные вклады (SHAP для деревьев): объяснение по объекту x с последующей агрегацией до global-важностей.

32.3 LIME

LIME (*Local Interpretable Model-agnostic Explanations*) строит **локальное** объяснение предсказа "чёрного ящика" $f(x)$, аппроксимируя его **простой интерпретируемой моделью** g в окрестно точки x_0.

Основная идея: локальный суррогат. Хотим найти простую модель g (например, линейную) которая хорошо приближает f около x_0:

$$g^\star = \arg\min_{g \in \mathcal{G}} \; \mathcal{L}(f, g, \pi_{x_0}) + \Omega(g),$$

где:

- $\mathcal{G}$ — класс интерпретируемых моделей (часто линейные модели или короткие правила);
- $\pi_{x_0}(x)$ — **локальная весовая функция** (kernel), которая больше весит точки ближе к x_0;
- $\mathcal{L}$ — мера несоответствия f и g в окрестности x_0 (часто weighted MSE);
- $\Omega(g)$ — штраф за сложность (например, ограничение числа ненулевых коэффициентов).

Практический алгоритм LIME (типовой):

1. Сгенерировать набор возмущённых объектов $\{z_k\}$ около x_0 (перебор/шум/маскирование признаков).
2. Получить ответы "чёрного ящика": $f(z_k)$.
3. Назначить веса близости: $w_k = \pi_{x_0}(z_k)$ (например, экспонента от расстояния).
4. Обучить простую модель g по парам $(z_k, f(z_k))$ с весами w_k и регуляризацией на простоту.

Интерпретация результата. Если g линейная, то коэффициенты g рассматриваются как локальные "важности" признаков для решения в точке x_0.

Ограничения LIME (важно проговорить):

- чувствителен к способу генерации возмущений ("что значит окрестность" в вашем пространс
- локальная верность (fidelity) может быть низкой, если f сильно нелинеен в окрестности;
- объяснение не единственно: разные seed/параметры дают разные ответы (stability-проблема

32.4 Подход SHAP (Shapley additive explanations)

32.4.1 Аксиомы, формула

SHAP основан на **значениях Шепли** из кооперативной теории игр: есть "игра", где признаки — "игроки", а ценность коалиции признаков — это некоторый functional $v(S)$.

Аксиомы (в классической формулировке), которые приводят к значениям Шепли:

- **Efficiency (полнота):** сумма вкладов равна общей ценности:
$$\sum_{j=1}^{d} \phi_j = v(\{1, \dots, d\}) - v(\varnothing).$$
- **Symmetry (симметрия):** если два признака вносят одинаковый вклад во все коалиции, то их объяснения равны.
- **Dummy / Null player (пустой игрок):** если признак не меняет ценность ни в одной коалиции, то его вклад 0.
- **Additivity (аддитивность):** для суммы игр значения Шепли суммируются.

Формула значений Шепли. Для признака j:
$$\boxed{\phi_j(v) = \sum_{S \subseteq N \setminus \{j\}} \frac{|S|!\,(|N| - |S| - 1)!}{|N|!} \Big(v(S \cup \{j\}) - v(S)\Big),}$$
где $N = \{1, \dots, d\}$.

32.4.2 SHAP-значения для ML

Для ML есть предиктор $f : \mathbb{R}^d \to \mathbb{R}$ и конкретный объект x. Нужно определить "ценность коалиции" признаков S.

Типичная (концептуальная) идея: оставляем известными признаки из S, а остальные "маргинализуем" по распределению данных. Одна из стандартных форм:
$$v_x(S) = \mathbb{E}\big[f(X) \mid X_S = x_S\big], \qquad v_x(\varnothing) = \mathbb{E}[f(X)].$$
Тогда SHAP-вклады $\phi_j(x)$ интерпретируются как вклад признака j в отличие предсказания от базового уровня.

Аддитивное объяснение (Shapley additive explanations). SHAP строит разложение:
$$\boxed{f(x) = \phi_0 + \sum_{j=1}^{d} \phi_j(x), \qquad \phi_0 = \mathbb{E}[f(X)].}$$

- $\phi_j(x) > 0$ — признак увеличивает предсказание относительно базового уровня;
- $\phi_j(x) < 0$ — уменьшает.

Почему это удобно:

- локально объясняет конкретный объект (instance-level);
- суммируется в глобальные выводы (например, среднее $|\phi_j|$ по датасету даёт global importance).

Практический нюанс: зависимые признаки. Если признаки коррелированы, условное ожидание $\mathbb{E}[f(X) \mid X_S = x_S]$ зависит от того, как именно моделируется совместное распределени признаков; разные приближения (independent vs conditional) дают разные SHAP-значения.

32.4.3 KernelSHAP, TreeSHAP

KernelSHAP — model-agnostic приближение значений Шепли через взвешенную регрессию на бинарных масках признаков.

Идея KernelSHAP (схематично):

1. Рассматриваем бинарные маски $z \in \{0,1\}^d$, где $z_j = 1$ означает "признак доступен", 0 — "скрыт".
2. Для каждой маски строим "замаскированный" объект и вычисляем отклик f (через маргинализацию/подстановку базовых значений), получая $y(z) \approx v_x(S)$.
3. Обучаем линейную модель по маскам:
$$g(z) = \phi_0 + \sum_{j=1}^{d} \phi_j z_j,$$
минимизируя взвешенную квадратичную ошибку с **ядровыми весами Шепли**:
$$\min_{\phi} \sum_{z} \pi(z)\big(y(z) - g(z)\big)^2, \qquad \pi(z) \propto \frac{(d-1)}{\binom{d}{|z|}\,|z|\,(d-|z|)}.$$

KernelSHAP универсален, но дорог: в общем случае требуется много запросов к f (экспоненциаль природа коалиций).

TreeSHAP — model-specific метод, который вычисляет SHAP-значения **точно и быстро** для деревьев и ансамблей деревьев (Decision Tree, Random Forest, GBDT) за полиномиальное время, используя структуру дерева.

Суть TreeSHAP (интуитивно):

- Для дерева можно аналитически учесть, как признак участвует в прохождении по путям, и как меняется ожидание предсказания при "добавлении" признака в коалицию.
- Вместо перебора всех 2^d коалиций применяется динамическое программирование по путям дерева, поэтому TreeSHAP практически применим для табличных моделей (CatBoost/Light и др.).

32.5 GradCAM

Grad-CAM (*Gradient-weighted Class Activation Mapping*) — метод локального объяснения для CNN, который строит тепловую карту (heatmap) важных областей входного изображения для выбранного класса.

Пусть A^k — k-я карта признаков (feature map) некоторого сверточного слоя, размером $H \times W$, а y^c — скор (logit) класса c до softmax.

Шаг 1: веса важности каналов через градиенты. Считаем градиенты $\frac{\partial y^c}{\partial A_{ij}^k}$ и усредняем по пространственным координатам:

$$\alpha_k^c = \frac{1}{HW} \sum_{i=1}^{H} \sum_{j=1}^{W} \frac{\partial y^c}{\partial A_{ij}^k}.$$

Интуиция: α_k^c показывает, насколько канал k важен для повышения скора класса c.

Шаг 2: линейная комбинация карт признаков и ReLU.

$$L_{\text{GradCAM}}^c(i,j) = \text{ReLU}\left(\sum_k \alpha_k^c A_{ij}^k\right).$$

ReLU оставляет только положительные влияния (то, что поддерживает класс c).

Шаг 3: апсемплинг к размеру входа. Карту L_{GradCAM}^c обычно интерполируют до размера изображения и визуализируют поверх исходного входа.

Сильные и слабые стороны GradCAM:

- **Плюсы:** работает для широкого класса CNN, не требует изменения архитектуры, даёт наглядные карты "куда смотрит" модель.
- **Минусы:** это объяснение на уровне активаций слоя (не причинность), зависит от выбранно слоя, может быть грубым по пространственному разрешению.

33 Оптимизация гиперпараметров (HP)

33.1 Классы задач оптимизации

Оптимизация гиперпараметров (Hyperparameter Optimization, HPO) формулируется как задача:

$$\lambda^\star \in \arg\min_{\lambda \in \Lambda} \mathcal{J}(\lambda),$$

где λ — набор гиперпараметров (например, `max_depth`, `learning_rate`, `C`, `gamma`), Λ — пространс поиска, а $\mathcal{J}(\lambda)$ — целевая функция (обычно качество на валидации или кросс-валидации), например:

$$\mathcal{J}(\lambda) = \frac{1}{K}\sum_{k=1}^{K} \mathrm{Loss}\Big(\mathcal{A}(D^{(k)}_{\text{train}}; \lambda),\ D^{(k)}_{\text{val}}\Big),$$

где $\mathcal{A}$ — алгоритм обучения модели.

В терминах теории оптимизации HPO почти всегда относится к **0-order (black-box)** оптимизац

- мы можем вычислить значение $\mathcal{J}(\lambda)$, запустив обучение и оценку;
- градиенты $\nabla_\lambda \mathcal{J}(\lambda)$ обычно недоступны или слишком дороги/нестабильны.

Типовые классы задач оптимизации, важные для HPO:

1. **По типу пространства поиска Λ:**
   - **непрерывные** (например, $\lambda \in \mathbb{R}$: `learning_rate`);
   - **дискретные/целочисленные** (например, число деревьев, глубина, `k` в kNN);
   - **категориальные** (выбор лосса, ядра, стратегии сплита);
   - **смешанные** (mixed-integer, mixed categorical+continuous) — самый частый случай;
   - **деревья решений по структуре пространства** (tree-structured / conditional): некоторые гиперпараметры активны только при определённых значениях других (например, `kernel` в SVM $\Rightarrow$ появляются `gamma`, `degree`).
2. **По свойствам целевой функции $\mathcal{J}(\lambda)$:**
   - **не выпуклая** и **мультимодальная** (много локальных минимумов);
   - **шумная** (noisy): качество зависит от случайности обучения/инициализации/сплита;
   - **дорогая** (expensive): один вызов $\mathcal{J}(\lambda)$ = полное обучение + валидация;
   - **недифференцируемая** (из-за дискретных параметров, ранней остановки, ветвлений)
3. **По наличию ограничений:**
   - **без ограничений** (unconstrained) — только границы диапазонов;
   - **ограниченная** (constrained) — например, ограничения по памяти/latency, монотоннос число параметров, требования к интерпретируемости.
4. **По числу целей:**
   - **одноцелевые** (single-objective): оптимизируем одну метрику/лосс;
   - **многокритериальные** (multi-objective): качество vs скорость/память/стабильность (ищут множество Парето-оптимальных решений).
5. **Multi-fidelity / ресурсная оптимизация (практически очень важный класс).** Вместо оценки $\mathcal{J}(\lambda)$ "до конца" используют приближённые оценки на меньшем ресурсе: меньше эпох, меньший сабсэмпл данных, меньшая модель, более грубая валидация.

33.2 Основные подходы к оптимизации HP

Так как HPO — это 0-order / black-box оптимизация, базовые подходы строятся вокруг стратегии **выбора следующего** набора гиперпараметров λ при ограниченном бюджете вычислений.

1) Ручной подбор (manual trials).

- Инженер задаёт несколько конфигураций на основе опыта/эвристик и итеративно улучшает
- Плюс: быстро стартовать, полезно как sanity-check.
- Минус: плохо масштабируется, высок риск упустить хорошие области, сложно воспроизводи

2) Grid search (полный перебор по сетке).

- Задаётся конечная сетка значений для каждого гиперпараметра и перебираются все комбин
- Плюсы: простота, воспроизводимость, легко параллелится.
- Минусы: **проклятие размерности** (число комбинаций растёт экспоненциально), плохо использует бюджет в высоких размерностях и при неравной важности параметров.

3) Random search (случайный поиск).

- Семплируем $\lambda \sim p(\lambda)$ (равномерно/лог-равномерно/по заданным распределениям) и оцени $\mathcal{J}(\lambda)$.
- Плюсы: лучше использует бюджет, особенно когда важны лишь некоторые гиперпараметры легко распараллеливается (embarrassingly parallel).
- Минусы: не использует информацию о предыдущих запусках, кроме "лучшего найденного".

4) Байесовская оптимизация (Bayesian / SMBO family). Идея: строим **суррогатную модель** $\widehat{\mathcal{J}}(\lambda)$ по уже измеренным точкам и выбираем новую λ через **функцию приобретения** (acquisition), балансируя exploration/exploitation.

- Общая схема:
  1. Имеем историю $\{(\lambda_t, \mathcal{J}(\lambda_t))\}_{t=1}^{T}$.
  2. Обучаем surrogate $\widehat{\mathcal{J}}(\lambda)$ (например, GP/RandomForest/TPE).
  3. Выбираем следующую точку:
  $$\lambda_{T+1} \in \arg\max_{\lambda \in \Lambda} a(\lambda; \widehat{\mathcal{J}}),$$
  где $a(\cdot)$ — acquisition (например, Expected Improvement, UCB и т.п.).
  4. Вычисляем реальную $\mathcal{J}(\lambda_{T+1})$, добавляем в историю, повторяем.
- **TPE (Tree-structured Parzen Estimator).** Вместо прямой аппроксимации $\mathcal{J}(\lambda)$ оценива распределения гиперпараметров в "хорошей" и "плохой" областях:
$$p(\lambda \mid \mathcal{J}(\lambda) \text{ мал}) \quad \text{и} \quad p(\lambda \mid \mathcal{J}(\lambda) \text{ велик}),$$
и выбирают λ, которые с большей вероятностью попадают в "хорошую" область. TPE удобен для **смешанных и условных** (tree-structured) пространств гиперпараметров.

5) Эволюционные методы (Evolutionary methods).

- Поддерживается популяция конфигураций $\{\lambda\}$; применяются мутации/кроссовер/отбор по качеству.
- Плюсы: естественно работают со смешанными и дискретными параметрами, легко параллел
- Минусы: часто требуют большого бюджета; качество сильно зависит от настроек операторов и популяции.

6) Multi-fidelity и ранняя остановка (ускорение HPO). Поскольку один прогон модели дорог, широко применяют подходы, которые **отсеивают плохие конфигурации рано**:

- Successive Halving / Hyperband: выделяют ресурс (эпохи/данные) многим конфигурациям, затем оставляют лучших и увеличивают ресурс оставшимся.
- Практический смысл: за тот же бюджет можно проверить больше конфигураций, сохранив шанс найти хорошие.

Комментарий про параллелизм HPO.

- Grid/Random: идеально параллельны.
- Байесовская оптимизация: по природе более **последовательна**, но возможны батчи и асинхронные схемы (несколько предложений λ одновременно).
- Эволюционные методы: естественная параллельность по оценке популяции.

33.3 Библиотеки для оптимизации HP

Популярные библиотеки HPO (и что обычно используют):

- **Optuna**: современная библиотека HPO; поддерживает разные samplers (в т.ч. TPE), pruners (ранняя остановка неудачных трейлов), удобный API для экспериментов и логирова
- **Hyperopt**: одна из классических библиотек, также известна использованием TPE и tree-structured пространств.

Дополнительно (часто встречается на практике):

- **scikit-learn**: `GridSearchCV`, `RandomizedSearchCV` (удобно для baseline и CV).
- **Ray Tune**: фреймворк для масштабируемого HPO и распределённых экспериментов (планировщики, early stopping).
- **SMAC**: SMBO-подходы (часто на базе деревьев/лесов как surrogate), удобны для смешанны параметров.
- **Nevergrad**: эволюционные/градиент-свободные методы в едином интерфейсе.

Практический выбор.

- Для быстрых экспериментов и сильных дефолтов: Optuna.
- Для базового CV-подбора в sklearn-пайплайнах: GridSearchCV/RandomizedSearchCV.
- Для больших распределённых прогонов с scheduler-ами и ранней остановкой: Ray Tune и аналоги.

34 Матричное исчисление и производные матричных операций

34.1 Производная скалярного произведения

34.1.1 Для двух векторов

Далее считаем, что все векторы — **столбцы** ($x, y \in \mathbb{R}^d$), а градиент по вектору определён как

$$\nabla_x f(x) \;=\; \begin{pmatrix} \frac{\partial f}{\partial x_1} & \cdots & \frac{\partial f}{\partial x_d} \end{pmatrix}^\top \in \mathbb{R}^d.$$

Пусть

$$f(x, y) = x^\top y = \sum_{i=1}^{d} x_i y_i.$$

Тогда по компонентам:

$$\frac{\partial f}{\partial x_i} = y_i, \qquad \frac{\partial f}{\partial y_i} = x_i,$$

и, следовательно,

$$\boxed{\nabla_x (x^\top y) = y, \qquad \nabla_y (x^\top y) = x.}$$

Короткий вывод через дифференциалы (удобно для матричного исчисления).

$$df = d(x^\top y) = (dx)^\top y + x^\top (dy).$$

Сопоставляя с формой $df = (\nabla_x f)^\top dx + (\nabla_y f)^\top dy$, получаем:

$$(\nabla_x f)^\top dx = (dx)^\top y \Rightarrow \nabla_x f = y, \qquad (\nabla_y f)^\top dy = x^\top dy \Rightarrow \nabla_y f = x.$$

34.1.2 Для матрицы и вектора

На практике скалярное произведение часто возникает как произведение вектора на линейное преобразование. Рассмотрим матрицу $A \in \mathbb{R}^{m \times d}$ и векторы $x \in \mathbb{R}^d$, $b \in \mathbb{R}^m$. Определим скаляр:

$$f(x) = \langle Ax,\, b \rangle = (Ax)^\top b = b^\top Ax.$$

Это скалярное произведение двух векторов Ax и b.

Производная по x (матрица фиксирована). Перепишем:

$$f(x) = b^\top Ax = (A^\top b)^\top x.$$

Тогда по правилу из предыдущего пункта:

$$\boxed{\nabla_x \left(b^\top Ax\right) = A^\top b.}$$

Альтернативный вывод через дифференциалы.

$$df = d(b^\top Ax) = b^\top A\, dx = (A^\top b)^\top dx \;\;\Rightarrow\;\; \nabla_x f = A^\top b.$$

Полезное обобщение (часто используется). Если

$$f(x, y) = y^\top Ax,$$

то

$$\boxed{\nabla_x (y^\top Ax) = A^\top y, \qquad \nabla_y (y^\top Ax) = Ax.}$$

(Это прямое следствие формы $df = y^\top A\, dx + (Ax)^\top dy$.)

35 Концепция нейросети

35.1 Мотивация нейросетей

Нейросети (нейронные сети, NN) — класс моделей, которые строят **композицию** простых преобразований, обучаемых по данным, чтобы аппроксимировать сложные зависимости между признаками и целевой переменной.

Ключевая мотивация (интуиция из лекций):

- **Проблема линейных моделей: нелинейные зависимости.** Логистическая регрессия и другие линейные модели хорошо работают, когда целевая зависимость близка к линейной в исходных признаках. Но при наличии существенной нелинейности линейные модели требуют **feature engineering** (создание нелинейных признаков), чтобы разделить классы/с зависимость.
- **Feature engineering — "искусство" и узкое место.** В классическом пайплайне качество часто определяется тем, насколько экспертно построены признаки. Это дорого, долго, слабо переносится между задачами и плохо масштабируется.
- **Нейросети автоматизируют извлечение признаков.** В NN признаки (представления) извлекаются **автоматически из данных** в ходе обучения: ранние слои учат более простые преобразования, более глубокие — более абстрактные. Это позволяет уменьшить зависимость от ручного конструирования признаков, если есть достаточный объём данных и вычислений.
- **End-to-end обучение.** Нейросеть оптимизируется "от входа до выхода" по одной функции потерь, обучая и представление, и предсказатель одновременно:

$$x \xrightarrow{\text{NN}} \hat{y}, \qquad \theta^\star \in \arg\min_\theta \sum_{i=1}^{n} \ell\big(y_i, f(x_i;\theta)\big).$$

Практический смысл мотивации: нейросети особенно полезны там, где (1) зависимость сильно нелинейна, (2) вручную трудно описать релевантные признаки, (3) есть много данных и/или возможно применить перенос обучения (pretraining → finetuning).

35.2 Задача XOR

XOR (исключающее ИЛИ) для бинарных признаков $x_1, x_2 \in \{0,1\}$:

$$\text{XOR}(x_1,x_2) = \begin{cases} 1, & x_1 \neq x_2, \\ 0, & x_1 = x_2. \end{cases}$$

Таблица истинности:

x_1	x_2	$\text{XOR}(x_1,x_2)$
0	0	0
0	1	1
1	0	1
1	1	0

Почему 1-слойный перцептрон (линейный классификатор) не решает XOR. Линейны классификатор имеет вид

$$\hat{y} = \mathbb{I}[w^\top x + b \geq 0], \qquad x = (x_1, x_2)^\top.$$

Но множество точек класса 1 для XOR: $(0,1)$ и $(1,0)$, а класса 0: $(0,0)$ и $(1,1)$. Эти два класса **линейно неразделимы** в $\mathbb{R}^2$: не существует прямой, которая разделит их без ошибок. Следовательно, один линейный слой (даже с сигмоидой) на исходных признаках x_1, x_2 не может реализовать XOR.

Как решает XOR 2-слойная нейросеть. Добавим скрытый слой, который создаёт **нелинейн признаки** (промежуточные представления). Конструктивная логическая форма:

$$\text{XOR}(x_1,x_2) = (x_1 \vee x_2) \ \wedge \ \neg(x_1 \wedge x_2).$$

То есть можно построить скрытые нейроны, приближающие операции OR и AND (например, через сигмоиды от линейных комбинаций), а выходной нейрон комбинирует их.

Замечание про feature engineering. Даже 1-слойная модель может решить XOR, если вручную добавить признак взаимодействия:

$$x_3 = x_1 x_2,$$

тогда XOR становится линейно разделимым в пространстве (x_1, x_2, x_3). Этот пример показывает мотивацию NN: скрытый слой способен **сам** построить подобные взаимодействия, уменьшая ручную работу.

35.3 Определения слоёв

35.3.1 Графическая нотация

Нейрон (unit). Базовый вычислительный элемент (в простейшем виде):

$$z = w^\top x + b, \qquad a = \sigma(z),$$

где x — вход, w, b — параметры, σ — нелинейность (activation).

Слой (layer) как преобразование. В современном ML под слоем чаще всего понимают преобразование вида

$$x \mapsto h(x),$$

которое может быть:

- **обучаемым (trainable transform):** имеет параметры, оптимизируемые по данным (Linear/Dense, Convolution, Self-Attention);
- **необучаемым:** фиксированное преобразование (Activation, Dropout, BatchNorm в режиме инференса и т.п.).

Линейный (полносвязный) слой (Linear / Dense / Fully Connected). Если вход $x \in \mathbb{R}^{d_{\text{in}}}$ и выход $h \in \mathbb{R}^{d_{\text{out}}}$:

$$h = Wx + b, \qquad W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}, \quad b \in \mathbb{R}^{d_{\text{out}}}.$$

Полносвязный означает, что каждый выходной компонент зависит от всех входных.

Активационный слой (Activation). Применяется поэлементно:

$$h = \sigma(z),$$

вводит нелинейность и делает возможной аппроксимацию сложных зависимостей.

Три распространённые нотации “сколько слоёв в сети” (важно не путаться):

- **Нотация 1 (академическая):** считают только **обучаемые преобразования** (trainable layers). В этом смысле говорят про “LeNet-5” как про сеть с 5 обучаемыми слоями и т.п.
- **Нотация 2 (инженерная, кодовая):** считают **каждый модуль/преобразование** как слой (Linear, Activation, Dropout, BatchNorm и др.), как в контейнере `Sequential`.
- **Нотация 3 (по представлениям):** слоями называют **векторы признаков между преобразованиями**: **Input** (входной слой), **Hidden** (скрытые), **Output** (выходной).

Графическая нотация (интуитивная). Нейросеть удобно представлять ориентированным графом вычислений:

- вершины — скаляры/векторы активаций (нейроны или группы нейронов);
- рёбра — умножения на веса и суммирование (линейные преобразования);
- активации часто “подразумеваются” (на рисунке могут не рисоваться как отдельные блоки).

В типовой схеме MLP слой часто изображают как **набор стрелок** (весов), соединяющих два набора нейронов (входные активации $\rightarrow$ выходные активации).

35.4 Определение эмбеддинга

Эмбеддинг (*embedding*) в ML — это **векторное представление объекта** в вещественном линейном пространстве:

$$\text{emb}: \ \mathcal{X} \to \mathbb{R}^d,$$

где $\mathcal{X}$ — множество объектов (слова, документы, изображения, пользователи, товары, события), а d — размерность эмбеддинга.

Интерпретация:

- эмбеддинг кодирует "смысл"/свойства объекта так, что в $\mathbb{R}^d$ можно измерять близость (например, косинусная близость, L_2-расстояние) и решать прикладные задачи (поиск, кластеризацию, рекомендации);
- часто эмбеддинги — это выход **предпоследнего слоя** нейросети:
$$x \xrightarrow{\text{backbone}} e(x) \in \mathbb{R}^d \xrightarrow{\text{head}} \hat{y},$$
где **backbone** извлекает представление (embedding), а **head** решает конкретную задачу (классификация/регрессия).

Примеры:

- word embeddings: каждому слову сопоставляется вектор $e(w) \in \mathbb{R}^d$;
- image embeddings: изображение $\to$ вектор признаков для поиска похожих;
- user/item embeddings в рекомендациях: пользователи и товары в одном пространстве для ранжирования/матчинга.

36 Линейный слой (Linear layer)

36.1 Все названия линейного слоя

Линейный слой в нейросетях встречается под несколькими (почти эквивалентными) названиями

- **Linear layer** (линейный слой) — наиболее частое название в PyTorch (`torch.nn.Linear`).
- **Dense layer** (плотный слой) — распространено в Keras/TensorFlow (`Dense`).
- **Fully Connected layer (FC)** (полносвязный слой) — подчёркивает, что каждый выход зависит от всех входов.
- **Affine layer** (аффинный слой) — более точное математическое название, т.к. обычно есть смещение b (строго линейным является Wx, а $Wx + b$ — аффинное отображение).
- **Perceptron layer** (слой перцептронов) — историческое/интуитивное название (нейроны = линейная комбинация + нелинейность).
- **Matrix multiplication + bias** (умножение на матрицу + bias) — инженерное описание операции.

Важно: в большинстве фреймворков под "линейным слоем" подразумевают именно **аффинное** преобразование $Wx + b$.

36.2 Формула

Пусть входной вектор $x \in \mathbb{R}^{d_{\text{in}}}$, выход $y \in \mathbb{R}^{d_{\text{out}}}$. Линейный (аффинный) слой задаётся:

$$y = Wx + b, \qquad W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}, \;\; b \in \mathbb{R}^{d_{\text{out}}}.$$

Для батча $X \in \mathbb{R}^{n \times d_{\text{in}}}$ (каждая строка — объект) часто пишут:

$$Y = XW^\top + \mathbf{1}b^\top, \qquad Y \in \mathbb{R}^{n \times d_{\text{out}}},$$

где $\mathbf{1} \in \mathbb{R}^n$ — вектор из единиц (технически, это broadcast смещения).

36.3 Гиперпараметры, число параметров

Гиперпараметры линейного слоя (задаются до обучения):

- **Размер входа** d_{in} (`in_features`).
- **Размер выхода** d_{out} (`out_features`).
- **Наличие смещения** (`bias=True/False`).

Число обучаемых параметров:

- матрица весов W: $d_{\text{out}} \cdot d_{\text{in}}$ параметров;
- смещение b: d_{out} параметров (если bias включён).

Итого:

$$\boxed{\#\text{params} = d_{\text{out}}\, d_{\text{in}} + \mathbb{I}[\text{bias}] \cdot d_{\text{out}}.}$$

Пример. Если $d_{\text{in}} = 128$, $d_{\text{out}} = 64$, bias включён, то

$$\#\text{params} = 64 \cdot 128 + 64 = 8192 + 64 = 8256.$$

36.4 Инициализация

Зачем инициализировать аккуратно. Если веса слишком большие, активации и градиенты могут **взрываться**; если слишком маленькие — могут **затухать** (особенно в глубоких сетях). Кроме того, **нельзя** инициализировать все веса одинаково (например, нулями), иначе нейроны слоя будут симметричны и будут обучаться одинаково (не произойдёт "разрыва симметрии").

Общая идея современной инициализации: выбрать случайные веса с дисперсией, зависящей от размеров слоя, чтобы сохранять масштабы сигналов при прохождении вперёд и назад.

Обозначим:

$$\text{fan_in} = d_{\text{in}}, \qquad \text{fan_out} = d_{\text{out}}.$$

Xavier/Glorot initialization (часто для tanh / sigmoid и близких режимов):

$$W_{ij} \sim \mathcal{N}\left(0,\ \frac{2}{\text{fan_in} + \text{fan_out}}\right) \quad \text{или} \quad W_{ij} \sim \mathcal{U}\left[-\sqrt{\frac{6}{\text{fan_in} + \text{fan_out}}},\ \sqrt{\frac{6}{\text{fan_in} + \text{fan_out}}}\right]$$

He/Kaiming initialization (часто для ReLU/LeakyReLU):

$$W_{ij} \sim \mathcal{N}\left(0,\ \frac{2}{\text{fan_in}}\right) \quad \text{или} \quad W_{ij} \sim \mathcal{U}\left[-\sqrt{\frac{6}{\text{fan_in}}},\ \sqrt{\frac{6}{\text{fan_in}}}\right].$$

Интуиция: ReLU обнуляет примерно половину активаций, поэтому дисперсию весов увеличивают относительно Xavier.

Смещения (bias). Часто инициализируют нулями:

$$b = 0,$$

поскольку симметрию нарушает именно случайность весов W, а bias обычно не требует случайной инициализации.

Практическая ремарка. Во многих фреймворках (PyTorch, TF) `Linear/Dense` уже имеет адекватную инициализацию по умолчанию; при построении глубоких сетей важно согласовывать выбор инициализации с активацией.

36.5 Теорема универсальной аппроксимации

Теорема универсальной аппроксимации (в контексте нейросетей) утверждает, что **достаточно широкая** полносвязная сеть с **одним скрытым слоем** и подходящей нелинейностью способна аппроксимировать любую непрерывную функцию на компакте.

Одна из стандартных формулировок (схематично): пусть $K \subset \mathbb{R}^d$ — компакт, $f : K \to \mathbb{R}$ — непрерывная функция, $\sigma(\cdot)$ — нелинейная активация (например, sigmoid, tanh, ReLU; важно, чтобы она была не "вырожденной"). Тогда для любого $\varepsilon > 0$ существует число нейронов m и параметры $\{a_i\}_{i=1}^m$, $\{w_i\}_{i=1}^m$, $\{b_i\}_{i=1}^m$ такие, что функция

$$F(x) = \sum_{i=1}^{m} a_i\, \sigma(w_i^\top x + b_i)$$

удовлетворяет

$$\boxed{\sup_{x \in K} |f(x) - F(x)| < \varepsilon.}$$

Как это связано с линейным слоем. Полносвязная сеть строится как композиция линейных (аффинных) преобразований и нелинейностей:

$$x \xrightarrow{W_1 x + b_1} z \xrightarrow{\sigma(\cdot)} h \xrightarrow{W_2 h + b_2} \hat{y}.$$

Один линейный слой без нелинейности — это лишь аффинное отображение и оно **не** универсально. Универсальность появляется именно из-за **композиции** линейных слоёв с **нелинейными** активациями.

Важные оговорки (что теорема не обещает):

- Теорема говорит о **существовании** параметров, но не гарантирует, что оптимизация (SGD/Adam) их найдёт быстро.
- Не гарантирует **эффективности**: для некоторых функций ширина m может потребоваться очень большая.

- В практике глубина (несколько слоёв) часто даёт **более параметрически эффективные** представления, чем один очень широкий слой.

37 Функции активации

37.1 Примеры (минимум 4)

Функция активации (activation) — **поэлементная нелинейность** $\sigma : \mathbb{R} \to \mathbb{R}$, которая применяется к предактивации z и позволяет нейросети моделировать **нелинейные** зависимости

$$a = \sigma(z).$$

Если убрать все нелинейности, композиция линейных слоёв останется линейной, и сеть не сможет выражать сложные функции.

Ниже приведены распространённые активации (больше чем 4, чтобы было из чего выбирать на экзамене).

1. **Sigmoid**:
$$\sigma(z) = \frac{1}{1+e^{-z}}, \qquad \sigma'(z) = \sigma(z)\big(1-\sigma(z)\big).$$
Диапазон: $(0, 1)$.

2. tanh:
$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}, \qquad \frac{d}{dz}\tanh(z) = 1 - \tanh^2(z).$$
Диапазон: $(-1, 1)$.

3. **ReLU** (*Rectified Linear Unit*):
$$\mathrm{ReLU}(z) = \max(0, z), \qquad \mathrm{ReLU}'(z) = \begin{cases} 1, & z > 0, \\ 0, & z < 0, \end{cases}$$
(в $z = 0$ производная не определена, на практике берут субградиент).

4. **Leaky ReLU**:
$$\mathrm{LeakyReLU}(z) = \max(\alpha z, z), \qquad \alpha \in (0, 1),$$
$$\mathrm{LeakyReLU}'(z) = \begin{cases} 1, & z > 0, \\ \alpha, & z < 0. \end{cases}$$

5. **ELU** (*Exponential Linear Unit*):
$$\mathrm{ELU}(z) = \begin{cases} z, & z > 0, \\ \alpha(e^z - 1), & z \le 0, \end{cases} \qquad \alpha > 0.$$
(и производная определяется по частям).

6. **GELU** (*Gaussian Error Linear Unit*, популярна в трансформерах):
$$\mathrm{GELU}(z) = z\,\Phi(z),$$
где Φ — CDF стандартного нормального распределения. На практике часто используют приближение:
$$\mathrm{GELU}(z) \approx 0.5z\left(1 + \tanh\left(\sqrt{\frac{2}{\pi}}\left(z + 0.044715z^3\right)\right)\right).$$

7. **Softplus** (гладкая версия ReLU):
$$\mathrm{Softplus}(z) = \log(1 + e^z), \qquad \frac{d}{dz}\mathrm{Softplus}(z) = \sigma(z).$$

8. **Softmax** (для многоклассовой классификации, векторная активация на выходе):
$$\mathrm{softmax}(z)_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad k = 1, \dots, K,$$
где $z \in \mathbb{R}^K$. Даёт вероятностное распределение по классам.

37.2 Основные свойства (сравнительный анализ)

Ниже — основные свойства, по которым сравнивают активации, и краткий анализ наиболее популярных вариантов.

Ключевые свойства активаций:

- **Нелинейность:** необходима для аппроксимации сложных функций.
- **Диапазон выходов (range):** ограниченный/неограниченный.
- **Насыщение (saturation):** есть ли области, где производная близка к нулю (приводит к затухающим градиентам).
- **Zero-centered:** является ли выход центрированным вокруг нуля (важно для динамики оптимизации).
- **Гладкость/дифференцируемость:** нужна для корректной работы градиентных методов (в практике допускаются субградиенты, как у ReLU).
- **Стабильность градиента:** склонность к vanishing/exploding gradients.
- **Cparsity:** даёт ли активация много нулей (ReLU даёт разреженность, что может помогать).
- **Вычислительная стоимость:** насколько дёшево считать $\sigma(z)$ и $\sigma'(z)$.

Сравнение основных активаций (смысловые выводы).

- **Sigmoid.**
  - **Плюсы:** удобна на выходе для вероятности в бинарной классификации; гладкая.
  - **Минусы:** насыщается при больших $|z| \Rightarrow \sigma'(z) \approx 0 \Rightarrow$ **затухание градиентов**. Выход не zero-centered ($\in (0,1)$), что ухудшает оптимизацию в скрытых слоях.
  - **Типовое использование:** выход бинарного классификатора, иногда гейты в RNN (LSTM/GRU).
- tanh**.**
  - **Плюсы:** zero-centered (диапазон $(-1,1)$), часто лучше sigmoid в скрытых слоях.
  - **Минусы:** тоже насыщается $\Rightarrow$ vanishing gradients при больших $|z|$.
  - **Типовое использование:** скрытые состояния в RNN, иногда MLP на умеренной глубине.
- **ReLU.**
  - **Плюсы:** нет насыщения на положительной полуоси, производная $= 1$ при $z > 0 \Rightarrow$ хорошо проходит градиент; дешёвая; даёт разреженность (много нулей).
  - **Минусы:** на отрицательной полуоси производная $0 \Rightarrow$ риск **"dead ReLU"** (нейрон перестал активироваться и не обучается). Выход не zero-centered и неограничен сверху (иногда требует нормировок).
  - **Типовое использование:** базовая активация для CNN/MLP, особенно с He/Kaiming init и нормировками.
- **Leaky ReLU / ELU.**
  - **Плюсы:** уменьшают проблему dead ReLU, т.к. на отрицательной стороне есть ненулева производная. ELU дополнительно даёт более плавное поведение и может давать более "центрированный" выход.
  - **Минусы:** LeakyReLU добавляет гиперпараметр α; ELU дороже вычислительно (экспон
  - **Типовое использование:** когда ReLU даёт много "мертвых" нейронов или нужна устойчивость.
- **GELU.**
  - **Плюсы:** гладкая, ведёт себя как "мягкая" ReLU с вероятностным gating, хорошо работает в трансформерах и глубоких моделях.
  - **Минусы:** дороже вычислительно, чем ReLU (хотя есть быстрые аппроксимации).

  - **Типовое использование:** трансформеры (NLP/vision), современные большие сети.

- **Softplus.**
  - **Плюсы:** гладкая аппроксимация ReLU, всегда дифференцируема.
  - **Минусы:** дороже, чем ReLU; менее разреженная; может давать меньшую "жёсткость".
  - **Типовое использование:** когда важна гладкость (например, некоторые вероятностны модели).

- **Softmax (выход).**
  - **Смысл:** переводит логиты $z \in \mathbb{R}^K$ в распределение вероятностей по классам.
  - **Где применяют:** выход многоклассового классификатора совместно с кросс-энтропией
  - **Нюанс:** вычислять стабильно через вычитание $\max(z)$:

$$\mathrm{softmax}(z)_k = \frac{e^{z_k - \max(z)}}{\sum_j e^{z_j - \max(z)}}.$$

Практические правила выбора (связь со свойствами):

- Скрытые слои в MLP/CNN: чаще всего ReLU/LeakyReLU/ELU; в трансформерах — GELU.
- Выход бинарной классификации: sigmoid (или logits + `BCEWithLogitsLoss`).
- Выход многоклассовой: softmax + cross-entropy (или logits + `CrossEntropyLoss`).
- Если наблюдается vanishing gradients: избегать насыщаемых активаций в глубоких сетях, использовать ReLU-подобные + нормировки.

38 Обратное распространение ошибки (backpropagation)

38.1 Правило цепочки

Backpropagation (обратное распространение ошибки) — это эффективный способ вычислять градиенты функции потерь по параметрам модели, используя **правило цепочки** на вычислительном графе.

Правило цепочки (скалярный случай). Если $u = g(x)$ и $L = f(u)$, то

$$\frac{dL}{dx} = \frac{dL}{du} \cdot \frac{du}{dx}.$$

Если композиция длиннее, $x \to u \to v \to L$, то

$$\frac{dL}{dx} = \frac{dL}{dv} \cdot \frac{dv}{du} \cdot \frac{du}{dx}.$$

Векторный случай (через якобианы). Пусть $u = g(x)$, где $x \in \mathbb{R}^d$, $u \in \mathbb{R}^m$, и $L = f(u) \in \mathbb{R}$. Тогда

$$\nabla_x L = J_g(x)^\top \nabla_u L,$$

где $J_g(x) \in \mathbb{R}^{m \times d}$ — якобиан g.

Основная идея backprop на вычислительном графе. Пусть в графе есть промежуточные величины $v_1, \ldots, v_T$ и итоговый лосс L. Вводят **сопряжённые (adjoint) величины**:

$$\bar{v} \stackrel{\text{def}}{=} \frac{\partial L}{\partial v}.$$

Тогда для узла $v = h(u_1, \ldots, u_k)$ локально применяют цепочку:

$$\bar{u}_j \; += \; \bar{v} \, \frac{\partial v}{\partial u_j}, \qquad j = 1, \ldots, k,$$

и распространяют $\bar{\cdot}$ **в обратном порядке** (от L к входам/параметрам).

Часто используемые локальные производные (минимальный набор):

- $v = u + w \Rightarrow \frac{\partial v}{\partial u} = 1, \; \frac{\partial v}{\partial w} = 1.$
- $v = u \cdot w \Rightarrow \frac{\partial v}{\partial u} = w, \; \frac{\partial v}{\partial w} = u.$
- $z = Wx + b \Rightarrow \nabla_x L = W^\top \nabla_z L, \; \nabla_W L = (\nabla_z L)\, x^\top, \; \nabla_b L = \nabla_z L.$
- $\sigma(z) = \frac{1}{1+e^{-z}} \Rightarrow \sigma'(z) = \sigma(z)(1 - \sigma(z)).$
- $\mathrm{ReLU}(z) = \max(0, z) \Rightarrow \mathrm{ReLU}'(z) = \mathbb{I}[z > 0]$ (субградиент в $z = 0$).

38.2 Посчитать простой пример вручную

Посчитаем backprop вручную на **простой сети с одним скрытым нейроном** и MSE.

Модель (граф).

$$z_1 = w_1 x + b_1, \qquad a_1 = \sigma(z_1), \qquad z_2 = w_2 a_1 + b_2, \qquad L = \frac{1}{2}(z_2 - y)^2.$$

Здесь $\sigma(\cdot)$ — сигмоида.

Числа для примера.

$$x = 1, \quad y = 0, \quad w_1 = 2, \quad b_1 = -1, \quad w_2 = -3, \quad b_2 = 0.5.$$

Forward pass.

$$z_1 = 2 \cdot 1 + (-1) = 1, \qquad a_1 = \sigma(1) = \frac{1}{1 + e^{-1}} \approx 0.731058579.$$

$$z_2 = (-3) \cdot 0.731058579 + 0.5 \approx -1.693175736.$$

$$L = \frac{1}{2}(z_2 - y)^2 = \frac{1}{2}(-1.693175736)^2.$$

(Значение L для градиентов не обязательно, важнее производные.)

Backward pass. Начинаем с

$$\bar{z}_2 = \frac{\partial L}{\partial z_2} = z_2 - y \approx -1.693175736.$$

Градиенты верхнего линейного слоя:

$$\frac{\partial L}{\partial w_2} = \bar{z}_2 \cdot \frac{\partial z_2}{\partial w_2} = \bar{z}_2 \cdot a_1 \approx (-1.693175736) \cdot 0.731058579 \approx -1.237810648,$$

$$\frac{\partial L}{\partial b_2} = \bar{z}_2 \cdot \frac{\partial z_2}{\partial b_2} = \bar{z}_2 \approx -1.693175736.$$

Проталкиваем градиент к a_1:

$$\bar{a}_1 = \frac{\partial L}{\partial a_1} = \bar{z}_2 \cdot \frac{\partial z_2}{\partial a_1} = \bar{z}_2 \cdot w_2 \approx (-1.693175736) \cdot (-3) = 5.079527208.$$

Производная сигмоиды в точке z_1:

$$\sigma'(z_1) = a_1(1 - a_1) \approx 0.731058579 \cdot 0.268941421 \approx 0.196611933.$$

Значит

$$\bar{z}_1 = \frac{\partial L}{\partial z_1} = \bar{a}_1 \cdot \frac{\partial a_1}{\partial z_1} = \bar{a}_1 \cdot \sigma'(z_1) \approx 5.079527208 \cdot 0.196611933 \approx 0.998695663.$$

Градиенты нижнего линейного слоя:

$$\frac{\partial L}{\partial w_1} = \bar{z}_1 \cdot \frac{\partial z_1}{\partial w_1} = \bar{z}_1 \cdot x \approx 0.998695663 \cdot 1 \approx 0.998695663,$$

$$\frac{\partial L}{\partial b_1} = \bar{z}_1 \cdot \frac{\partial z_1}{\partial b_1} = \bar{z}_1 \approx 0.998695663.$$

Итог (градиенты параметров):

$$\boxed{\frac{\partial L}{\partial w_2} \approx -1.237810648, \quad \frac{\partial L}{\partial b_2} \approx -1.693175736, \quad \frac{\partial L}{\partial w_1} \approx 0.998695663, \quad \frac{\partial L}{\partial b_1} \approx 0.998695663.}$$

Проверка здравым смыслом (интуитивно): так как $z_2 < 0$ и $y = 0$, градиент $\bar{z}_2 = z_2 - y < 0$ — модель “хочет” поднять z_2 ближе к нулю. Из $\frac{\partial L}{\partial b_2} \approx -1.69$ следует, что шаг градиентного спуска увеличит b_2 (так как $b_2 \leftarrow b_2 - \eta\, \partial L/\partial b_2$), что действительно увеличивает z_2.

39 Многоклассовая классификация в нейросетях

39.1 Softmax

39.1.1 Основные свойства

В многоклассовой классификации обычно предполагают K взаимоисключающих классов $c \in \{1, \dots, K\}$. Нейросеть выдаёт **логиты** (скоры) $z(x) \in \mathbb{R}^K$, где z_k — "сырое" (не нормированное) предпочтение класса k. Чтобы получить вероятностное распределение по классам, используют **softmax**:

$$p_k = \mathrm{softmax}(z)_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad k = 1, \dots, K.$$

Основные свойства softmax:

- **Нормировка до вероятностей:**

$$p_k \in (0, 1), \qquad \sum_{k=1}^{K} p_k = 1.$$

  Значит p — корректное распределение.

- **Инвариантность к добавлению константы (shift-invariance):** для любого $c \in \mathbb{R}$:

$$\mathrm{softmax}(z)_k = \mathrm{softmax}(z + c\mathbf{1})_k.$$

  Доказательство: домножение числителя и знаменателя на e^c сокращается.

- **Сохранение порядка по логитам:** если $z_a > z_b$, то $p_a > p_b$ (монотонность по каждому компоненту при фиксированных остальных).

- **"Усиление" разницы логитов:** если разность $z_a - z_b$ растёт, то отношение вероятностей растёт экспоненциально:

$$\frac{p_a}{p_b} = \exp(z_a - z_b).$$

- **Связь с многоклассовой логистической регрессией:** softmax — естественное обобщени сигмоиды на $K > 2$ классов.

Якобиан softmax (часто используют в выводах). Пусть $p = \mathrm{softmax}(z)$. Тогда для $k, l \in \{1, \dots, K\}$:

$$\boxed{\frac{\partial p_k}{\partial z_l} = p_k(\delta_{kl} - p_l),}$$

где δ_{kl} — символ Кронекера.

39.1.2 Численная стабильность

Наивное вычисление e^{z_k} может приводить к переполнению (overflow), если логиты большие. Используют свойство shift-invariance и вычитают максимум:

$$z_{\max} = \max_j z_j, \qquad p_k = \frac{e^{z_k - z_{\max}}}{\sum_{j=1}^{K} e^{z_j - z_{\max}}}.$$

Тогда все экспоненты считают числа ≤ 1, что резко повышает устойчивость.

Численно стабильный log-softmax. Часто требуется $\log p_k$ (для кросс-энтропии). Тогда используют формулу

$$\log p_k = z_k - \log \sum_{j=1}^{K} e^{z_j}.$$

Численно стабильно:

$$\boxed{\log p_k = (z_k - z_{\max}) - \log \sum_{j=1}^{K} e^{z_j - z_{\max}}.}$$

В практических фреймворках обычно реализуют `log_softmax` как отдельную устойчивую операцию.

39.2 Потеря кросс-энтропии

39.2.1 Определения информационного содержания и энтропии

В информационной теории:

Информационное содержание (self-information). Для события A с вероятностью $p = \mathbb{P}(A)$:

$$\boxed{I(A) = -\log p.}$$

Интуиция: чем событие менее вероятно, тем больше "информации" несёт его наступление. Обычно логарифм берут по основанию 2 (бит) или e (нат).

Энтропия распределения. Для дискретного распределения $q = (q_1, \dots, q_K)$:

$$\boxed{H(q) = -\sum_{k=1}^{K} q_k \log q_k.}$$

Интуиция: мера неопределённости. Максимальна при равномерном распределении, минимальна (0) при вырожденном (one-hot).

39.2.2 Мотивация и смысл кросс-энтропии

Кросс-энтропия между "истинным" распределением q и "модельным" распределением p:

$$\boxed{H(q, p) = -\sum_{k=1}^{K} q_k \log p_k.}$$

Смысл: ожидаемая длина кодирования (в среднем количество информации), если данные приходят из q, а код (или модель) устроен под p.

В ML: в многоклассовой классификации истинная метка y обычно кодируется one-hot вектором q:

$$q_k = \mathbb{I}[k = y].$$

Тогда кросс-энтропия для одного объекта упрощается:

$$H(q, p) = -\log p_y.$$

То есть минимизировать кросс-энтропию — значит максимизировать вероятность правильного класса.

Кросс-энтропия как отрицательный лог-правдоподобие (NLL). Если модель задаёт распределение $p_\theta(y \mid x)$ (softmax над логитами), то для выборки:

$$\mathcal{L}(\theta) = -\sum_{i=1}^{n} \log p_\theta(y_i \mid x_i).$$

Это ровно **MLE** для параметров θ (максимизация правдоподобия эквивалентна минимизации NLL).

Градиент по логитам (важный результат для backprop). Пусть $p = \mathrm{softmax}(z)$, q — one-hot. Тогда

$$\boxed{\frac{\partial}{\partial z_k}\Big(-\sum_j q_j \log p_j\Big) = p_k - q_k.}$$

Это объясняет, почему для softmax+cross-entropy градиенты получаются особенно простыми.

39.2.3 Perplexity

Перплексия (perplexity) — метрика качества вероятностной модели, особенно в языковом моделировании. Если кросс-энтропия измеряется в **натах** (логарифм по основанию e), то:

$$\boxed{\mathrm{PPL} = \exp\big(H(q, p)\big).}$$

Если логарифм по основанию 2, то:

$$\mathrm{PPL} = 2^{H(q,p)}.$$

Интуиция: "эффективное" число равновероятных вариантов, которое модель "видит" в среднем.

- PPL меньше $\Rightarrow$ модель лучше (меньше неопределённость).

- для идеального one-hot предсказания $p_y = 1$ кросс-энтропия 0, перплексия 1.

39.2.4 Дивергенция Кульбака–Лейблера

KL-дивергенция (Kullback–Leibler divergence) между распределениями q и p:

$$\boxed{D_{\mathrm{KL}}(q\|p) = \sum_{k=1}^{K} q_k \log \frac{q_k}{p_k}.}$$

Свойства:

- $D_{\mathrm{KL}}(q\|p) \geq 0$ и равна 0 тогда и только тогда, когда $q = p$ (почти всюду).
- Не симметрична: $D_{\mathrm{KL}}(q\|p) \neq D_{\mathrm{KL}}(p\|q)$.

Связь кросс-энтропии, энтропии и KL. Раскроем:

$$D_{\mathrm{KL}}(q\|p) = \sum_k q_k \log q_k - \sum_k q_k \log p_k = -H(q) + H(q,p).$$

Следовательно:

$$\boxed{H(q,p) = H(q) + D_{\mathrm{KL}}(q\|p).}$$

Поскольку $H(q)$ не зависит от модели p, минимизация кросс-энтропии по параметрам модели эквивалентна минимизации $D_{\mathrm{KL}}(q\|p)$, то есть приближению модельного распределения к истинн

Частный случай one-hot меток. Если q — one-hot, то $H(q) = 0$, и

$$H(q,p) = D_{\mathrm{KL}}(q\|p) = -\log p_y.$$

40 Компьютерное железо для ML

40.1 Архитектура фон Неймана

Архитектура фон Неймана — классическая модель вычислительной системы, в которой **програм и данные хранятся в общей памяти**, а процессор выполняет цикл **fetch–decode–execute** (выборка инструкции, декодирование, исполнение).

Ключевые компоненты (упрощённо):

- **CPU**: блок управления (Control Unit), арифметико-логическое устройство (ALU), регистры
- **Память**: хранит и инструкции, и данные.
- **Шины (bus)**: соединяют CPU с памятью и устройствами ввода-вывода.

Историческая (материнские платы до 2010-х) двухчиповая схема:

- **Northbridge** (memory controller hub): связь CPU–RAM, часто также высокоскоростная связь с GPU.
- **Southbridge** (I/O controller hub): периферия и ввод-вывод (PCI, SATA, Ethernet, USB и др.).

Узкое место: фон-неймановское "бутылочное горлышко" (von Neumann bottleneck). Скорость вычислений часто ограничена не ALU, а пропускной способностью и латентност доступа к памяти: процессору нужно постоянно подкачивать данные/инструкции, и при промаха кэша он простаивает.

40.1.1 Принцип локальности данных

Принцип локальности (locality of reference) объясняет, почему кэширование работает: обращения к памяти *не случайны*, а имеют структуру.

- **Временная локальность (temporal locality):** если элемент данных использовали недавно, с высокой вероятностью его используют снова скоро (например, веса модели и активные фрагменты кода в циклах обучения).
- **Пространственная локальность (spatial locality):** если обратились к адресу памяти, с высокой вероятностью скоро обратятся к соседним адресам (например, последовательный проход по массиву, тензору, батчу).

Иерархия памяти и порядок величин латентностей (идея):

- регистры / L1 cache: единицы наносекунд;
- L2/L3 cache: десятки наносекунд;
- DRAM (оперативная память): сотни наносекунд;
- SSD/HDD: микросекунды–миллисекунды;
- сеть: миллисекунды и выше.

Отсюда практический вывод для ML: важно организовывать вычисления так, чтобы данные использовались **блоками** и **переиспользовались**, минимизируя дорогие обращения в "нижние уровни иерархии.

40.1.2 Поток данных при обучении модели

При обучении модели типовой поток данных (на примере фреймворков уровня PyTorch) выглядит так:

- **Источник данных:** интернет/файлы на диске (HDD/SSD) → чтение в систему.
- **Dataset:** логика чтения/декодирования одного объекта за раз (пример: открыть файл, распарсить строку, декодировать изображение).
- **CPU + RAM:** данные попадают в оперативную память; CPU выполняет подготовку: токенизация/аугментации/нормализация/фичи, а также **batching**.
- **Dataloader:** сборка батчей (*collate*), предобработка, распараллеливание по потокам/процес префетчинг нескольких батчей в RAM для уменьшения простоев.
- **Передача на GPU:** подготовленный батч копируется в видеопамять (VRAM) по PCIe/ана шине.
- **GPU compute:** на GPU выполняются прямой проход (forward), вычисление функции потерь, обратный проход (backward) и вычисление градиентов; оптимизатор обновляет параметры (часто тоже на GPU).

Практические следствия для скорости обучения:

- если CPU/dataloader не успевает готовить батчи, GPU простаивает (bottleneck в input pipeline);
- если батч слишком мал или модель слишком лёгкая, GPU может быть недогружен (недостат параллелизма);
- если данные часто "ездят" CPU↔GPU, возрастает overhead копирований и синхронизаций;
- в multi-GPU (например, DDP) добавляется обмен градиентами между GPU (коммуникации могут стать bottleneck).

40.2 Таксономия Флинна

Таксономия Флинна (Flynn's taxonomy) — классификация архитектур по количеству потоков инструкций и данных:

$$\text{Instruction streams} \times \text{Data streams}.$$

Четыре базовых класса:

- **SISD** (Single Instruction, Single Data): один поток инструкций, один поток данных (классиче последовательный процессор).
- **SIMD** (Single Instruction, Multiple Data): одна инструкция применяется параллельно к множеству данных (векторные инструкции).
- **MISD** (Multiple Instruction, Single Data): несколько потоков инструкций обрабатывают один поток данных (редко встречается как "чистый" класс).
- **MIMD** (Multiple Instruction, Multiple Data): несколько потоков инструкций и данных (многопроцессорные/многоядерные системы).

40.2.1 Место CPU и GPU

CPU (современный многоядерный):

- на уровне ядер и потоков ОС — в первую очередь **MIMD** (разные ядра выполняют разные потоки, разные ветвления);
- внутри одного ядра часто есть элементы **SIMD** (векторные расширения, обрабатывающие несколько чисел за инструкцию).

GPU:

- по смыслу ближе к **SIMD**-классу (массовая параллельная обработка данных одной и той же программой), так как тысячи потоков выполняют одну и ту же "ядро-функцию" (kernel) над разными элементами данных;
- на практике модель исполнения у GPU обычно описывают как **SIMT** (Single Instruction, Multiple Threads) — концептуально близко к SIMD, но формально описывается через потоки (threads) и группировку в "варпы/вэйвфронты".

40.2.2 CPU vs GPU, основные различия

Интуитивная метафора: CPU похож на "легковой автомобиль" (быстро выполняет разнообраз задачи с ветвлениями), GPU — на "автобус" (эффективен, когда нужно одновременно перевезти много однотипных операций).

Главные различия архитектурно и по профилю задач:

- **Цель проектирования:**
  - CPU оптимизирован под **низкую задержку** (latency) и сложный control-flow (ветвлени указатели, системные вызовы).
  - GPU оптимизирован под **высокую пропускную способность** (throughput) и data-parallel вычисления (матрицы/тензоры, свёртки, батчевые операции).
- **Ресурсы на управление vs вычисления:**
  - CPU содержит сравнительно **мало** мощных ядер, много логики управления, предсказан ветвлений, большие кэши (L1/L2 на ядро, общий L3).
  - GPU содержит **много** более простых вычислительных блоков (много "ядер"/ALU), относительно меньше логики управления на единицу вычисления, обычно иной баланс кэш/память.
- **Параллелизм:**
  - CPU: ограниченное число потоков, но каждый поток "универсален" (MIMD).
  - GPU: огромный параллелизм, но наиболее эффективен при одинаковом коде и схожем доступе к памяти (избегание divergence ветвлений внутри варпа).
- **Память и данные:**
  - CPU работает в тесной связке с RAM и развитой кэш-иерархией, хорошо справляется с нерегулярными доступами.
  - GPU имеет собственную DRAM (VRAM) с высокой пропускной способностью; выигрыв на плотных тензорных операциях, но чувствителен к паттернам доступа и к стоимости копирования CPU↔GPU.
- **ML-следствие:**
  - обучение нейросетей и большие матричные операции обычно быстрее на GPU;
  - подготовка данных, сложный препроцессинг, логика пайплайна часто остаются на CPU (и могут ограничивать скорость).

40.3 NVIDIA CUDA, что это

CUDA (Compute Unified Device Architecture) — проприетарная (vendor-specific) платформа параллельных вычислений и API, которая позволяет использовать GPU NVIDIA для **ускоренн вычислений общего назначения** (GPGPU).

Что даёт CUDA на практике:

- модель программирования, где разработчик описывает **kernel** (функцию), выполняемую параллельно на множестве потоков;

- управление **параллелизмом** (threads, blocks, grid) и **памятью** (копирование CPU↔GPU, размещение данных в VRAM);
- доступ к оптимизированным библиотекам для линейной алгебры и DL-операций (типовой стек: матричные умножения, свёртки и т.п.).

Связь с языком C/C++ (идея из лекций): CUDA API рассматривают как расширение, позволяющее задавать **параллелизм на уровне потоков** и использовать GPU-специфичные операции (включая передачу данных между CPU и GPU и использование виртуального "инструк слоя GPU).

Связь с Python/ML-фреймворками:

- NumPy ориентирован на векторизацию на CPU;
- PyTorch/TF могут исполнять тензорные операции на GPU, используя CUDA как backend (когда выбран device `cuda`).

41 Библиотека PyTorch

41.1 Зачем нужна ещё одна матричная библиотека

PyTorch — это не только "ещё одна" библиотека линейной алгебры, а **платформа для обучения нейросетей end-to-end**, которая объединяет:

- **тензоры** (обобщение матриц на n измерений) и быстрые операции над ними;
- **автоматическое дифференцирование (autograd)** — вычисление градиентов по вычисл графу;
- **исполнение на разных устройствах** (CPU/GPU и др.) с одинаковым Python-API;
- **готовые модули** для нейросетей, оптимизации, расписаний learning rate, работы с данным

Сравнение по мотивации (идея курса про backprop и "автоматические градиенты"):

- **NumPy**:
  - сильна как CPU-библиотека для научных вычислений;
  - **не** хранит вычислительный граф и не считает градиенты автоматически;
  - в чистом виде не решает задачу "обучаем модель по градиенту" без ручного вывода производных.
- **PyTorch**:
  - ключевая добавка — **autograd**: можно описывать модель как композицию операций, а градиенты будут вычислены автоматически (backprop как "framework for automatic gradients calculation");
  - удобен для экспериментов: Python-уровень, динамический граф (строится при выполне forward), быстрые итерации разработки;
  - ускорение на GPU и оптимизированные ядра операций делают обучение практичным по времени.

Практический вывод: PyTorch нужен, когда вы хотите не просто посчитать математику, а **обучать параметризованные модели** (линейные слои, свёртки, attention и т.д.) градиентным методами на больших данных и/или GPU.

41.2 Доступные бекенды

Под "бекендами" в PyTorch обычно понимают два близких слоя:

(A) Бекенд устройства (device backend) — где выполняются тензорные операции:

- **CPU** (`device='cpu'`): выполнение на центральном процессоре с использованием оптимизац (векторизация, многопоточность, BLAS/oneDNN-подобные библиотеки в зависимости от сборки).
- **CUDA GPU** (`device='cuda'`): выполнение на GPU NVIDIA через CUDA-стек (ядра матмулов, свёрток и др.).
- **Альтернативные GPU/ускорители** (в зависимости от сборки/пакетов):
  - AMD GPU через ROCm (обычно также используется `cuda`-совместимый API на уровне кода, но это зависит от окружения);
  - Apple Silicon GPU через MPS (`device='mps'`);
  - TPU/прочие XLA-устройства через `torch_xla` (отдельный пакет, `device='xla'` в логике выполнения).

(B) Математические/низкоуровневые бекенды (kernel libraries) — чем именно реализую операции:

- для CPU: BLAS/векторные оптимизации, специализированные ядра для свёрток/нормировок и т.п.;
- для CUDA: библиотеки и ядра для матричных операций и deep learning примитивов (умножения матриц, свёртки, нормализации).

Также в распределённом обучении есть отдельное понятие **communication backends** (для обмена тензорами/градиентами): типичные варианты — `nccl` (GPU), `gloo` (CPU/GPU), `mpi` (если подключён).

41.3 Архитектура, основные модули и классы

41.3.1 nn.Module, Sequential, Optimizers, Schedulers, Criterions

Слой архитектуры PyTorch (сверху вниз):

- `torch.Tensor` — базовый тип данных (тензор), хранит:
  - значения (данные),
  - устройство (`cpu`/`cuda`/`mps`/...),
  - dtype,
  - флаг `requires_grad` (нужно ли считать градиенты).
- `torch.autograd` — автоматическое дифференцирование:
  - при выполнении операций над тензорами строится вычислительный граф;
  - вызов `loss.backward()` запускает backprop и заполняет `param.grad`.
- `torch.nn` — модуль нейросетей (слои, контейнеры, функции потерь):
  - содержит `nn.Module` (базовый класс для моделей/слоёв),
  - готовые слои (`Linear`, `Conv2d`, `LayerNorm`, ...),
  - функции потерь (`CrossEntropyLoss`, `MSELoss`, ...).
- `torch.optim` — оптимизаторы (SGD/Adam/и др.) и шаг обновления параметров.
- `torch.utils.data` — работа с данными: `Dataset`, `DataLoader`, `Sampler`.

`nn.Module`. Это базовый класс для слоёв и моделей. Типичные свойства/методы:

- **параметры** (`nn.Parameter`) автоматически регистрируются и возвращаются через `model.p`
- **буферы** (buffers) для не обучаемых, но сохраняемых тензоров (например, статистики нормировок).
- `forward` описывает вычисление прямого прохода. Вызов `model(x)` внутри вызывает `forward` и строит вычислительный граф для autograd.
- режимы `train()` и `eval()` (важно для Dropout/BatchNorm).
- **перенос устройства** `model.to(device)` и сохранение `state_dict()`.

`nn.Sequential`. Контейнер, который последовательно применяет модули:

$$x \xrightarrow{m_1} h_1 \xrightarrow{m_2} h_2 \xrightarrow{\cdots} \hat{y}.$$

Удобен для "слой за слоем" архитектур без ветвлений/скипов. Для сложных графов обычно пишут собственный `nn.Module` и `forward`.

Optimizers (оптимизаторы). Оптимизатор получает список параметров и обновляет их по градиентам:

- `optimizer = torch.optim.SGD(model.parameters(), lr=..., momentum=...)`
- `optimizer = torch.optim.Adam(model.parameters(), lr=..., weight_decay=...)`

Ключевые шаги итерации обучения:

`zero_grad` → `forward` → `loss` → `backward` → `step`.

На практике (важно помнить): перед новым шагом нужно обнулять градиенты, т.к. они **накапливаются**.

Schedulers (расписания learning rate). Планировщик меняет learning rate во времени (по эпохам или итерациям):

- StepLR, CosineAnnealingLR, ReduceLROnPlateau и др.

Это связано с динамикой оптимизации: иногда выгодно начать с большего шага и постепенно снижать.

Criterions (loss functions). В PyTorch потери обычно берут из `torch.nn`:

- `nn.CrossEntropyLoss` для многоклассовой классификации (обычно ожидает **логиты**).
- `nn.BCEWithLogitsLoss` для бинарной (логиты + стабильная сигмоида внутри).
- `nn.MSELoss`, `nn.L1Loss` для регрессии и т.д.

Потеря возвращает скаляр (или батч-скаляры) и служит входом в `backward()`.

Мини-псевдокод train-step (идея, без привязки к конкретной задаче):

```
model.train()
for x, y in train_loader:
    x, y = x.to(device), y.to(device)
    optimizer.zero_grad()
    logits = model(x)           # forward
    loss = criterion(logits, y) # loss
    loss.backward()             # backward (autograd)
    optimizer.step()            # update params
```

41.4 Dataflow в PyTorch, классы и их назначение

Dataflow в PyTorch удобно описывать как путь данных:
Storage (HDD/SSD) → `Dataset` → (Transforms) → `DataLoader(workers, collate)` → Ba

Dataset. Определяет, *как получить один элемент* данных:

- **Map-style Dataset** (индексируемый): реализует `__len__()` и `__getitem__(i)`.
  Тогда один элемент обычно: (x_i, y_i) или словарь/кортеж тензоров.
- **IterableDataset**: реализует `__iter__()`, полезен для потоковых источников, больших файлов, генераторов.

Transforms / preprocessing. Преобразования данных (нормализация, аугментации, токенизац и т.п.) обычно делают:

- либо внутри `Dataset` (в `__getitem__`),
- либо отдельными объектами/функциями, которые применяются к каждому элементу.

Идея из лекции: `Dataset` "достаёт" сырой объект, затем применяется **transform**.

DataLoader. Оборачивает `Dataset` и создаёт итератор по батчам:

- `batch_size`: размер мини-батча (batch / mini-batch).
- `shuffle`: перемешивание (обычно **True** для train и **False** для val/test).
- `num_workers`: число процессов/воркеров для параллельной загрузки и preprocessing.
- `collate_fn`: функция сборки батча из списка элементов Dataset (например, из списка объектов сделать один батч-тензор или батч словарей).

Sampler / BatchSampler (контроль порядка и батчинга).

- `Sampler` задаёт, *в каком порядке* брать индексы (последовательно, случайно, с весами).
- `BatchSampler` группирует индексы в батчи (может быть полезно для нестандартных схем батчинга).

Train/Val/Test splits (практический поток). Часто делают три выборки и три DataLoader:

- **train_loader**: `shuffle=True`, используются градиенты и обновление параметров.
- **val_loader**: `shuffle=False`, измерение качества для подбора гиперпараметров/early stopping.
- **test_loader**: `shuffle=False`, финальная оценка.

Epoch и iteration (связь с SGD).

- **iteration (шаг)**: один батч → один шаг оптимизатора (`optimizer.step()`).
- **epoch (эпоха)**: один полный проход по train_loader (по всем батчам обучающей выборки).

Перенос батча на устройство и режимы вычислений.

- Обычно батч формируется в RAM, затем переносится на GPU: `x = x.to(device)`.
- Для инференса/валидации используют:
  `model.eval()` и `torch.no_grad()`,
  чтобы отключить dropout/обновление статистик нормировок и не хранить вычислительный граф (экономия памяти/времени).

42 Обработка текстов

42.1 Подходы к токенизации (минимум 3)

Токенизация — разбиение исходного текста на последовательность единиц (токенов), с которым дальше работает модель:

$$\text{“сырой текст”} \longrightarrow (t_1, t_2, \ldots, t_n),$$

где токены t_i могут быть словами, подсловами, символами или специальными единицами.

Токенизация определяет:

- размер словаря (vocabulary) и долю **неизвестных** токенов (OOV — out-of-vocabulary);
- длину последовательностей (а значит и стоимость обучения/инференса);
- устойчивость к опечаткам, редким словам, морфологии (особенно важно для русского языка).

Основные подходы к токенизации (минимум 3, на практике их больше):

1. **Токенизация по словам (word-level).**
   Самый простой подход: разделение по пробелам и знакам пунктуации (часто с регулярными выражениями).
   - **Плюсы:** интерпретируемо, короткие последовательности.
   - **Минусы:** большой словарь, проблема OOV, плохо переносит опечатки и богатую морфологию.
2. **Токенизация по символам (character-level).**
   Текст представляется как последовательность символов (включая пробелы/пунктуацию).
   - **Плюсы:** почти нет OOV (алфавит ограничен), высокая устойчивость к опечаткам.
   - **Минусы:** длинные последовательности $\Rightarrow$ дороже обучение, сложнее захватывать дальние зависимости.
3. **Токенизация по подсловам (subword-level) — стандарт де-факто для трансформе**
   Идея: разложить слово на частотные части (морфемоподобные фрагменты), чтобы:
   - уменьшить OOV (редкие слова собираются из известных подслов);
   - ограничить размер словаря;
   - сохранить разумную длину последовательности.

   Примеры подсловных алгоритмов (их можно назвать как отдельные подходы):
   - **BPE** (Byte-Pair Encoding): итеративно склеивает наиболее частотные пары символов/п
   - **WordPiece**: близок к BPE, но выбор слияний основан на максимизации правдоподобия сегментации.
   - **Unigram LM** (SentencePiece Unigram): выбирает сегментации вероятностной моделью подслов и оптимизирует словарь через “удаление” наименее полезных токенов.

   Часто используются специальные маркеры продолжения подслова (например, “##” или знак границы слова).
4. **N-граммная токенизация (обычно на уровне символов/слов).**
   Разбиение на последовательности фиксированной длины (bi-grams, tri-grams и т.д.).
   - **Плюсы:** может кодировать локальный контекст, устойчивее к редким словам, чем word-level.
   - **Минусы:** словарь и количество признаков быстро растут; часто используется в классич моделях (BoW/TF-IDF) и реже в современных DL-моделях как основная токенизация.
5. **Токенизация байтами (byte-level).**
   Представление текста как последовательности байтов (0–255) либо BPE поверх байтов.

- **Плюсы:** универсально для любых языков/символов, почти нет OOV.
- **Минусы:** более длинные последовательности, чем у subword; не всегда удобно интерпре

Практическая ремарка: для русского языка word-level токенизация сильно страдает из-за морфологии (много словоформ), поэтому подсловная токенизация (BPE/WordPiece/Unigram) обычно предпочтительнее.

42.2 Лемматизация, стемминг

Лемматизация — приведение слова к **лемме** (словарной форме) с учётом морфологии и контекста:

$$\text{“кошками”} \to \text{“кошка”}, \qquad \text{“went”} \to \text{“go”}.$$

- Обычно требует морфологического анализатора и словарей (или статистической модели).
- **Плюсы:** более “правильная” нормализация, полезна для поиска, классического NLP, некоторых задач тематического моделирования.
- **Минусы:** сложнее, медленнее, зависит от языка, возможны ошибки на омонимии и нестандартных формах.

Стемминг — эвристическое усечение слова до **основы (stem)** без гарантии получения реальной словарной формы:

$$\text{“connecting”} \to \text{“connect”}, \qquad \text{“машины”} \to \text{“машин”}.$$

- Как правило, это набор правил по удалению/замене суффиксов.
- **Плюсы:** быстро и просто, уменьшает словарь.
- **Минусы:** может давать несуществующие “корни”, хуже различает смыслы, возможны ошибки двух типов:
  - **over-stemming:** разные слова склеиваются в один stem;
  - **under-stemming:** близкие формы остаются разными.

Когда что использовать (типовая логика):

- для классических пайплайнов (BoW/TF-IDF, простые классификаторы) стемминг/леммати часто улучшают качество;
- для современных нейросетей с subword токенизацией (BPE/WordPiece/Unigram) часто **не нужно** делать лемматизацию/стемминг, потому что модель и токенизатор уже обрабатыва вариативность форм.

42.2.1 Назвать базовые стеммеры

Классические (базовые) алгоритмы стемминга, которые обычно называют:

- **Porter Stemmer** (Алгоритм Портера) — один из самых известных для английского; правило-ориентированное удаление суффиксов.
- **Snowball Stemmer** (Porter2) — улучшенная и расширяемая версия, реализована для многих языков.
- **Lancaster Stemmer** (Paice/Husk) — более агрессивный стеммер (сильнее “режет” слова).

Для русского языка часто упоминают:

- **Russian Snowball Stemmer** (вариант Snowball для русского);
- (на практике) **стеммеры на правилах суффиксов** в составе NLP-библиотек, аналогичны идеям Snowball.

43 Векторизация слов

Векторизация слов — преобразование дискретных токенов (слов) в численные векторы, пригодные для ML-моделей:

$$w \in \mathcal{V} \longrightarrow x(w) \in \mathbb{R}^d,$$

где $\mathcal{V}$ — словарь.

43.1 One-hot encoding

One-hot encoding — самый простой способ кодирования слова: если словарь размера $|\mathcal{V}| = V$, то слову w соответствует вектор $e(w) \in \{0,1\}^V$, где ровно один компонент равен 1 (на позиции слова), остальные 0:

$$e(w)_i = \begin{cases} 1, & i = \mathrm{idx}(w), \\ 0, & \text{иначе.} \end{cases}$$

Тогда bag-of-words представление документа можно строить суммой/частотами one-hot векторов

43.1.1 Недостатки

- **Очень высокая размерность.** V может быть 10^5–10^7 (в зависимости от корпуса и токенизации), что делает хранение и вычисления дорогими (хотя разреженность помогает, это всё равно неудобно как представление "слова").
- **Нет семантики: ортогональность.** Для разных слов $w_1 \neq w_2$:
  $$e(w_1)^\top e(w_2) = 0,$$
  то есть по one-hot нельзя сказать, что "кошка" ближе к "кот", чем к "космодром": все разные слова одинаково "далеки".
- **Не обобщает на редкие слова и OOV.** Если слово не в словаре, оно либо теряется, либо маппится в один общий `<UNK>`, что уничтожает различия между редкими словами.
- **Плохо работает с морфологией и опечатками.** "машина", "машины", "машиной" – разные one-hot, хотя смысл близок; опечатка превращает слово в OOV.
- **В линейных моделях не кодирует контекст.** Один и тот же one-hot для слова в разных контекстах, омонимия не различается.

Вывод: one-hot полезен как базовый способ индексации и для простых классических моделей (BoW/TF-IDF), но для смысловых представлений нужны **плотные эмбеддинги**.

43.2 Word2Vec, GloVe, FastText

43.2.1 Постановка задачи, функция потерь, признаки

Общая идея распределённых представлений (distributional hypothesis): слова, встреча в похожих контекстах, должны иметь близкие векторы. Эмбеддинг обучается так, чтобы предсказывать контекст по слову или слово по контексту.

Нотация. Пусть есть корпус токенов $(w_1, \ldots, w_T)$ и окно контекста размера c. Контекст для позиции t:

$$C_t = \{w_{t-j}\}_{j=1}^{c} \cup \{w_{t+j}\}_{j=1}^{c}.$$

Обучаем эмбеддинги слов (и часто отдельно эмбеддинги "контекстов").

1) Word2Vec.
Есть две базовые постановки:

(a) CBOW (Continuous Bag-of-Words): по контексту предсказать центральное слово.

$$p(w_t \mid C_t).$$

(b) Skip-gram: по центральному слову предсказать слова контекста.

$$p(w_{t+j} \mid w_t), \qquad j \in \{-c, \ldots, -1, 1, \ldots, c\}.$$

Параметризация через softmax (идея). Пусть v_w — эмбеддинг слова w ("input"), u_w — эмбеддинг слова как "контекст" ("output"). Тогда

$$p(w_o \mid w_i) = \frac{\exp(u_{w_o}^\top v_{w_i})}{\sum_{w \in \mathcal{V}} \exp(u_w^\top v_{w_i})}.$$

Функция потерь для Skip-gram (NLL):

$$\mathcal{L} = -\sum_{t=1}^{T} \sum_{\substack{-c \le j \le c \\ j \neq 0}} \log p(w_{t+j} \mid w_t).$$

Негативное сэмплирование (Negative Sampling) — практическая версия. Полный softmax дорог (сумма по словарю), поэтому используют бинарную классификацию "пара (центр, контекст) истинная или шумовая". Для наблюдаемой пары (w_i, w_o) максимизируют:

$$\log \sigma(u_{w_o}^\top v_{w_i}) + \sum_{k=1}^{K} \mathbb{E}_{w_k \sim P_n} \left[\log \sigma(-u_{w_k}^\top v_{w_i}) \right],$$

где K — число негативных примеров, P_n — распределение шума, σ — сигмоида.

Какие "признаки" у Word2Vec.

- Вход в модели — индексы слов (one-hot), но обучаемое представление — **матрица эмбедди** $V \times d$ (строка на слово).
- Контекст задаётся через окно в тексте; фактически модель учит векторы так, чтобы скалярное произведение отражало совместную встречаемость слова и контекста.

2) GloVe.

GloVe (Global Vectors) использует **глобальную** статистику совместной встречаемости слов. Строится матрица $X \in \mathbb{R}^{V \times V}$, где X_{ij} — число раз, когда слово j встречалось в контексте слова i.

Постановка: подобрать векторы v_i, u_j и смещения b_i, c_j так, чтобы

$$v_i^\top u_j + b_i + c_j \approx \log X_{ij}.$$

Функция потерь (взвешенная MSE):

$$\boxed{\mathcal{L} = \sum_{i=1}^{V} \sum_{j=1}^{V} f(X_{ij}) \left(v_i^\top u_j + b_i + c_j - \log X_{ij} \right)^2,}$$

где $f(\cdot)$ — весовая функция, которая уменьшает влияние очень редких и очень частых пар. Интуиция: модель приближает лог-ко-встречаемости, балансируя локальную (окно) и глобальну (по всему корпусу) информацию.

Какие "признаки" у GloVe.

- Основной "признак" — глобальная статистика co-occurrence X_{ij}.
- Эмбеддинги оптимизируются так, чтобы скалярные произведения отражали отношения встречаемости.

3) FastText.

FastText расширяет word2vec тем, что представляет слово через **символьные n-граммы** (subword units). Слово w рассматривается как набор n-грамм $G(w)$ (например, для $n = 3$ это триграммы символов).

Представление слова:

$$v_w = \sum_{g \in G(w)} z_g,$$

где z_g — эмбеддинг n-граммы g. Далее обучение проводится аналогично Skip-gram/CBOW (обычно с negative sampling), но вместо отдельного вектора на слово используется сумма по n-граммам.

Плюсы FastText (следуют из постановки):

- лучше работает для **редких слов** и **морфологически богатых языков** (русский): общие части слов разделяют параметры;
- способен строить векторы для **OOV-слов**: даже если слово не было в словаре, его n-граммы могут быть известны.

Сравнение по смыслу (коротко):

- Word2Vec — локальная предсказательная постановка по окнам контекста (predictive, local).
- GloVe — регрессия на глобальную матрицу совместной встречаемости (count-based, global).
- FastText — Word2Vec + subword (символьные n-граммы) для морфологии и OOV.

44 Векторизация текста

44.1 Bag-of-words

Векторизация текста — преобразование документа (строки/текста) в численный вектор признаков для ML-модели.

Bag-of-words (BoW) — классический подход, в котором документ представляют как **мешок слов**: порядок слов игнорируется, учитываются только факты появления и/или частоты.

Пусть словарь $\mathcal{V} = \{w_1, \ldots, w_V\}$, документ d — последовательность токенов. Тогда BoW-вектор:

$$x(d) = (x_1, \ldots, x_V) \in \mathbb{R}^V,$$

где компонент x_j обычно равен:

- **count BoW:** $x_j = \#(w_j \in d)$ — число вхождений слова w_j в документ;
- **binary BoW:** $x_j = \mathbb{I}[w_j \in d]$ — индикатор наличия слова.

Плюсы BoW:

- простота и высокая скорость построения;
- хорошо работает как baseline для линейных моделей;
- интерпретируемость (каждая координата = конкретное слово).

Минусы BoW:

- теряется порядок слов ⇒ плохо захватывается синтаксис и смысловые зависимости;
- высокая размерность (V большой), сильная разреженность;
- проблемы с синонимией и омонимией: разные слова не связываются, значения не различают по контексту;
- чувствительность к морфологии (особенно в русском), если не делать нормализацию (леммы/стеммы/подслова).

44.2 TF-IDF

TF-IDF (Term Frequency – Inverse Document Frequency) — взвешивание BoW, где:

- **TF** увеличивает вес слов, частых внутри документа;
- **IDF** уменьшает вес слов, которые часто встречаются во многих документах (менее информа

Пусть:

- N — число документов в коллекции;
- $\mathrm{df}(t)$ — document frequency: сколько документов содержат термин t;
- $\mathrm{tf}(t, d)$ — term frequency: сколько раз термин t встречается в документе d (или нормированна частота).

Одна из стандартных формул:

$$\mathrm{tfidf}(t, d) = \mathrm{tf}(t, d) \cdot \mathrm{idf}(t),$$

где

$$\mathrm{idf}(t) = \log \frac{N}{\mathrm{df}(t)}.$$

На практике используют сглаживание, чтобы избежать деления на 0 и экстремальны значений:

$$\boxed{\mathrm{idf}(t) = \log \frac{N+1}{\mathrm{df}(t)+1} + 1.}$$

(Это популярная “sklearn-подобная” форма.)

Варианты TF (примерные):

- сырая частота: $\mathrm{tf}(t, d) = \#(t \in d)$;
- нормированная: $\mathrm{tf}(t, d) = \frac{\#(t \in d)}{|d|}$;
- лог-нормировка: $\mathrm{tf}(t, d) = 1 + \log(\#(t \in d))$ при $\# > 0$.

Смысл TF-IDF: слово, которое встречается часто в конкретном документе, но редко в остальных документах, получает высокий вес и становится сильным признаком для различения документов.

Плюсы TF-IDF:

- остаётся простым и интерпретируемым;
- часто даёт заметный прирост качества относительно raw BoW на задачах классификации/по
- хорошо работает с линейными моделями (логистическая регрессия, SVM).

Минусы TF-IDF:

- по-прежнему игнорирует порядок слов и глубокий контекст;
- требует корпуса (коллекции документов) для оценки IDF;
- плохо обобщает семантику (синонимы не сближаются автоматически).

44.3 * BM25

BM25 (Okapi BM25) — ранжирующая функция из области информационного поиска (IR), которая является развитием TF-IDF и учитывает:

- насыщение TF (частые повторы слова дают убывающую отдачу);
- нормировку по длине документа (длинные документы не должны автоматически получать преимущество).

Пусть запрос q состоит из терминов $t \in q$, а документ — d. Тогда оценка BM25 обычно записывается как:

$$\boxed{\mathrm{BM25}(q, d) = \sum_{t \in q} \mathrm{idf}(t) \cdot \frac{\mathrm{tf}(t, d) \cdot (k_1 + 1)}{\mathrm{tf}(t, d) + k_1 \left(1 - b + b \cdot \frac{|d|}{\mathrm{avgdl}}\right)}.}$$

Где:

- $|d|$ — длина документа (число токенов);
- avgdl — средняя длина документов в коллекции;
- $k_1 > 0$ — параметр насыщения TF (обычно порядка 1–2);
- $b \in [0, 1]$ — сила нормировки по длине (часто около 0.75);
- $\mathrm{idf}(t)$ — "информативность" термина, например:

$$\mathrm{idf}(t) = \log \frac{N - \mathrm{df}(t) + 0.5}{\mathrm{df}(t) + 0.5}.$$

Интуиция формулы BM25:

- множитель TF:

$$\frac{\mathrm{tf}(t, d) \cdot (k_1 + 1)}{\mathrm{tf}(t, d) + k_1(\dots)}$$

растёт с TF, но **насыщается** (убывающая отдача от повторов);

- нормировка по длине через $|d|/\mathrm{avgdl}$ снижает "бонус" длинных документов;

- IDF повышает вес редких терминов.

Где используется BM25:

- классический поиск по текстовой коллекции (retrieval);
- как сильный baseline для retrieval в RAG-пайплайнах (часто как sparse retriever);
- как компонент гибридного поиска вместе с dense embeddings.

Сравнение: BoW/TF-IDF vs BM25.

- TF-IDF даёт линейный рост с TF (в простых вариантах) и часто слабее нормирует длину.
- BM25 обычно лучше ранжирует в IR-сценариях за счёт насыщения TF и длины документа.

45 Экспоненциальное сглаживание

45.1 Формула

Экспоненциальное сглаживание (simple exponential smoothing, SES; также EWMA — exponentially weighted moving average) используется для сглаживания временного ряда y_t и получения сглаженной оценки уровня s_t.

Рекуррентная формула (основная):

$$\boxed{s_t = \alpha\, y_t + (1-\alpha)\, s_{t-1}, \qquad 0 < \alpha \le 1,}$$

где:

- y_t — наблюдение ряда в момент t,
- s_t — сглаженное значение (оценка уровня) в момент t,
- α — коэффициент сглаживания (чем больше α, тем сильнее учитывается текущее наблюден

Прогноз на один шаг вперёд (типичная интерпретация SES):

$$\boxed{\hat{y}_{t+1|t} = s_t.}$$

Эквивалентная форма для прогноза: если обозначить текущий прогноз как $\hat{y}_{t|t-1}$, то

$$\boxed{\hat{y}_{t+1|t} = \alpha\, y_t + (1-\alpha)\, \hat{y}_{t|t-1}.}$$

Развёрнутая форма (показывает веса прошлых наблюдений): раскрывая рекурсию, получаем

$$s_t = \alpha y_t + \alpha(1-\alpha) y_{t-1} + \alpha(1-\alpha)^2 y_{t-2} + \cdots$$

или компактно:

$$\boxed{s_t = \alpha \sum_{k=0}^{\infty} (1-\alpha)^k\, y_{t-k} \quad \text{(при достаточно “давнем” старте).}}$$

При этом веса неотрицательны и (в пределе) суммируются к 1:

$$\alpha \sum_{k=0}^{\infty} (1-\alpha)^k = 1.$$

Инициализация. Нужно задать s_0 (например, $s_0 = y_0$ или среднее первых наблюдений). На коротких рядах выбор s_0 может заметно влиять на начало сглаженной кривой.

45.2 Мотивация названия

Название “экспоненциальное сглаживание” отражает два факта:

1) “Экспоненциальное” — из-за формы весов.
Во взвешенной сумме веса прошлых наблюдений убывают **экспоненциально** по мере удаления во времени:

$$w_k = \alpha(1-\alpha)^k, \qquad k = 0, 1, 2, \ldots$$

То есть наблюдение y_{t-k} влияет тем меньше, чем больше лаг k, и убывание происходит геометрически (экспоненциально по k).

2) “Сглаживание” — потому что подавляется шум.
Операция

$$s_t = \alpha y_t + (1-\alpha) s_{t-1}$$

является фильтрацией: случайные высокочастотные колебания (шум) в y_t усредняются, и s_t меняется более плавно. Параметр α задаёт компромисс:

- большое $\alpha \Rightarrow$ меньше сглаживание, быстрее реакция на изменения;
- малое $\alpha \Rightarrow$ сильнее сглаживание, но медленнее реакция на сдвиги уровня.

46 Индуктивные смещения (inductive bias) в Deep Learning

46.1 Регуляризация

Индуктивное смещение (inductive bias) — это набор предположений, которые модель или алгоритм обучения **встраивают заранее**, чтобы обобщение на новых данных стало возможным. В deep learning индуктивные смещения задаются архитектурой (например, свёртки ⇒ локальность и трансляционная инвариантность), а также **регуляризацией** и процедурами обучения.

Регуляризация — добавление ограничений/предпочтений к решению, чтобы уменьшить переобучение и улучшить обобщение. Формально, при обучении минимизируют регуляризованный риск:

$$\theta^\star \in \arg\min_\theta \Big[\underbrace{\frac{1}{n}\sum_{i=1}^{n}\ell\big(y_i, f(x_i;\theta)\big)}_{\text{эмпирический риск}} + \lambda \underbrace{\Omega(\theta)}_{\text{штраф/регуляризатор}}\Big].$$

Где $\Omega(\theta)$ задаёт индуктивное смещение: какие решения считаются "проще" или "предпочтительнее".

Типовые регуляризаторы и их смысл:

- **L_2-регуляризация (weight decay):** $\Omega(\theta) = \|\theta\|_2^2$. Предпочитает решения с меньшей нормой весов, подавляет излишне большие параметры и делает модель более "гладкой".
- **L_1-регуляризация:** $\Omega(\theta) = \|\theta\|_1$. Стимулирует разреженность (много нулевых параметров), но в DL применяется реже как основной инструмент.
- **Early stopping:** остановка обучения по валидации до того, как модель начнёт переобучаться.
- **Data-dependent regularization:** аугментации, dropout, label smoothing и др.

Индуктивное смещение через регуляризацию: модель вынуждают выбирать решения, которые лучше объясняют данные *простыми* закономерностями и хуже "запоминают" шум.

46.2 Batch normalization, layer normalization

Нормализации — это методы, которые стабилизируют распределения активаций и градиентов, ускоряют и упрощают обучение, а также выступают как **индуктивное смещение** (вносят специфическую структуру и некоторую регуляризацию).

Batch Normalization (BN). Для активаций x внутри слоя (по каналу) BN нормирует их по статистикам батча:

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}, \qquad y = \gamma\hat{x} + \beta,$$

где μ_B, σ_B^2 — среднее и дисперсия по батчу (и обычно по пространственным координатам для CNN), γ, β — обучаемые параметры масштаба и сдвига.

Смысл BN:

- уменьшает "дрейф распределений" активаций по слоям во время обучения;
- позволяет использовать более высокие learning rate;
- даёт некоторый регуляризующий эффект за счёт шумности статистик батча.

Ограничения BN:

- чувствительна к размеру батча (малый batch ⇒ плохие оценки μ_B, σ_B);
- в некоторых задачах (NLP/seq) и при переменной длине/малых батчах может работать хуже или требовать альтернатив.

Layer Normalization (LN). Нормирует **внутри одного объекта** по признаковому измерению для вектора активаций $x \in \mathbb{R}^d$ (одного примера):

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \varepsilon}}, \qquad y = \gamma\hat{x} + \beta,$$

где μ, σ^2 считаются по координатам признакового вектора этого же примера.

Смысл LN:

- не зависит от размера батча;
- хорошо подходит для трансформеров и последовательностей (где батч может быть малым или неоднородным).

Сравнение BN vs LN (коротко):

- BN нормирует по батчу ⇒ хороша для CNN и больших батчей, но зависит от batch size.
- LN нормирует по признакам внутри объекта ⇒ стабильна при малых батчах, популярна в трансформерах.
- В инференсе BN использует накопленные "running" статистики, LN — нет (она всегда считает статистики по объекту).

46.3 Dropout

Dropout — регуляризация, которая на шаге обучения случайно "выключает" часть нейронов (или активаций), уменьшая ко-адаптацию признаков и действуя как обучаемый ансамбль подмоделей.

Пусть h — активации слоя. Тогда при dropout с вероятностью выключения p (drop probability) вводят маску m:

$$m_i \sim \text{Bernoulli}(1-p), \qquad \tilde{h}_i = m_i \cdot h_i.$$

Inverted dropout (самая частая реализация): чтобы математическое ожидание активации сохранялось, используют масштабирование:

$$\boxed{\tilde{h}_i = \frac{m_i}{1-p}\, h_i.}$$

Тогда $\mathbb{E}[\tilde{h}_i] = h_i$.

46.3.1 Отличия режимов обучения и инференса:

- **Training (обучение):**
  - dropout **включён**: маска m семплируется случайно на каждом шаге;
  - нейроны случайно зануляются, создаётся шум ⇒ регуляризация;
  - при inverted dropout дополнительно есть масштабирование $\frac{1}{1-p}$ для сохранения матожид
- **Inference / Evaluation (инференс/оценка):**
  - dropout **выключён**: все нейроны активны, случайная маска не применяется;
  - если использовался inverted dropout, дополнительного масштабирования **не нужно**, потому что оно уже учитывалось в режиме обучения.

Интерпретация как ансамбля: обучение с dropout можно понимать как усреднение по большому числу "подсетей", а инференс — как приближённое усреднение этого ансамбля одной полной сетью.

46.4 Аугментации

Аугментации (data augmentations) — синтетические преобразования обучающих данных, которые сохраняют семантику целевой метки, но меняют наблюдение:

$$x \;\mapsto\; \tilde{x} = T(x),$$

где T — случайное преобразование из набора допустимых трансформаций.

Индуктивное смещение аугментаций: мы явно утверждаем, что целевая функция должна быть (примерно) **инвариантна** к таким преобразованиям (например, небольшие повороты/сдви изображения не меняют класс объекта).

Типовые аугментации:

- **CV (изображения):** random crop, flip, rotation, color jitter, blur/noise, cutout, mixup/cutmix random erasing и др.
- **NLP:** dropout токенов, замены синонимов, back-translation, шумы в тексте (зависят от задачи).
- **Аудио:** time shift, noise, pitch shift, specaugment и др.

46.4.1 Основные библиотеки (на практике чаще всего):

- **torchvision.transforms** — базовые аугментации для изображений в экосистеме PyTorch.
- **Albumentations** — популярная библиотека для CV, много быстрых и богатых трансформа удобно для пайплайнов.
- **imgaug** — классическая библиотека аугментаций (встречается, хотя сейчас реже, чем Albumentations).
- **kornia** — дифференцируемые аугментации на тензорах PyTorch (часть может выполняться на GPU).

46.4.2 * Test-time augmentations (TTA)

TTA — приём, когда аугментации применяются не только при обучении, но и **на инференсе**: делают несколько преобразованных версий одного объекта, прогоняют через модель и агрегирую предсказания.

Пусть $T_1, \dots, T_M$ — набор преобразований, тогда для классификации:

$$p^{(m)} = f(T_m(x)), \qquad \bar{p} = \frac{1}{M} \sum_{m=1}^{M} p^{(m)},$$

и итоговый класс выбирают по $\arg\max_k \bar{p}_k$.

Смысл TTA:

- уменьшает дисперсию предсказаний (похоже на ансамблирование);
- может повысить качество, если модель не идеально инвариантна к преобразованиям.

Цена TTA:

- увеличение времени инференса примерно в M раз;
- нужно аккуратно выбирать преобразования, чтобы они не разрушали метку (иначе качество упадёт).

47.1 Gradient descent, SGD, Momentum, RMSProp, Adam

Обозначения: параметры модели θ, функция потерь $L(\theta)$, градиент $g_t = \nabla_\theta L(\theta_t)$, шаг обучения (learning rate) $\eta > 0$. В deep learning на практике почти всегда используют **мини-батчи**.

Gradient Descent (GD, полный градиентный спуск).
На шаге t градиент считается по *всей* обучающей выборке:

$$\boxed{\theta_{t+1} = \theta_t - \eta\, g_t.}$$

Плюсы: стабильное направление. Минусы: очень дорого на больших данных $\Rightarrow$ редко применяют напрямую для DL.

Stochastic Gradient Descent (SGD).
Вместо полного градиента используют оценку по мини-батчу B_t:

$$g_t = \nabla_\theta\Big(\frac{1}{|B_t|}\sum_{(x,y)\in B_t} \ell(y, f(x;\theta_t))\Big), \qquad \boxed{\theta_{t+1} = \theta_t - \eta\, g_t.}$$

Плюсы: дешёвые шаги, хорошо масштабируется. Минусы: шумный градиент $\Rightarrow$ может требовать аккуратной настройки η.

Momentum (SGD + momentum).
Идея: усреднять направления градиента во времени, снижая шум и ускоряя движение вдоль согласованного направления. Вводят "скорость" (momentum buffer) v_t:

$$\boxed{v_t = \mu v_{t-1} + g_t, \qquad \theta_{t+1} = \theta_t - \eta\, v_t,} \qquad \mu \in [0, 1).$$

Интуиция: если градиенты несколько шагов указывают примерно в одну сторону, v_t растёт и ускоряет спуск. Если направление часто меняется (шум), усреднение уменьшает дрожание.

Nesterov momentum (NAG, вариант momentum).
Идея: градиент считается в "точке взгляда вперёд" (lookahead), что часто даёт более стабильную динамику:

$$\boxed{\tilde\theta_t = \theta_t - \eta\mu v_{t-1}, \qquad v_t = \mu v_{t-1} + \nabla L(\tilde\theta_t), \qquad \theta_{t+1} = \theta_t - \eta v_t.}$$

(Есть эквивалентные записи; ключевое отличие — градиент берётся после предварительного "сдвига".)

RMSProp.
Идея: адаптивно масштабировать шаг по координатам, деля градиент на корень из экспоненциал среднего квадратов градиентов. Вводят второй момент (накопитель) s_t:

$$\boxed{s_t = \rho s_{t-1} + (1-\rho)\, g_t^2, \qquad \theta_{t+1} = \theta_t - \eta\frac{g_t}{\sqrt{s_t} + \varepsilon},}$$

где операции g_t^2, деление и корень выполняются поэлементно, $\rho \in [0, 1)$ (часто близко к 1), $\varepsilon > 0$ для численной устойчивости.

Adam (Adaptive Moment Estimation).
Adam совмещает идеи momentum (1-й момент) и RMSProp (2-й момент). Вводят:

$$\boxed{m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2,}$$

далее делают bias-correction:

$$\boxed{\hat m_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat v_t = \frac{v_t}{1-\beta_2^t},}$$

и обновление:

$$\boxed{\theta_{t+1} = \theta_t - \eta\frac{\hat m_t}{\sqrt{\hat v_t} + \varepsilon}.}$$

Практический смысл: каждый параметр получает адаптивный эффективный шаг $\propto 1/(\sqrt{\hat v_t} + \varepsilon)$, а направление обновления сглаживается через $\hat m_t$.

47.2 Muon

Muon (MomentUm Orthogonalized by Newton–Schulz) — оптимизатор, ориентированный на **2D-параметры** (матрицы весов скрытых слоёв), который берёт обновление как у SGD-

momentum (часто с Nesterov), а затем применяет **постобработку матрицы обновления** через итерации Ньютона–Шульца, чтобы приблизительно **ортогонализовать** обновление.

Типовая постановка для одного матричного параметра $W \in \mathbb{R}^{n\times m}$:

1. Считаем градиент:
$$G_t = \nabla_W L(W_t).$$
2. Строим momentum-обновление (схематично, как в SGD+momentum):
$$M_t = \mu M_{t-1} + G_t.$$
3. Применяем ортогонализацию обновления (идея: сделать шаг "более изотропным" по сингуля направлениям):
$$\tilde{M}_t = \mathrm{NewtonSchulz}(M_t;\ T, \varepsilon),$$
где T — число итераций (обычно фиксированное небольшое), ε — стабилизатор.
4. Обновляем веса:
$$\boxed{W_{t+1} = W_t - \eta\, \tilde{M}_t.}$$

Смысл Muon как индуктивного ограничения на обновления. Вместо того чтобы просто идти по градиенту (или его моменту), Muon стремится, чтобы матричное обновление не было "почти низкоранговым" и не доминировалось несколькими направлениями (что часто наблюдают на матрицах весов), а было ближе к ортогональному/равномерному по направлениям шагу.

Практические замечания применения:

- Muon целенаправленно применяют к **2D-матрицам скрытых линейных слоёв**;
- скаляры/векторы (bias, LayerNorm-параметры) и часто **эмбеддинги/выходной слой** оптимизируют стандартным методом (например, AdamW), поскольку их динамика и структура отличаются;
- для свёрток можно свести 4D-тензор к 2D (flatten последних измерений), если это предусмот реализацией.

47.3 Расход памяти для каждого

Оценим **память оптимизатора** через число дополнительных тензоров состояния на параметр. Пусть всего в модели d скалярных параметров (суммарное число элементов во всех тензорах параметров). Пусть состояние оптимизатора хранится в той же точности, что и "мастер-веса" (часто FP32 даже при mixed precision).

Что обычно хранится в обучении:

- параметры: d чисел;
- градиенты: ещё d чисел (как минимум во время backward/step);
- состояние оптимизатора: зависит от метода (это и спрашивается).

Дополнительное состояние (persisted state) по методам:

Метод	Состояние	Что хранится дополнительно
GD / SGD	$0 \cdot d$	Нет дополнительных буферов (кроме градиентов, которые и так нужны для шага).
Momentum (SGD+Momentum / NAG)	$1 \cdot d$	Один буфер скорости v (momentum buffer) размера d.
RMSProp	$1 \cdot d$	Один буфер s (EMA квадратов градиента) размера d.
Adam	$2 \cdot d$	Два буфера: m (1-й момент) и v (2-й момент), суммарно $2d$.
Muon	$\approx 1 \cdot d$	По постоянной памяти близок к SGD-momentum: хранит momentum-буфер для 2D параметров. Ортогонализация Ньютона–Шульца требует *временных* промежуточных матриц при вычислении шага, но они не являются постоянным состоянием между итерациями.

Перевод в байты (если нужно численно). Если один скаляр хранится в FP32, то это 4 байта, и дополнительная память состояния:

$$\mathrm{Mem}_{\mathrm{state}} \approx 4 \cdot (\text{коэффициент}) \cdot d \text{ байт.}$$

Например, Adam: $\mathrm{Mem}_{\mathrm{state}} \approx 8d$ байт (два буфера по d элементов в FP32).

Важная ремарка для DL (часто спрашивают устно): помимо параметров/градиентов/сос оптимизатора существенную память занимает **хранение активаций** для backprop (зависит от глубины сети, размера батча и checkpointing), но это уже не "память оптимизатора".

48.1 Постановка задачи

Постановка задачи ранжирования (learning-to-rank).
Дано N запросов (queries, anchors) $Q = \{q_1, \dots, q_N\}$. Для каждого запроса q_i задан набор из K_i документов/объектов (items)

$$D_i = \{d_{i1}, \dots, d_{iK_i}\},$$

и соответствующие метки релевантности

$$L_i = \{l_{i1}, \dots, l_{iK_i}\}, \qquad l_{ij} \in \mathbb{R} \ \text{(часто дискретные уровни } 0, 1, \dots).$$

Запрос q_i описывается признаками (вектор размерности E), документ d_{ij} — признаками (вектор размерности M). Одна запись данных имеет вид (q_i, d_{ij}, l_{ij}).

Цель: построить модель, которая по паре (q, d) выдаёт **скор (score)**:

$$f(q_i, d_{ij}) = s_{ij} \in \mathbb{R},$$

такой, что сортировка документов по s_{ij} максимально согласуется с сортировкой по истинной релевантности l_{ij}:

$$\operatorname{argsort}\big(s_{i1}, \dots, s_{iK_i}\big) \ \approx \ \operatorname{argsort}\big(l_{i1}, \dots, l_{iK_i}\big).$$

Важно: как правило, **не требуется** точно предсказывать значения l_{ij}; требуется **восстановить порядок**.

48.1.1 Отличия от регрессии и классификации

1) Групповая структура данных.
В регрессии/классификации объекты обычно рассматриваются независимо:

$$(x_i, y_i) \text{ для каждого } i.$$

В ранжировании данные **сгруппированы по запросам**, и качество определяется **внутри каждой группы** D_i.

2) Оптимизируется порядок, а не абсолютное значение.

- Регрессия: важна численная близость $\hat{y}$ к y (MSE/MAE и т.д.).
- Классификация: важна правильность класса/вероятности.
- Ранжирование: важны **относительные сравнения** (кто выше), особенно в **топе выдачи**.

3) Метки могут быть градуированными.
Часто l_{ij} — уровни релевантности (например, 0–4), а не только бинарные классы.

4) Другие функции потерь и метрики.
Вместо стандартных loss'ов регрессии/классификации применяют:

- **pairwise** (RankNet/LambdaRank): сравнение пар документов внутри запроса;
- **listwise** (ListNet/ListMLE): оптимизация списков целиком;
- метрики качества ориентированы на позицию в выдаче (mAP, NDCG, MRR и т.д.).

5) Цена ошибки зависит от позиции.
Перепутать документы на местах 1 и 2 обычно существенно хуже, чем перепутать на местах 97 и 98 (для многих продуктов), поэтому метрики и потери часто **дисконтируют** вклад ошибок по позиции.

48.2 Оффлайн-метрики

48.2.1 mAP

Оффлайн-метрики считаются на фиксированном датасете и используются для сравнения моделей до запуска в продакшн.

Average Precision@K (AP@K).
Пусть для запроса q модель вернула упорядоченный список K документов. Обозначим $\mathrm{rel}(k) \in$

$\{0, 1\}$ — индикатор релевантности документа на позиции k (бинарная релевантность), а $P@k$ — precision на первых k позициях:

$$P@k = \frac{1}{k} \sum_{t=1}^{k} \text{rel}(t).$$

Тогда:

$$\boxed{AP@K = \frac{1}{\min(R, K)} \sum_{k=1}^{K} P@k \cdot \text{rel}(k),}$$

где $R = \sum_{k \geq 1} \text{rel}(k)$ — число релевантных документов (для данного запроса) в идеале/разметке, а нормировка $\min(R, K)$ означает "сколько релевантных мы максимум можем собрать в топ-K".

Mean Average Precision@K (mAP@K).
Среднее AP@K по всем N запросам:

$$\boxed{mAP@K = \frac{1}{N} \sum_{i=1}^{N} AP_i@K.}$$

Смысл mAP: награждает ранжирование, где релевантные документы встречаются как можно раньше, так как в сумму входят $P@k$ именно в тех позициях, где $\text{rel}(k) = 1$.

Ограничения (типичные):

- в базовой форме предполагает **бинарную релевантность** (релевант/нерелевант);
- вклад позиций задаётся косвенно через $P@k$ и может не отражать "правильную" цену ошибок для продукта;
- в реальных системах известны не все метки (часто есть только частичная обратная связь).

48.2.2 NDCG

DCG@K (Discounted Cumulative Gain).
Пусть l_k — уровень релевантности документа на позиции k в ранжированном списке (может быть $0, 1, 2, \dots$). Тогда:

$$\boxed{DCG@K = \sum_{k=1}^{K} \frac{2^{l_k} - 1}{\log_2(k + 1)}.}$$

- числитель $(2^{l_k} - 1)$ усиливает разницу между уровнями релевантности (graded relevance);
- знаменатель $\log_2(k+1)$ реализует **логарифмическую скидку по позиции**: документы выше в выдаче влияют сильнее.

Почему логарифм?
Логарифм задаёт убывающую по позиции важность: переход с 1-й на 2-ю позицию штрафуется сильнее, чем переход с 90-й на 91-ю, при этом убывание происходит плавно и монотонно.

IDCG@K (Ideal DCG).
Это значение DCG@K для **идеальной сортировки** документов по истинной релевантности (самое большое возможное DCG@K для данного набора документов).

NDCG@K (Normalized DCG). Нормировка делает метрику сопоставимой между запросами и ограничивает диапазон:

$$\boxed{NDCG@K = \frac{DCG@K}{IDCG@K} \in [0, 1].}$$

Смысл NDCG: напрямую оценивает качество порядка с акцентом на верх выдачи и поддержив градуированные метки.

48.2.3 MRR

Reciprocal Rank (RR).
Пусть rank_i — позиция **первого релевантного** документа в выдаче для запроса q_i. Тогда

$$RR_i = \frac{1}{\text{rank}_i}.$$

Если релевантных документов нет, часто полагают $RR_i = 0$.

Mean Reciprocal Rank (MRR).
Среднее RR по запросам:

$$\boxed{MRR = \frac{1}{N}\sum_{i=1}^{N}\frac{1}{\text{rank}_i}.}$$

Смысл MRR: особенно полезна, когда важно, чтобы **хотя бы один** релевантный документ был как можно выше (например, поиск ответа, навигационный запрос).

48.3 Онлайн-метрики

48.3.1 Как измеряются

Онлайн-метрики измеряются на реальных пользователях и отражают фактическое поведение в продукте. Как правило, валидируются через **A/B-тесты** (или много-рукавные эксперименты)

Типовой процесс измерения в A/B:

- пользователей (или запросы/сессии) случайно распределяют по вариантам (control vs treatment);
- для каждого пользователя фиксируют события и итоговые продуктовые метрики;
- сравнивают распределения метрик между группами (с доверительными интервалами/стат. тестами).

Событийные и продуктовые метрики.
Событийные метрики (click, view, add-to-cart, call, purchase) **опасны** интерпретационно: на них могут влиять роботы/спам/фрод, ошибки трекинга и изменение трафика.

Снижение влияния роботов (практический приём): агрегировать события **на уровне пользователя**, затем сравнивать средние по пользователям:

$$\bar{m}_{\text{split}} = \frac{1}{U}\sum_{u=1}^{U} m(u),$$

где $m(u)$ — метрика пользователя (например, число целевых действий пользователя за период), а не сырая сумма событий по всем логам.

Почему онлайн — "истина последней инстанции": онлайн-метрики учитывают динамику продукта (новые товары, новые пользователи, UX-эффекты) и реальную экономику/поведение, которые оффлайн-датасет может не отражать.

48.3.2 Дерево метрик, North star метрика

Дерево метрик (metrics tree).
Метрики продукта организуют в иерархию, отражающую вклад в конечную цель:

- в листьях — технические и локальные метрики (CTR, конверсия на шаге, время до действия, ошибки);
- выше — агрегированные продуктовые показатели (retention, GMV/выручка, доля удовлетво сессий);
- на вершине — метрика, которая задаёт направление всей оптимизации.

North Star metric (северная звезда).
Это **главная** метрика продукта (корневая вершина дерева), которая:

- максимально близка к долгосрочной ценности продукта;
- согласует команды (ML, продукт, маркетинг) вокруг единого критерия улучшений;
- помогает разрешать конфликты, когда рост одних метрик ведёт к падению других (наприме рост novelty может снижать точность/CTR).

Практический вывод: идеальных метрик почти не существует; важно смотреть на набор метрик и их причинно-следственные связи в дереве.

48.4 Функция потерь RankNet

48.4.1 Что она предсказывает

RankNet — **pairwise** подход к обучению ранжированию. Для одного запроса рассматривают пары документов (i, j), где известен порядок по релевантности.

Пусть модель выдаёт скоры s_i и s_j для документов d_i и d_j (внутри одного запроса). RankNet предсказывает **вероятность того, что i должен быть выше j**:

$$\boxed{P_{ij} = \mathbb{P}(i \succ j) = \sigma\big(\sigma_0(s_i - s_j)\big) = \frac{1}{1 + \exp\big(-\sigma_0(s_i - s_j)\big)},}$$

где $\sigma(\cdot)$ — сигмоида, $\sigma_0 > 0$ — масштаб (часто берут $\sigma_0 = 1$).

Определяют целевую "метку пары" через

$$S_{ij} = \begin{cases} 1, & \text{если } l_i > l_j, \\ -1, & \text{если } l_i < l_j, \\ 0, & \text{если } l_i = l_j. \end{cases}$$

Её часто переводят в $P^\star_{ij} \in \{0, \frac{1}{2}, 1\}$:

$$\boxed{P^\star_{ij} = \frac{1 + S_{ij}}{2}.}$$

Функция потерь RankNet (кросс-энтропия по парам):

$$\boxed{\mathcal{L}_{ij} = -P^\star_{ij} \log P_{ij} - (1 - P^\star_{ij}) \log(1 - P_{ij}).}$$

Эквивалентная компактная запись (логистическая pairwise loss):

$$\boxed{\mathcal{L}_{ij} = \log\big(1 + \exp(-\sigma_0 S_{ij}(s_i - s_j))\big).}$$

Смысл: минимизация этой потери стремится сделать $s_i > s_j$ для пар, где $l_i > l_j$, и наоборот.

48.4.2 * Lambda-gradients

Идея: вместо явного "loss → backward" можно рассматривать **градиенты по скорам** как "силы", толкающие документы вверх/вниз в выдаче. Эти величины часто называют **lambda**.

Для RankNet производная по разности скорингов даёт простой вид. Обозначим

$$\Delta s_{ij} = s_i - s_j.$$

Тогда производная потери по Δs_{ij}:

$$\frac{\partial \mathcal{L}_{ij}}{\partial \Delta s_{ij}} = \sigma_0 \, (P_{ij} - P^\star_{ij}).$$

Отсюда градиенты по отдельным скорам:

$$\boxed{\frac{\partial \mathcal{L}_{ij}}{\partial s_i} = \sigma_0 \, (P_{ij} - P^\star_{ij}), \qquad \frac{\partial \mathcal{L}_{ij}}{\partial s_j} = -\sigma_0 \, (P_{ij} - P^\star_{ij}).}$$

То есть они **симметричны по модулю и противоположны по знаку**:

$$\frac{\partial \mathcal{L}_{ij}}{\partial s_i} = -\frac{\partial \mathcal{L}_{ij}}{\partial s_j}.$$

Это свойство полезно вычислительно: вклад пары перераспределяется только между двумя документами.

Часто вводят

$$\boxed{\lambda_{ij} \stackrel{\text{def}}{=} \sigma_0 \, (P_{ij} - P^\star_{ij}),}$$

и затем суммарная “lambda” для документа i (внутри запроса) получается суммированием по всем парам с его участием:

$$\boxed{\lambda_i = \sum_{j:\,(i,j)\in\mathcal{P}} \lambda_{ij} - \sum_{j:\,(j,i)\in\mathcal{P}} \lambda_{ji},}$$

где $\mathcal{P}$ — множество рассматриваемых пар в группе запроса.

Связь с LambdaRank (интуиция): в LambdaRank берут базовую величину RankNet (по сути, λ_{ij}), но **взвешивают** её на величину изменения целевой ранжирующей метрики (например $\Delta NDCG$) при перестановке пары:

$$\lambda_{ij}^{\text{LambdaRank}} \;\propto\; \lambda_{ij}^{\text{RankNet}} \cdot |\Delta\text{NDCG}_{ij}|.$$

Тем самым алгоритм сильнее исправляет ошибки, которые критичны для метрики в топе.

48.4.3 Другие функции потерь (просто назвать)

- **LambdaRank**
- **LambdaMART**
- **YetiRank**
- **ListNet**
- **ListMLE**

49 Uplift modelling

49.1 Постановка задачи, отличие look-alike и response моделей

Uplift modelling (uplift / incremental response / treatment effect modelling) — класс задач, где цель не просто предсказать отклик Y, а оценить **прирост** отклика от воздействия (treatment) по сравнению с отсутствием воздействия (control).

Типовая постановка (бинарное воздействие). Для каждого объекта (пользователя) есть:

- признаки $X \in \mathbb{R}^d$;
- индикатор воздействия (treatment) $T \in \{0, 1\}$: $T = 1$ — применили действие (показали промо/письмо/скидку), $T = 0$ — не применили;
- наблюдаемый исход $Y \in \{0, 1\}$ (или $Y \in \mathbb{R}$): конверсия/покупка/клик/выручка и т.д.

В терминах потенциальных исходов:
$Y(1)$ — исход, если применили воздействие, $Y(0)$ — исход, если не применяли.
Наблюдаем только

$$Y = T\,Y(1) + (1 - T)\,Y(0),$$

то есть для каждого объекта доступен только один из двух потенциальных исходов ("fundamental problem of causal inference").

Индивидуальный эффект воздействия (ITE):

$$\tau(X) = Y(1) - Y(0).$$

Поскольку $Y(1)$ и $Y(0)$ одновременно не наблюдаются, на практике оценивают условный средний эффект (CATE):

$$\boxed{u(x) = \mathbb{E}[Y \mid X = x, T = 1] - \mathbb{E}[Y \mid X = x, T = 0].}$$

В задачах маркетинга/продукта $u(x)$ называют **uplift**.

Цель в прикладных задачах:

- **ранжировать** пользователей по $u(x)$ и воздействовать на тех, у кого ожидаемый **инкреме** эффект максимален;
- при ограниченном бюджете (cost of treatment) максимизировать **инкрементальную прибыль**:

$$\sum_{u \in \mathcal{S}} \big(\Delta\mathrm{Value}(u) - \mathrm{Cost}(u)\big),$$

где $\mathcal{S}$ — выбранная аудитория для воздействия.

Отличие от response моделей.
Response модель предсказывает вероятность отклика при воздействии (или вообще отклика):

$$p_{\mathrm{resp}}(x) \approx \mathbb{P}(Y = 1 \mid X = x, T = 1) \quad \text{или} \quad \mathbb{P}(Y = 1 \mid X = x).$$

Если таргетировать по $p_{\mathrm{resp}}(x)$, вы часто выбираете **"sure things"**: людей, которые бы совершили целевое действие *и без* воздействия. Это увеличивает наблюдаемый отклик, но может не давать **инкрементального** прироста.

Отличие от look-alike моделей.
Look-alike ищет аудиторию, похожую на уже известных ценных пользователей ("похожесть на портрет"):

$$p_{\mathrm{la}}(x) \approx \mathbb{P}(Z = 1 \mid X = x),$$

где Z — метка "похож на целевой сегмент" (например, похож на покупателей). Это не отвечает на вопрос причинного эффекта: **поможет ли** воздействие именно этому пользователю.

Интуиция сегментов в uplift (часто приводят в маркетинге):

- **Sure things:** $Y(1) = 1$, $Y(0) = 1$ — купят в любом случае.
- **Lost causes:** $Y(1) = 0$, $Y(0) = 0$ — не купят в любом случае.
- **Persuadables:** $Y(1) = 1$, $Y(0) = 0$ — именно они дают положительный uplift.

- **Do-not-disturb:** $Y(1) = 0, Y(0) = 1$ — воздействие ухудшает результат (негативный uplift).

Uplift-модель нацелена на **persuadables** и избегает **do-not-disturb**.

49.2 Two-model uplift technique

Two-model (двухмодельный, T-learner) подход: обучаем **две независимые модели** вероятности отклика:

$$p_1(x) \approx \mathbb{P}(Y = 1 \mid X = x, T = 1), \qquad p_0(x) \approx \mathbb{P}(Y = 1 \mid X = x, T = 0).$$

Затем оцениваем uplift как разность:

$$\boxed{\hat{u}(x) = \hat{p}_1(x) - \hat{p}_0(x).}$$

Алгоритм:

1. Разделить датасет на группы treatment ($T = 1$) и control ($T = 0$).
2. Обучить модель f_1 на подвыборке $T = 1$ для предсказания Y.
3. Обучить модель f_0 на подвыборке $T = 0$ для предсказания Y.
4. Для нового объекта с признаками x посчитать:
$$\hat{p}_1(x) = f_1(x), \qquad \hat{p}_0(x) = f_0(x), \qquad \hat{u}(x) = \hat{p}_1(x) - \hat{p}_0(x).$$

Достоинства:

- простая и прозрачная конструкция;
- можно использовать любые модели (логистическая регрессия, деревья, градиентный бустин нейросети);
- хорошо работает, когда эффект воздействия выражен и данных много.

Недостатки/риски:

- ошибки двух моделей **накапливаются** в разности: даже умеренные ошибки в p_1 и p_0 могут дать шумный $\hat{u}(x)$;
- если группы $T = 1$ и $T = 0$ различаются по распределению X (нерандомизированный эксперимент), модели будут экстраполировать вне своей области (selection bias).

Практическая ремарка. При отсутствии рандомизации требуется корректировка (например, через propensity score / IPW / doubly robust подходы), иначе разность $\hat{p}_1 - \hat{p}_0$ может быть смещённой оценкой эффекта.

49.3 Class transformation technique

Class transformation — семейство техник, которые превращают uplift-задачу в задачу (обычно классификации над **преобразованной** целевой переменной.

Вариант: бинарное преобразование "совпадения" (часто используется при рандомиз 50/50).

Определим новую бинарную метку Z как:

$$\boxed{Z = \begin{cases} 1, & (T = 1 \wedge Y = 1) \text{ или } (T = 0 \wedge Y = 0), \\ 0, & (T = 1 \wedge Y = 0) \text{ или } (T = 0 \wedge Y = 1). \end{cases}}$$

Эквивалентно: $Z = 1$ тогда и только тогда, когда $Y = T$ ("согласование" исхода и воздействия).

Далее обучаем обычный вероятностный классификатор:

$$\hat{p}_Z(x) = \mathbb{P}(Z = 1 \mid X = x).$$

Связь с uplift при сбалансированной рандомизации. Если эксперимент рандомизирован и $\mathbb{P}(T = 1) = \mathbb{P}(T = 0) = \frac{1}{2}$, то можно показать:

$$\mathbb{E}[Z \mid X = x] = \tfrac{1}{2}\big(1 + u(x)\big),$$

откуда:

$$\boxed{u(x) = 2\,\mathbb{E}[Z \mid X = x] - 1 \;\approx\; 2\,\hat{p}_Z(x) - 1.}$$

То есть uplift восстанавливается напрямую из вероятности преобразованного класса.

Плюсы class transformation:

- позволяет использовать любой бинарный классификатор "как есть";
- часто проще инженерно, чем обучение двух отдельных моделей и последующая разность.

Ограничения:

- простая формула $u(x) = 2\hat{p}_Z(x) - 1$ опирается на предположение о сбалансированной рандомизации (50/50);
- при несбалансированной раздаче ($\mathbb{P}(T = 1) \neq \frac{1}{2}$) обычно нужны модификации/взвешивания (propensity-aware преобразования).

49.4 Библиотеки для uplift modelling

Python (наиболее используемые):

- **scikit-uplift** (`sklift`): набор мета-алгоритмов (two-model, class transformation и др.) и метрик uplift.
- **causalml**: практические реализации uplift/ITE, метрики и визуализации uplift-кривых.
- **EconML**: модели для оценки CATE/ITE (мета-лернеры, DR-оценки, causal forests и т.д.).
- **DoWhy / PyWhy ecosystem**: причинно-следственный анализ, оценка эффектов и пайпла (часто используют вместе с EconML).

R (часто встречается в классическом uplift):

- **uplift** (классические uplift-модели и метрики),
- **grf** (generalized random forests, включая causal forest для CATE),
- **causalTree**.

Практическая заметка: в промышленности uplift часто строят и "вручную" на базе **градиентного бустинга** (CatBoost/LightGBM/XGBoost) через мета-алгоритмы (T-learner / S-learner / X-learner), а оценку эффекта и метрики считают отдельным кодом.

49.5 Примеры применения

- **Маркетинг и CRM:** выбор, кому отправить промо/письмо/пуш, чтобы максимизировать **инкрементальную** конверсию, а не просто общий CTR/CR (избежать "sure things" и "do-not-disturb").
- **Удержание (churn prevention):** кому предложить скидку/бонус, чтобы снизить отток *именно из-за* предложения, а не раздавать бонусы тем, кто бы остался и так.
- **Персонализация офферов и рекомендаций:** какой баннер/оффер/ценовой стимул показать пользователю, чтобы получить прирост целевой метрики (покупки, подписки, LTV).
- **Финтех и кредитование:** кому повысить лимит/предложить рассрочку/рефинансирован так, чтобы увеличить прибыль и не ухудшить риск (uplift по прибыли/дефолту/комбиниров utility-функции).
- **Здравоохранение / клинические решения:** персонализация лечения (оценка CATE), выбор терапии, где ожидаемый эффект для пациента максимален.
- **Продуктовые эксперименты:** таргетирование новых фич/интерфейсных изменений на сегменты, где эффект наиболее положительный, при контроле рисков.